discrete-math-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `discrete_math_ai999x.about()` for the same answer.

Every capability discrete-math-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import discrete_math_ai999x

result = discrete_math_ai999x.solve('gcd_lcm', numbers='84, 36')
result.show()
# gcd = 12, lcm = 252
```

By topic

topic	capabilities
Combinatorics / basic counting	Combinations C(n, k), Permutations P(n, k), Sequences with repetition (n^k), Combinations with repetition (multisets), Multinomial coefficient, Distinct arrangements of the letters of a word, Lattice paths in a grid
Combinatorics / binomial theorem	Coefficient in a binomial expansion
Combinatorics / counting under symmetry	Necklaces (rotations identified), Bracelets (rotations and reflections identified)
Combinatorics / distributions	Integer solutions of x1 + ... + xk = n (stars and bars)
Combinatorics / inclusion-exclusion	Derangements, Counting multiples by inclusion-exclusion
Combinatorics / integer partitions	Integer partitions p(n), Partitions into exactly k parts, Partitions into distinct parts
Combinatorics / permutations and cycles	Stirling numbers of the first kind
Combinatorics / pigeonhole principle	Pigeonhole principle: guaranteed load, Pigeonhole principle: objects needed for a guarantee
Combinatorics / recurrences	Fibonacci numbers, Lucas numbers
Combinatorics / recursive counting	Catalan numbers
Combinatorics / set partitions	Stirling numbers of the second kind, Bell numbers
Combinatorics / special numbers	Bernoulli numbers

topic	capabilities
Cryptography / classical ciphers	Caesar (shift) cipher, Affine cipher, Vigenère cipher
Cryptography / key exchange	Diffie-Hellman key exchange
Cryptography / public-key encryption	ElGamal encryption
Cryptography / RSA	RSA key from p, q and e, RSA encryption, RSA decryption, RSA decryption from p, q and e (CRT)
Graph theory / adjacency matrices	Number of walks of a given length
Graph theory / basic structure	Vertex degrees and degree sequence
Graph theory / cliques and independent sets	Clique number, Independence number
Graph theory / colouring	Bipartiteness test, Chromatic number of a small graph
Graph theory / connectivity	Connected components, Articulation points (cut vertices), Bridges (cut edges), Strongly connected components
Graph theory / cycles	Cycles in an undirected graph (cyclomatic number), Girth (shortest cycle length)
Graph theory / degree sequences	Graphical degree sequence
Graph theory / directed acyclic graphs	Topological sort, Cycle detection in a directed graph
Graph theory / distances	Diameter, radius and center
Graph theory / Euler and Hamilton	Eulerian circuits and paths, Hamiltonian cycle of a small graph
Graph theory / graph search	Breadth-first search order, Depth-first search order
Graph theory / subgraph counts	Number of triangles
Graph theory / trees	Number of spanning trees (Kirchhoff), Prüfer code of a labelled tree, Tree from a Prüfer code, Tree test
Logic / Boolean minimization	Minimal sum of products (DNF), Minimal product of sums (CNF), Minimal sum of products from minterms (with don't-cares)
Logic / propositional logic	Evaluate a propositional formula, Truth table and minterms, Logical equivalence of two formulas
Logic / satisfiability	Satisfiability of a propositional formula, Contradiction test
Logic / validity	Tautology test
Modular arithmetic / congruences	Linear congruence ax ≡ b (mod m), Chinese remainder theorem
Modular arithmetic / discrete logarithms	Discrete logarithm
Modular arithmetic / inverses	Modular multiplicative inverse
Modular arithmetic / orders and primitive roots	Multiplicative order, Smallest primitive root modulo a prime
Modular arithmetic / powers	Modular exponentiation

topic	capabilities
Modular arithmetic / residues	Remainder modulo m (Euclidean division)
Number representations / binary arithmetic	Two's complement representation
Number representations / positional systems	Base (radix) conversion
Number theory / arithmetic functions	Euler's totient function, Divisors, divisor count and divisor sum, Carmichael function, Möbius function
Number theory / continued fractions	Continued fraction of a rational number, Continued fraction of a square root
Number theory / Diophantine equations	Linear Diophantine equation ax + by = c, Pell's equation (fundamental solution)
Number theory / divisibility	Greatest common divisor and least common multiple, Extended Euclidean algorithm (Bezout coefficients)
Number theory / primes	Primality test, Prime factorization, Prime-counting function, The n-th prime, Next prime, Previous prime
Number theory / quadratic residues	Legendre symbol, Jacobi symbol, Square roots modulo a prime
Number theory / squares	Integer square root and perfect squares
Number theory / sums of squares	Sums of two squares

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)

field	meaning
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
gcd_lcm	Greatest common divisor and least common multiple	numbers	gcd, lcm
extended_gcd	Extended Euclidean algorithm (Bezout coefficients)	a, b	gcd, x, y, x_min, y_min
linear_diophantine	Linear Diophantine equation ax + by = c	a, b, c	solvable, x0, y0, x_step, y_step
linear_congruence	Linear congruence ax ≡ b (mod m)	a, b, m	smallest, count, step
is_prime	Primality test	n	is_prime, smallest_prime_factor
prime_factorization	Prime factorization	n	factorization, distinct_primes, prime_factors_with_multiplicity, largest_prime_factor, smallest_prime_factor, text
euler_phi	Euler's totient function	n	phi
divisor_functions	Divisors, divisor count and divisor sum	n	count, sum, proper_sum, classification, divisors
mod_reduce	Remainder modulo m (Euclidean division)	a, m	remainder, quotient
mod_pow	Modular exponentiation	base, exponent, modulus	result

id	operation	inputs	outputs
mod_inverse	Modular multiplicative inverse	a, m	inverse
crt	Chinese remainder theorem	residues, moduli	x, modulus
discrete_log	Discrete logarithm	base, target, modulus	x
legendre_symbol	Legendre symbol	a, p	symbol
jacobi_symbol	Jacobi symbol	a, n	symbol
sqrt_mod	Square roots modulo a prime	a, p	roots, count, smallest
multiplicative_order	Multiplicative order	a, n	order
primitive_root	Smallest primitive root modulo a prime	p	root
prime_counting	Prime-counting function	x	count
nth_prime	The n-th prime	n	prime
next_prime	Next prime	n	prime
previous_prime	Previous prime	n	prime
carmichael_lambda	Carmichael function	n	lambda
mobius	Möbius function	n	mu, squarefree
cf_rational	Continued fraction of a rational number	numerator, denominator	terms, length
cf_sqrt	Continued fraction of a square root	d	a0, period, period_length
pell_equation	Pell's equation (fundamental solution)	d	x, y
two_squares	Sums of two squares	n	representations, count, r2
integer_sqrt	Integer square root and perfect squares	n	isqrt, remainder, is_square
base_convert	Base (radix) conversion	digits, from_base, to_base	result, decimal
twos_complement	Two's complement representation	n, bits	bits_text, unsigned
combinations	Combinations C(n, k)	n, k	combinations
permutations	Permutations P(n, k)	n, k	permutations
sequences_with_repetition	Sequences with repetition (n^k)	n, k	sequences
multisets	Combinations with repetition (multisets)	n, k	multisets
multinomial	Multinomial coefficient	counts, total	coefficient, total
word_arrangements	Distinct arrangements of the letters of a word	word	arrangements, length, letter_counts
stirling2	Stirling numbers of the second kind	n, k	stirling2, surjections

id	operation	inputs	outputs
stirling1	Stirling numbers of the first kind	n, k	stirling1, signed
bell	Bell numbers	n	bell
catalan	Catalan numbers	n	catalan
fibonacci	Fibonacci numbers	n	fibonacci
lucas	Lucas numbers	n	lucas
derangements	Derangements	n	derangements, probability
integer_partitions	Integer partitions p(n)	n	partitions
partitions_k	Partitions into exactly k parts	n, k	partitions
partitions_distinct	Partitions into distinct parts	n	partitions
stars_and_bars	Integer solutions of x1 + ... + xk = n (stars and bars)	total, variables, lower, upper	solutions
inclusion_exclusion	Counting multiples by inclusion-exclusion	n, divisors	divisible_by_any, divisible_by_none
pigeonhole_max	Pigeonhole principle: guaranteed load	items, boxes	guaranteed
pigeonhole_needed	Pigeonhole principle: objects needed for a guarantee	boxes, at_least	needed
necklaces	Necklaces (rotations identified)	beads, colours	necklaces, bracelets
bracelets	Bracelets (rotations and reflections identified)	beads, colours	bracelets, necklaces
binomial_term	Coefficient in a binomial expansion	a, b, n, k	coefficient
lattice_paths	Lattice paths in a grid	right, up	paths
bernoulli	Bernoulli numbers	n	value, numerator, denominator
graph_degrees	Vertex degrees and degree sequence	edges	vertices, edges, degrees, degree_sequence, max_degree, min_degree
connected_components	Connected components	edges	count, sizes, is_connected, components
bfs_order	Breadth-first search order	edges, source, directed	order, reached, distances
dfs_order	Depth-first search order	edges, source, directed	order, reached
graph_diameter	Diameter, radius and center	edges	diameter, radius, center, periphery
topological_sort	Topological sort	edges	order, vertices
cycle_rank	Cycles in an undirected graph (cyclomatic number)	edges	cyclomatic_number, has_cycle
directed_cycle	Cycle detection in a directed graph	edges	has_cycle, is_dag

id	operation	inputs	outputs
bipartite	Bipartiteness test	edges	is_bipartite
eulerian	Eulerian circuits and paths	edges	odd_vertices, connected, has_circuit, has_path
chromatic_number	Chromatic number of a small graph	edges	chromatic_number, colouring
spanning_tree_count	Number of spanning trees (Kirchhoff)	edges	spanning_trees
articulation_points	Articulation points (cut vertices)	edges	count, points
bridges	Bridges (cut edges)	edges	count, bridges
strongly_connected	Strongly connected components	edges	count, sizes, components
hamiltonian_cycle	Hamiltonian cycle of a small graph	edges	exists, cycle
clique_number	Clique number	edges	clique_number, clique
independence_number	Independence number	edges	independence_number, independent_set
girth	Girth (shortest cycle length)	edges	girth
prufer_encode	Prüfer code of a labelled tree	edges	sequence, vertices
prufer_decode	Tree from a Prüfer code	sequence	edges, vertices
graphical_sequence	Graphical degree sequence	degrees	is_graphical
walks_count	Number of walks of a given length	edges, source, target, length, directed	walks
graph_triangles	Number of triangles	edges	triangles
is_tree	Tree test	edges	is_tree, vertices, edges, connected
logic_evaluate	Evaluate a propositional formula	formula, assignment	value
truth_table	Truth table and minterms	formula	variables, values, true_rows, minterms
logic_satisfiable	Satisfiability of a propositional formula	formula	satisfiable, models, first_model
logic_tautology	Tautology test	formula	tautology, false_rows, counterexample
logic_contradiction	Contradiction test	formula	contradiction, true_rows, witness
logic_equivalence	Logical equivalence of two formulas	formula1, formula2	equivalent, differing_rows, counterexample
minimal_dnf	Minimal sum of products (DNF)	formula	expression, terms, literals
minimal_cnf	Minimal product of sums (CNF)	formula	expression, clauses, literals
minimal_sop_minterms	Minimal sum of products from minterms (with don't-cares)	minterms, variables, dont_cares	expression, terms, literals

id	operation	inputs	outputs
rsa_keygen	RSA key from p, q and e	p, q, e	n, phi, d
rsa_encrypt	RSA encryption	m, e, n	c
rsa_decrypt	RSA decryption	c, d, n	m
rsa_decrypt_pq	RSA decryption from p, q and e (CRT)	c, p, q, e	m, d, n
diffie_hellman	Diffie-Hellman key exchange	p, g, a, b	A, B, shared
elgamal_encrypt	ElGamal encryption	p, g, x, k, m	h, c1, c2
caesar_cipher	Caesar (shift) cipher	message, shift, mode	result
affine_cipher	Affine cipher	message, a, b, mode	result
vigenere_cipher	Vigenère cipher	message, key, mode	result

Greatest common divisor and least common multiple — `gcd_lcm`

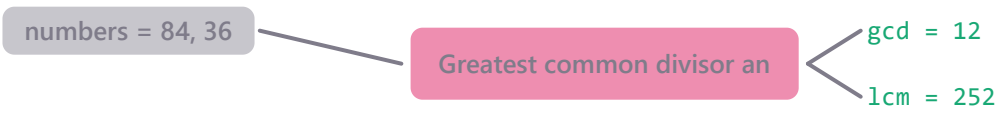
The gcd and lcm of two or more positive integers (Euclid's algorithm).

$$\gcd(a, b) = \gcd(b,\ a \bmod b), \qquad \operatorname{lcm}(a, b) = \frac{ab}{\gcd(a, b)}$$

input	meaning	condition
`numbers`	the positive integers (two or more)	

output	meaning
`gcd`	greatest common divisor
`lcm`	least common multiple

- **Method:** Euclid's algorithm gcd(a, b) = gcd(b, a mod b), folded over the list; lcm(a, b) = a / gcd(a, b) * b
- **Assumes:** two or more whole numbers, each at least 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Greatest common divisor and least common multiple as inputs and results: stated numbers = 84, 36; returned by discrete-math-ai999x: gcd = 12, lcm = 252.

```
discrete_math_ai999x.solve('gcd_lcm', numbers='84, 36')
# -> gcd = 12, lcm = 252   (SOLVED)
```

```
discrete_math_ai999x.solve('gcd_lcm', numbers='4, 6, 10')
# -> gcd = 2, lcm = 60   (SOLVED)
```

```
discrete_math_ai999x.solve('gcd_lcm', numbers=[17, 5])
# -> gcd = 1, lcm = 85   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the gcd of 84 and 36?	SOLVED	gcd = 12, lcm = 252
Find the greatest common divisor of 252 and 105.	SOLVED	gcd = 21, lcm = 1260
Find the LCM of 4, 6 and 10.	SOLVED	gcd = 2, lcm = 60
What is the gcd of 0 and 12?	INVALID_INPUT	Greatest common divisor and least common multiple: numbers must be at least 1.
Find the least common multiple of the numbers.	NEEDS_INPUT	gcd_lcm needs numbers (the positive integers (two or more)). Example: solve('gcd_lcm', numbers='84, 36').

Extended Euclidean algorithm (Bezout coefficients) — `extended_gcd`

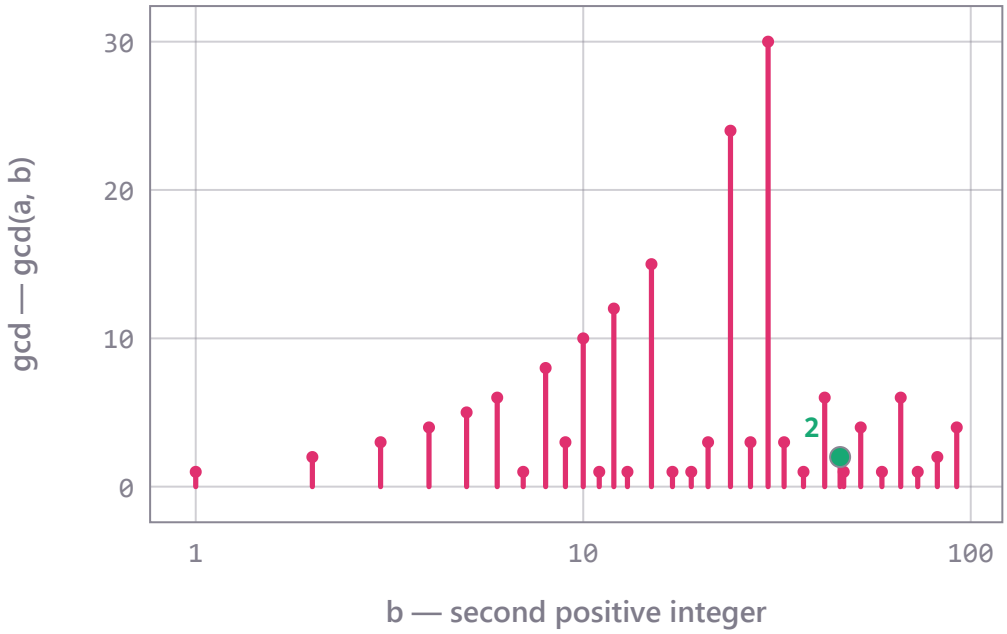
gcd(a, b) and integers x, y with a*x + b*y = gcd(a, b).

$$ax + by = \gcd(a, b)$$

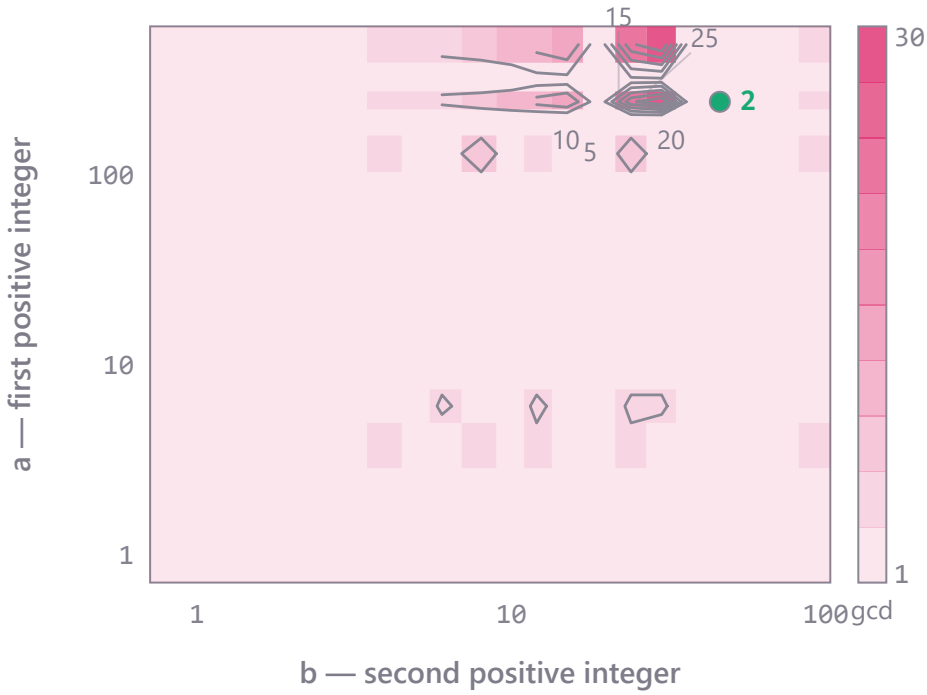
input	meaning	condition
`a`	first positive integer	
`b`	second positive integer	

output	meaning
`gcd`	gcd(a, b)
`x`	coefficient of a from the extended Euclidean algorithm
`y`	coefficient of b from the extended Euclidean algorithm
`x_min`	the coefficient of a with 0 <= x_min < b/gcd
`y_min`	the matching coefficient of b

- **Method:** extended Euclidean algorithm (back-substitution kept as running coefficients)
- **Assumes:** a, b whole numbers >= 1; (x, y) are the coefficients the standard extended Euclidean algorithm produces; (x_min, y_min) is the solution with the smallest non-negative x
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gcd (gcd(a, b)) against b (second positive integer) from 1 to 92 on a logarithmic axis. Other inputs as in the worked example: a = 240. 31 points computed by discrete-math-ai999x; marked: the worked example, b = 46, gcd = 2.



gcd (gcd(a, b)) over b (second positive integer) from 1 to 92 and a (first positive integer) from 1 to 480 (logarithmic axes): each of the 19 × 15 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 30); iso-lines at gcd = 5, 10, 15, 20, 25. Marked: the worked example, b = 46, a = 240, gcd = 2.

```
discrete_math_ai999x.solve('extended_gcd', a=240, b=46)
# -> gcd = 2, x = -9, y = 47, x_min = 14, y_min = -73   (SOLVED)
```

```
discrete_math_ai999x.solve('extended_gcd', a=3, b=7)
# -> gcd = 1, x = -2, y = 1, x_min = 5, y_min = -2   (SOLVED)
```

```
discrete_math_ai999x.solve('extended_gcd', a=12, b=18)
# -> gcd = 6, x = -1, y = 1, x_min = 2, y_min = -1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Bezout coefficients of 240 and 46.	SOLVED	gcd = 2, x = -9, y = 47, x_min = 14, y_min = -73
Apply the extended Euclidean algorithm to 3 and 7.	SOLVED	gcd = 1, x = -2, y = 1, x_min = 5, y_min = -2
Apply the extended Euclidean algorithm to -3 and 7.	INVALID_INPUT	a: must be at least 1, got -3.

Linear Diophantine equation ax + by = c — `linear_diophantine`

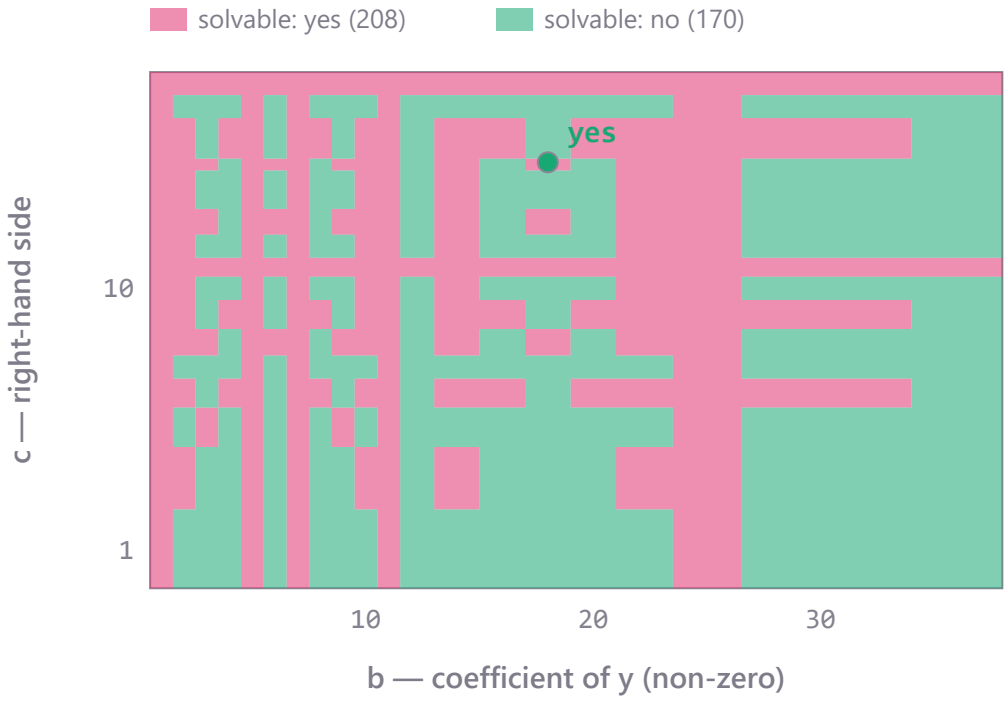
Whether $ax + by = c$ has integer solutions, and the general solution $x = x_0 + (b/g)$ t, $y = y_0 - (a/g)$ t.

$$ax + by = c \text{ solvable} \iff \gcd(a, b) \mid c; \quad x = x_0 + \tfrac{b}{g}t,\ y = y_0 - \tfrac{a}{g}t$$

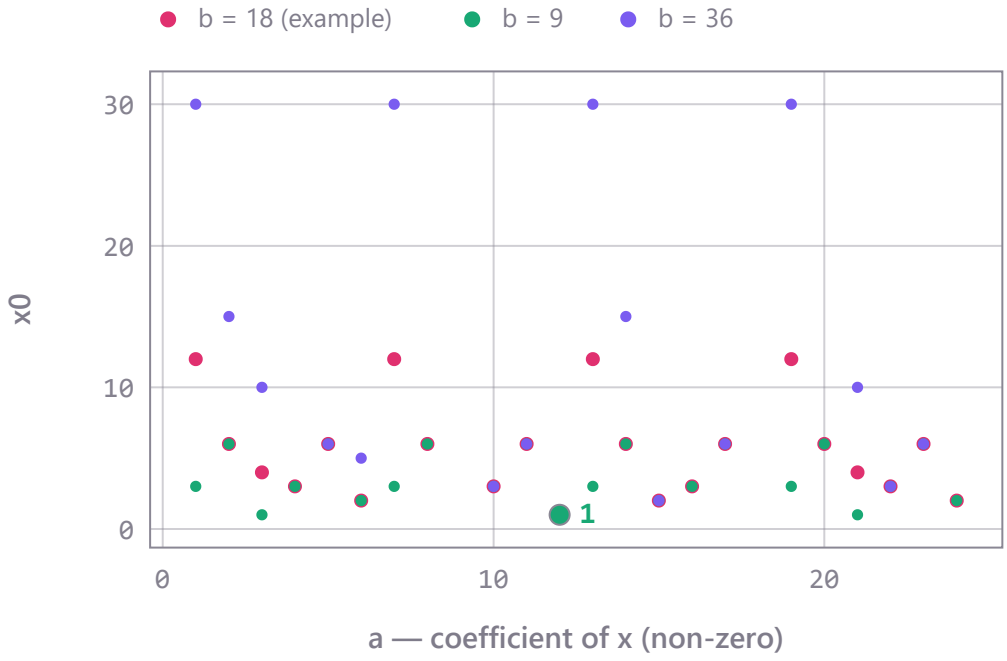
input	meaning	condition
`a`	coefficient of x (non-zero)	
`b`	coefficient of y (non-zero)	
`c`	right-hand side	

output	meaning
`solvable`	true when gcd(a, b) divides c
`x0`	the solution with the smallest non-negative x
`y0`	the matching y
`x_step`	b/g: the step of x between solutions
`y_step`	-a/g: the step of y between solutions

- **Method:** extended Euclidean algorithm, scaled by c/g, reduced to the smallest non-negative x
- **Assumes:** a, b non-zero integers; solutions exist exactly when gcd(a, b) divides c
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when gcd(a, b) divides c (solvable) over b (coefficient of y (non-zero)) from 1 to 36 and c (right-hand side) from 1 to 60 (logarithmic vertical axis): each of the 21 × 18 cells is one run of discrete-math-ai999x, coloured by the verdict it returned — yes: 208 cells, no: 170 cells. Other inputs as in the worked example: a = 12. Marked: the worked example, b = 18, c = 30 → yes.



x0 (the solution with the smallest non-negative x) against a (coefficient of x (non-zero)) from 1 to 24. One curve for each value of b (coefficient of y (non-zero)): 18, 9, 36. Other inputs as in the worked example: c = 30. 22 points per curve computed by discrete-math-ai999x; marked: the worked example, a = 12, x0 = 1.

```
discrete_math_ai999x.solve('linear_diophantine', a=12, b=18, c=30)
# -> solvable = True, x0 = 1, y0 = 1, x_step = 3, y_step = -2   (SOLVED)
```

```
discrete_math_ai999x.solve('linear_diophantine', a=3, b=-5, c=7)
# -> solvable = True, x0 = 4, y0 = 1, x_step = -5, y_step = -3   (SOLVED)
```

```
discrete_math_ai999x.solve('linear_diophantine', a=6, b=9, c=4)
# -> solvable = False, x0 = none, y0 = none, x_step = 3, y_step = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find all integer solutions of 12x + 18y = 30.	SOLVED	solvable = True, x0 = 1, y0 = 1, x_step = 3, y_step = -2
Solve the Diophantine equation 3x - 5y = 7.	SOLVED	solvable = True, x0 = 4, y0 = 1, x_step = -5, y_step = -3
Does 6x + 9y = 4 have integer solutions?	SOLVED	solvable = False, x0 = none, y0 = none, x_step = 3, y_step = -2

Linear congruence ax ≡ b (mod m) — `linear_congruence`

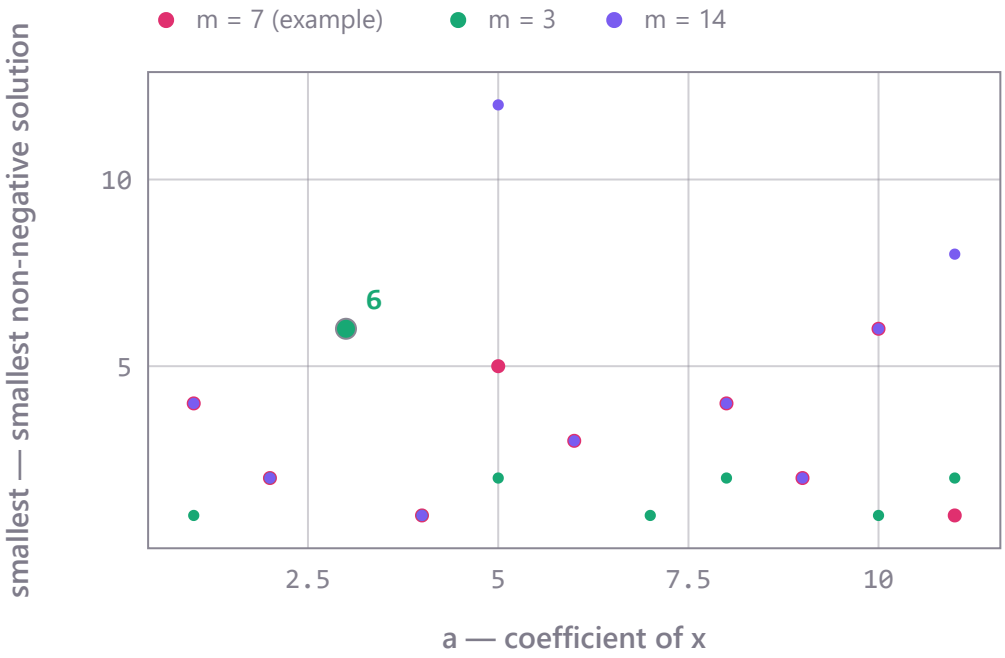
All solutions of $ax \equiv b \pmod{m}$: how many there are modulo m, the smallest, and their spacing.

$$ax \equiv b \pmod{m} \iff x \equiv \tfrac{b}{g}\left(\tfrac{a}{g}\right)^{-1} \pmod{\tfrac{m}{g}},\ g = \gcd(a,m) \mid b$$

input	meaning	condition
`a`	coefficient of x	
`b`	right-hand side	
`m`	modulus (at least 1)	

output	meaning
`smallest`	smallest non-negative solution (none when there is no solution)
`count`	number of solutions modulo m (gcd(a, m) when it divides b, else 0)
`step`	m / gcd(a, m): solutions are smallest + k*step

- **Method:** divide by g = gcd(a, m), then multiply by the inverse of a/g modulo m/g
- **Assumes:** $m \geq 1$; solvable exactly when gcd(a, m) divides b
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



smallest (smallest non-negative solution (none when there is no solution)) against a (coefficient of x) from 1 to 11. One curve for each value of m (modulus (at least 1)): 7, 3, 14. Other inputs as in the worked example: b = 4. 10 points per curve computed by discrete-math-ai999x; marked: the worked example, a = 3, smallest = 6.

```
discrete_math_ai999x.solve('linear_congruence', a=3, b=4, m=7)
# -> smallest = 6, count = 1, step = 7   (SOLVED)
```

```
discrete_math_ai999x.solve('linear_congruence', a=6, b=4, m=10)
# -> smallest = 4, count = 2, step = 5   (SOLVED)
```

```
discrete_math_ai999x.solve('linear_congruence', a=4, b=3, m=6)
# -> smallest = none, count = 0, step = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve 3x ≡ 4 (mod 7).	SOLVED	smallest = 6, count = 1, step = 7
Solve 6x ≡ 4 (mod 10).	SOLVED	smallest = 4, count = 2, step = 5
Solve 4x = 3 mod 6.	SOLVED	smallest = none, count = 0, step = none
Solve 3x ≡ 4 (mod 0).	INVALID_INPUT	m: must be at least 1, got 0.

Primality test — `is_prime`

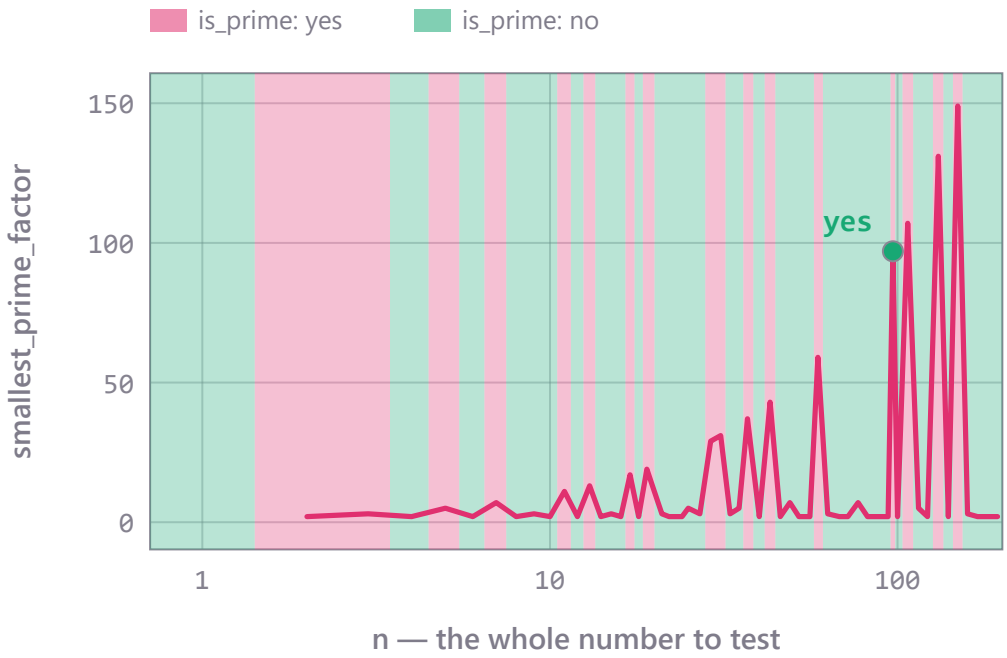
Whether n is prime (deterministic Miller-Rabin, exact for every n below 2^64), with its smallest prime factor.

$$a^d \equiv 1 \text{ or } a^{2^r d} \equiv -1 \pmod{n} \text{ for every base } a$$

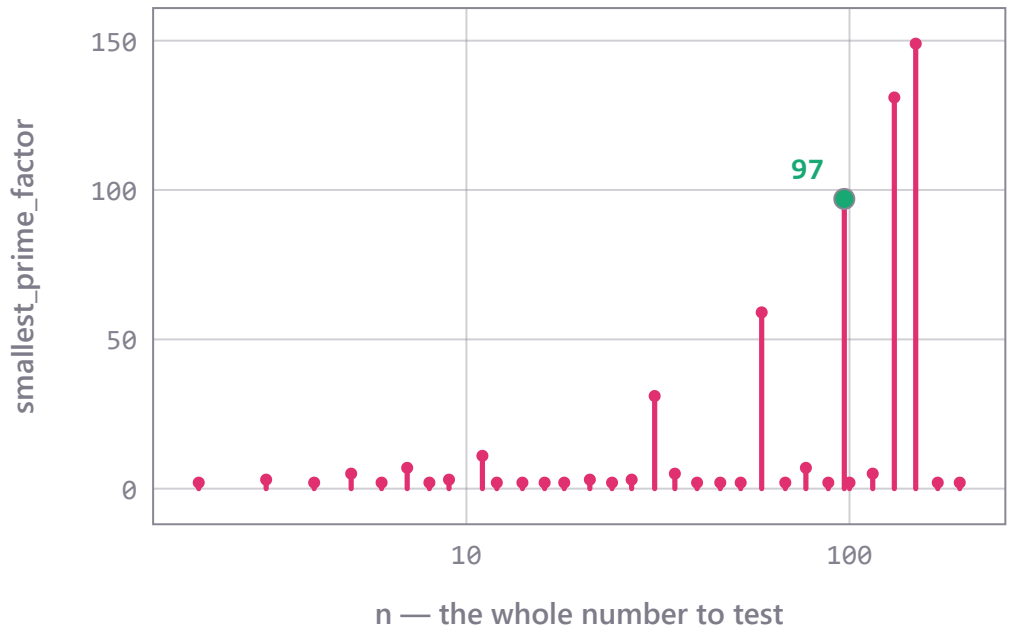
input	meaning	condition
`n`	the whole number to test	

output	meaning
`is_prime`	true when n is prime
`smallest_prime_factor`	smallest prime dividing n (n itself when prime; none for n < 2)

- **Method:** trial division by primes up to 37, then Miller-Rabin to the bases 2, 325, 9375, 28178, 450775, 9780504, 1795265022 (deterministic below 2^64); the smallest factor by Pollard's rho
- **Assumes:** $0 \le n < 2^{64}$ (questions accept n up to 10^15)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when n is prime (is_prime) along n (the whole number to test) from 1 to 194 (logarithmic axis): 55 runs of discrete-math-ai999x, each band coloured by the verdict returned (yes, no); it changes near n = 1.41421, 3.4641, 4.47214, 5.47723; the line is smallest_prime_factor (smallest prime dividing n (n itself when prime; none for n < 2)). Marked: the worked example, n = 97 → yes.



smallest_prime_factor (smallest prime dividing n (n itself when prime; none for n < 2)) against n (the whole number to test) from 1 to 194 on a logarithmic axis. 32 points computed by discrete-math-ai999x; marked: the worked example, n = 97, smallest_prime_factor = 97.

```
discrete_math_ai999x.solve('is_prime', n=97)
# -> is_prime = True, smallest_prime_factor = 97   (SOLVED)
```

```
discrete_math_ai999x.solve('is_prime', n=561)
# -> is_prime = False, smallest_prime_factor = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('is_prime', n=1)
# -> is_prime = False, smallest_prime_factor = none   (SOLVED)
```

```
discrete_math_ai999x.solve('is_prime', n=999999999989)
# -> is_prime = True, smallest_prime_factor = 1e+12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is 97 prime?	SOLVED	is_prime = True, smallest_prime_factor = 97
Is 1000003 a prime number?	SOLVED	is_prime = True, smallest_prime_factor = 1e+06
Test whether 561 is prime.	SOLVED	is_prime = False, smallest_prime_factor = 3
Is 91 composite?	SOLVED	is_prime = False, smallest_prime_factor = 7
Is -7 prime?	INVALID_INPUT	n: must be at least 0, got -7.

Prime factorization — `prime_factorization`

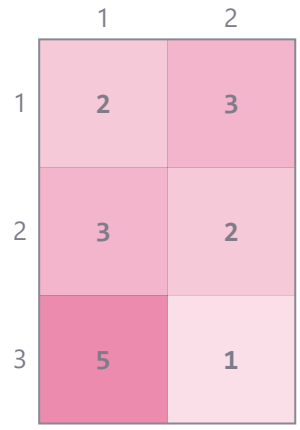
The prime factorization of n, its largest and smallest prime factors and the number of prime factors.

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

input	meaning	condition
`n`	the whole number to factor (at least 2)	

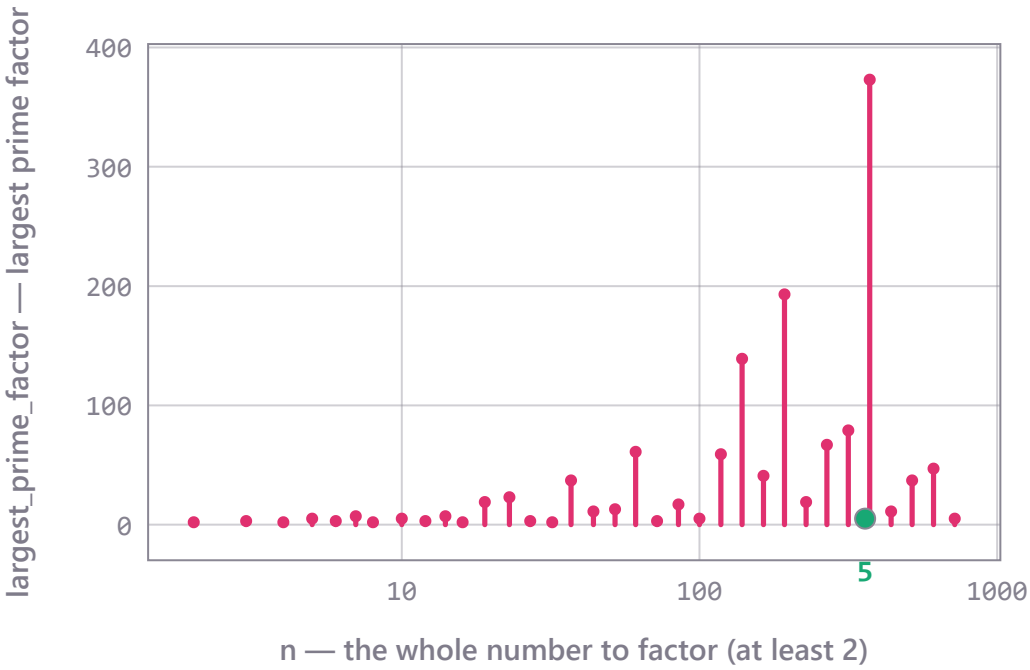
output	meaning
`factorization`	list of [prime, exponent] pairs, primes ascending
`distinct_primes`	number of distinct prime factors
`prime_factors_with_multiplicity`	number of prime factors counted with multiplicity
`largest_prime_factor`	largest prime factor
`smallest_prime_factor`	smallest prime factor
`text`	the factorization written as p^e × ...

- **Method:** trial division by small primes, then Pollard's rho with deterministic Miller-Rabin for the cofactors
- **Assumes:** n a whole number, $2 \le n \le 10^{15}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



factorization (returned)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): factorization (returned). Returned with them by discrete-math-ai999x: distinct_primes = 3, prime_factors_with_multiplicity = 6, largest_prime_factor = 5, smallest_prime_factor = 2. Other inputs: n = 360.



largest_prime_factor (largest prime factor) against n (the whole number to factor (at least 2)) from 1 to 720 on a logarithmic axis. 35 points computed by discrete-math-ai999x; marked: the worked example, n = 360, largest_prime_factor = 5.

```
discrete_math_ai999x.solve('prime_factorization', n=360)
# -> factorization = [[2, 3], [3, 2], [5, 1]], distinct_primes = 3, prime_factors_with_multiplicity
= 6, largest_prime_factor = 5, smallest_prime_factor = 2, text = 2^3 × 3^2 × 5   (SOLVED)
```

```
discrete_math_ai999x.solve('prime_factorization', n=13195)
# -> factorization = [[5, 1], [7, 1], [13, 1], [29, 1]], distinct_primes = 4,
prime_factors_with_multiplicity = 4, largest_prime_factor = 29, smallest_prime_factor = 5, text = 5
× 7 × 13 × 29   (SOLVED)
```

```
discrete_math_ai999x.solve('prime_factorization', n=600851475143)
# -> factorization = [[71, 1], [839, 1], [1471, 1], [6857, 1]], distinct_primes = 4,
prime_factors_with_multiplicity = 4, largest_prime_factor = 6857, smallest_prime_factor = 71, text
= 71 × 839 × 1471 × 6857   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the prime factorization of 360.	SOLVED	factorization = [[2, 3], [3, 2], [5, 1]], distinct_primes = 3, prime_factors_with_multiplicity = 6, largest_prime_factor = 5, smallest_prime_factor = 2, text = 2^3 × 3^2 × 5
Factorize 1001.	SOLVED	factorization = [[7, 1], [11, 1], [13, 1]], distinct_primes = 3, prime_factors_with_multiplicity = 3, largest_prime_factor = 13, smallest_prime_factor = 7, text = 7 × 11 × 13
What is the largest prime factor of 13195?	SOLVED	factorization = [[5, 1], [7, 1], [13, 1], [29, 1]], distinct_primes = 4, prime_factors_with_multiplicity = 4, largest_prime_factor = 29, smallest_prime_factor = 5, text = 5 × 7 × 13 × 29
Find the prime factorization of 1.	INVALID_INPUT	n: must be at least 2, got 1.

Euler's totient function — `euler_phi`

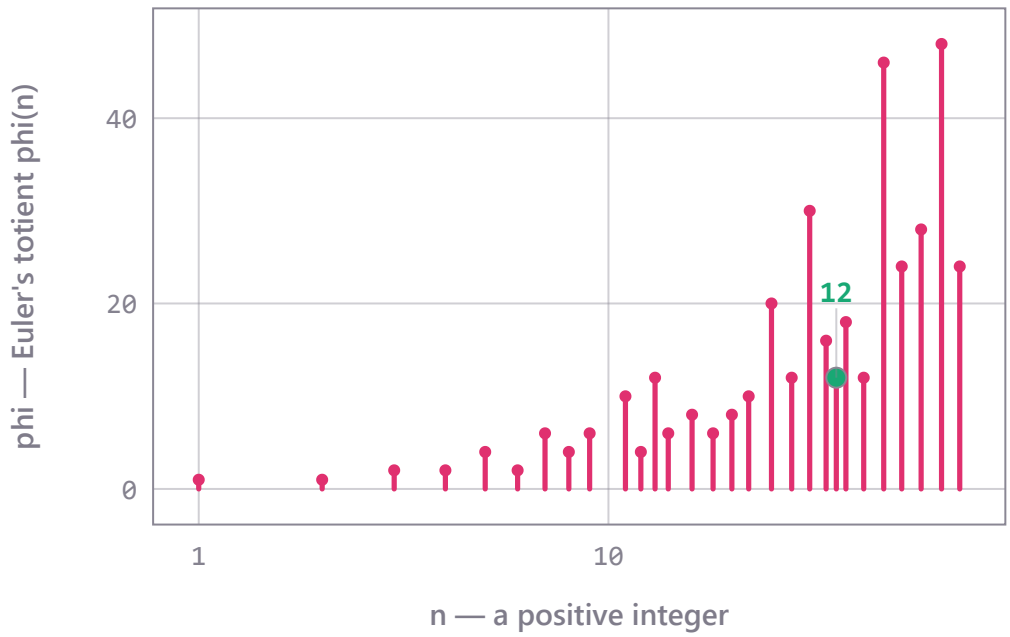
phi(n): how many integers in 1..n are coprime to n.

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$$

input	meaning	condition
`n`	a positive integer	

output	meaning
`phi`	Euler's totient phi(n)

- **Method:** product formula over the distinct prime factors of n
- **Assumes:** n a whole number, $1 \le n \le 10^{15}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



phi (Euler's totient phi(n)) against n (a positive integer) from 1 to 72 on a logarithmic axis. 29 points computed by discrete-math-ai999x; marked: the worked example, n = 36, phi = 12.

```
discrete_math_ai999x.solve('euler_phi', n=36)
# -> phi = 12   (SOLVED)
```

```
discrete_math_ai999x.solve('euler_phi', n=97)
# -> phi = 96   (SOLVED)
```

```
discrete_math_ai999x.solve('euler_phi', n=1)
# -> phi = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('euler_phi', n=1000000)
# -> phi = 400000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is Euler's totient of 36?	SOLVED	phi = 12
Compute phi(100).	SOLVED	phi = 40
How many positive integers less than 36 are coprime to 36?	SOLVED	phi = 12
Compute phi(0).	INVALID_INPUT	n: must be at least 1, got 0.

Divisors, divisor count and divisor sum — `divisor_functions`

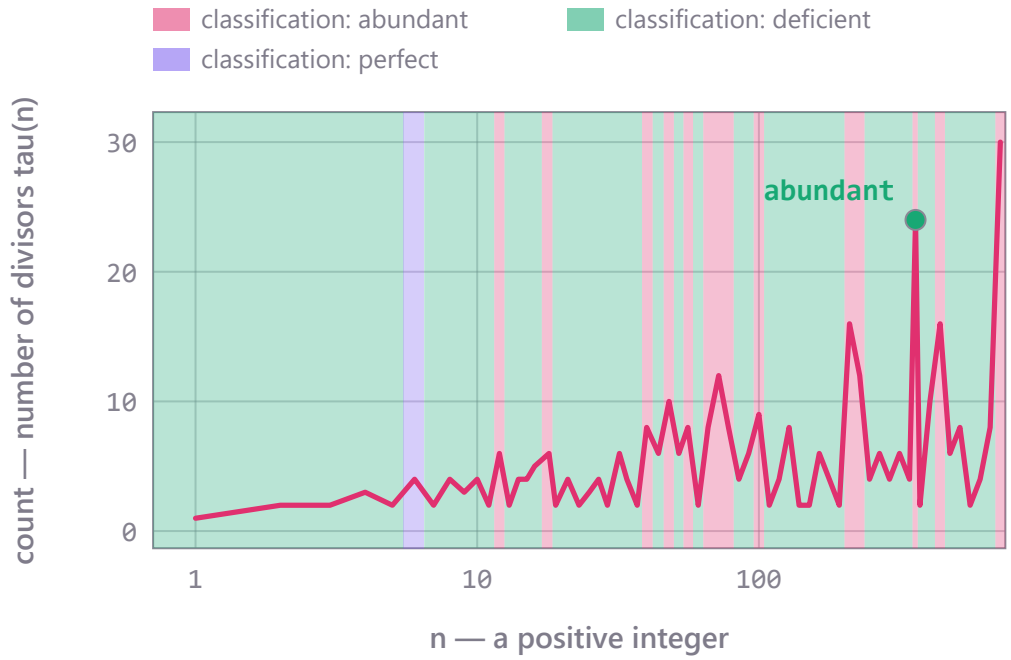
All positive divisors of n, their number tau(n), their sum sigma(n), and whether n is perfect, abundant or deficient.

$$\tau(n) = \prod (e_i + 1), \qquad \sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

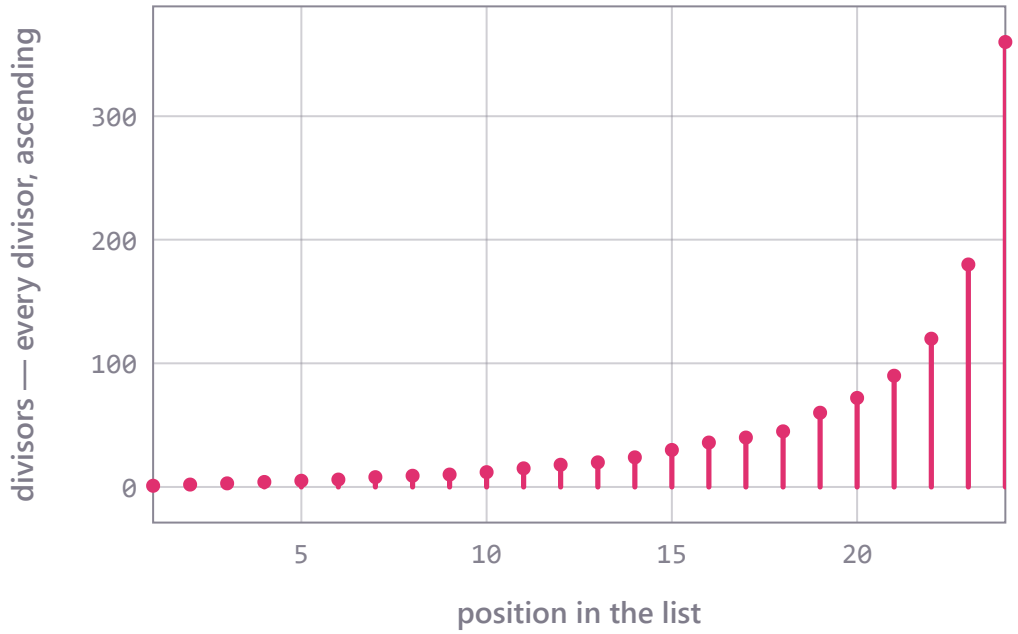
input	meaning	condition
`n`	a positive integer	

output	meaning
`count`	number of divisors tau(n)
`sum`	sum of divisors sigma(n)
`proper_sum`	sum of the proper divisors sigma(n) - n
`classification`	perfect, abundant or deficient
`divisors`	every divisor, ascending

- **Method:** divisors generated from the prime factorization; tau = prod(e + 1), sigma = sum of the divisors
- **Assumes:** n a whole number, $1 \le n \le 10^{12}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



perfect, abundant or deficient (classification) along n (a positive integer) from 1 to 720 (logarithmic axis): 63 runs of discrete-math-ai999x, each band coloured by the verdict returned (abundant, deficient, perfect); it changes near n = 5.47723, 6.48074, 11.4891, 12.49; the line is count (number of divisors tau(n)). Marked: the worked example, n = 360 → abundant.



every divisor, ascending (divisors) returned by discrete-math-ai999x for the worked example, one value for each position in the list (24 values, first 1, last 360, largest 360). Inputs: n = 360.

```
discrete_math_ai999x.solve('divisor_functions', n=360)
# -> count = 24, sum = 1170, proper_sum = 810, classification = abundant, divisors = [1, 2, 3, 4,
5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360]   (SOLVED)
```

```
discrete_math_ai999x.solve('divisor_functions', n=28)
# -> count = 6, sum = 56, proper_sum = 28, classification = perfect, divisors = [1, 2, 4, 7, 14,
28]   (SOLVED)
```

```
discrete_math_ai999x.solve('divisor_functions', n=12)
# -> count = 6, sum = 28, proper_sum = 16, classification = abundant, divisors = [1, 2, 3, 4, 6,
12]   (SOLVED)
```

```
discrete_math_ai999x.solve('divisor_functions', n=1)
# -> count = 1, sum = 1, proper_sum = 0, classification = deficient, divisors = [1]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many divisors does 360 have?	SOLVED	count = 24, sum = 1170, proper_sum = 810, classification = abundant, divisors = [1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360]
Find the sum of the divisors of 28.	SOLVED	count = 6, sum = 56, proper_sum = 28, classification = perfect, divisors = [1, 2, 4, 7, 14, 28]
Is 12 a perfect number?	SOLVED	count = 6, sum = 28, proper_sum = 16, classification = abundant, divisors = [1, 2, 3, 4, 6, 12]
List the divisors of 0.	INVALID_INPUT	n: must be at least 1, got 0.

Remainder modulo m (Euclidean division) — `mod_reduce`

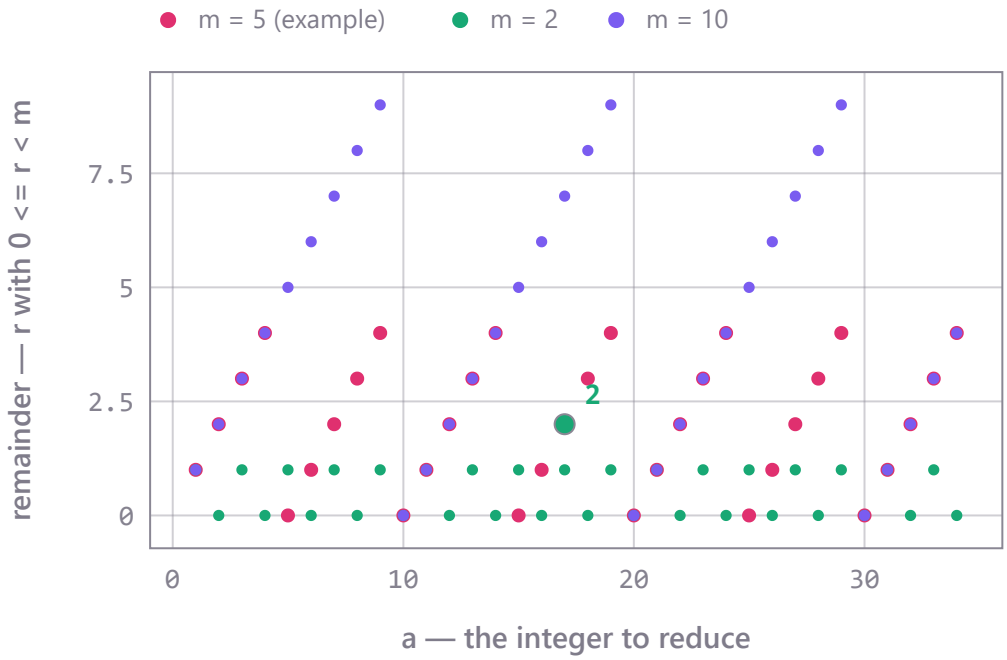
a mod m as the least non-negative residue, with the floor quotient: $a = qm + r$, $0 \le r < m$.

$$a = qm + r, \quad 0 \le r < m$$

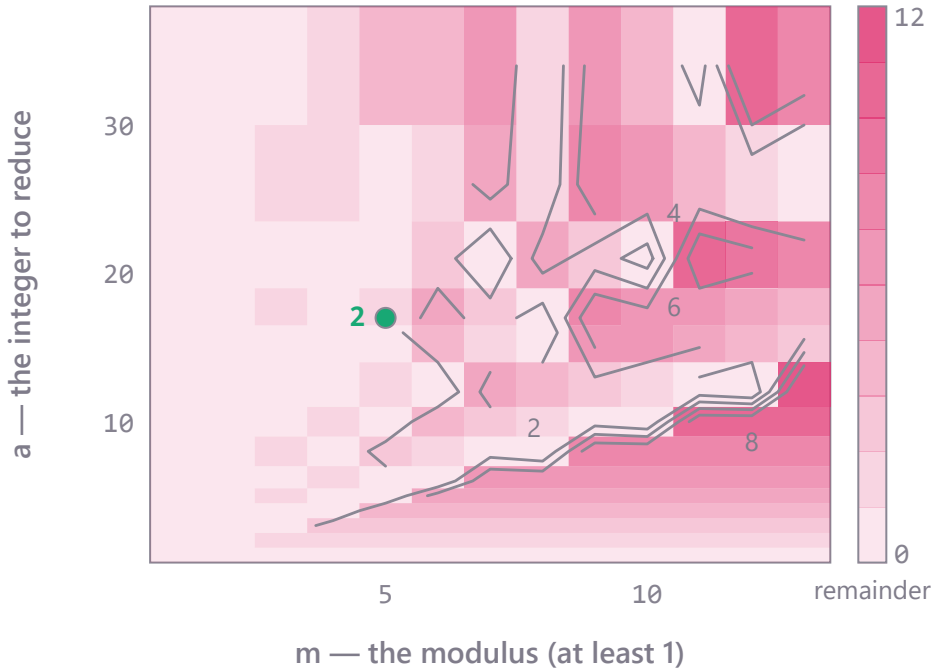
input	meaning	condition
`a`	the integer to reduce	
`m`	the modulus (at least 1)	

output	meaning
`remainder`	r with $0 \le r < m$
`quotient`	q = floor(a/m)

- **Method:** floor division
- **Assumes:** $m \ge 1$; the remainder is never negative, also for negative a
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



remainder (r with 0 <= r < m) against a (the integer to reduce) from 1 to 34. One curve for each value of m (the modulus (at least 1)): 5, 2, 10. 34 points per curve computed by discrete-math-ai999x; marked: the worked example, a = 17, remainder = 2.



remainder (r with 0 <= r < m) over m (the modulus (at least 1)) from 1 to 13 and a (the integer to reduce) from 1 to 34: each of the 13 × 14 cells is one run of discrete-math-ai999x, shaded by its value (light 0 to dark 12); iso-lines at remainder = 2, 4, 6, 8. Marked: the worked example, m = 5, a = 17, remainder = 2.

```
discrete_math_ai999x.solve('mod_reduce', a=17, m=5)
# -> remainder = 2, quotient = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_reduce', a=-17, m=5)
# -> remainder = 3, quotient = -4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is 17 mod 5?	SOLVED	remainder = 2, quotient = 3
Find the remainder when -17 is divided by 5.	SOLVED	remainder = 3, quotient = -4
What is 17 mod 0?	INVALID_INPUT	m: must be at least 1, got 0.

Modular exponentiation — `mod_pow`

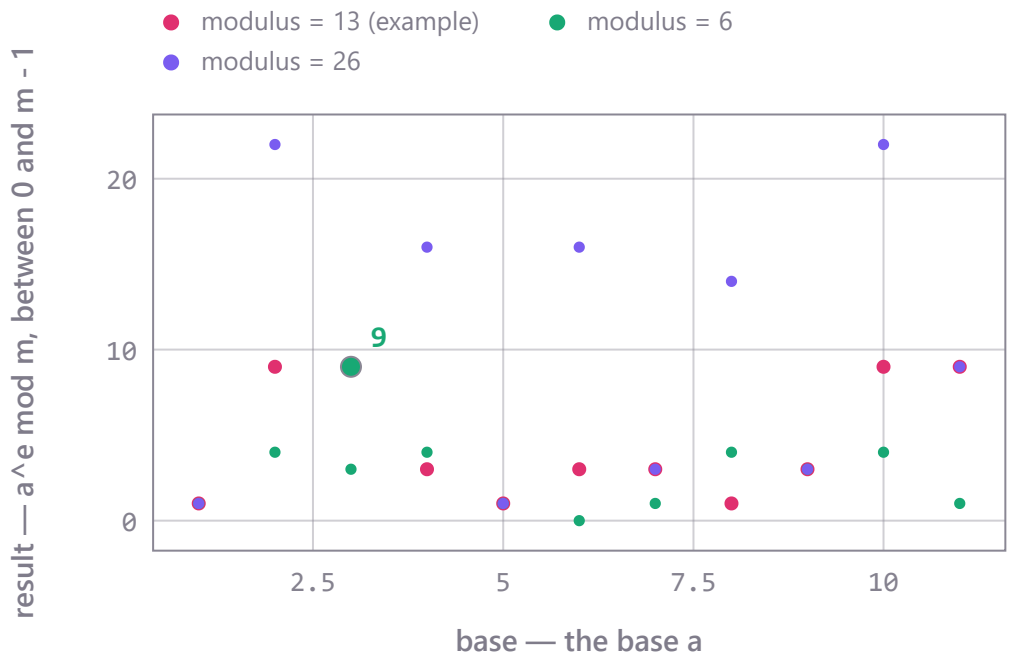
a^e mod m by repeated squaring (also the last k digits of a power).

$$a^e \bmod m, \quad a^{2k} = (a^k)^2,\ a^{2k+1} = a\,(a^k)^2$$

input	meaning	condition
`base`	the base a	
`exponent`	the exponent e (at least 0)	
`modulus`	the modulus m (at least 1)	

output	meaning
`result`	a^e mod m, between 0 and $m - 1$

- **Method:** binary (square-and-multiply) exponentiation, reducing modulo m at every step
- **Assumes:** $e \geq 0$; $m \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



result (a^e mod m, between 0 and m - 1) against base (the base a) from 1 to 11. One curve for each value of modulus (the modulus m (at least 1)): 13, 6, 26. Other inputs as in the worked example: exponent = 200. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, base = 3, result = 9.

```
discrete_math_ai999x.solve('mod_pow', base=3, exponent=200, modulus=13)
# -> result = 9   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_pow', base=7, exponent=222, modulus=1000)
# -> result = 49   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_pow', base=2, exponent=100, modulus=7)
# -> result = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_pow', base=5, exponent=0, modulus=1)
# -> result = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is 3^200 mod 13?	SOLVED	result = 9
Find the last 3 digits of 7^222.	SOLVED	result = 49
What is the remainder when 2^100 is divided by 7?	SOLVED	result = 2
What is 3^200 mod 0?	INVALID_INPUT	modulus: must be at least 1, got 0.

Modular multiplicative inverse — `mod_inverse`

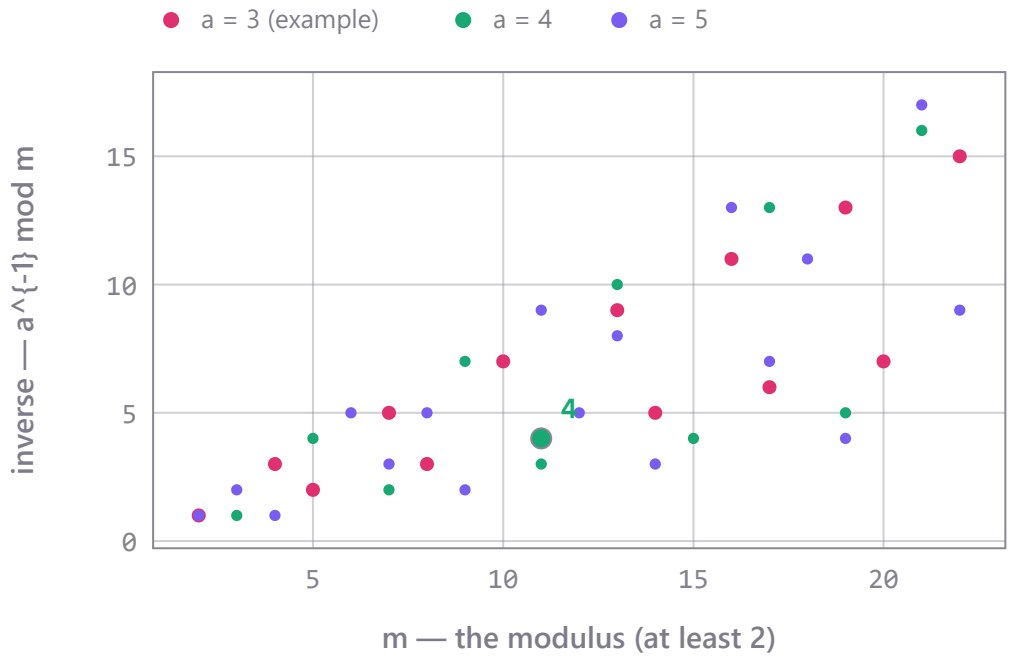
The x in $0..m-1$ with $ax \equiv 1 \pmod{m}$; it exists exactly when gcd(a, m) = 1.

$$ax \equiv 1 \pmod{m}$$

input	meaning	condition
`a`	the integer to invert	
`m`	the modulus (at least 2)	

output	meaning
`inverse`	a^{-1} mod m

- **Method:** extended Euclidean algorithm
- **Assumes:** $m \geq 2$; gcd(a, m) = 1 (otherwise no inverse exists)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



inverse (a^{-1} mod m) against m (the modulus (at least 2)) from 1 to 22. One curve for each value of a (the integer to invert): 3, 4, 5. 14 points per curve computed by discrete-math-ai999x; marked: the worked example, m = 11, inverse = 4.

```
discrete_math_ai999x.solve('mod_inverse', a=3, m=11)
# -> inverse = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_inverse', a=17, m=3120)
# -> inverse = 2753   (SOLVED)
```

```
discrete_math_ai999x.solve('mod_inverse', a=-3, m=7)
# -> inverse = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the inverse of 3 modulo 11.	SOLVED	inverse = 4
What is 17^-1 mod 3120?	SOLVED	inverse = 2753
Find the inverse of 4 modulo 10.	INVALID_INPUT	Modular multiplicative inverse: 4 has no inverse modulo 10 (gcd is 2).

Chinese remainder theorem — `crt`

The x with x ≡ r_i (mod m_i) for every i, unique modulo lcm(m_i); moduli need not be coprime.

$$x \equiv r_i \pmod{m_i},\ i = 1..k \ \Rightarrow\ x \bmod \operatorname{lcm}(m_1, \ldots, m_k)$$

input	meaning	condition
`residues`	the residues r_i	
`moduli`	the moduli m_i (each at least 1)	

output	meaning
`x`	the smallest non-negative solution

output	meaning
`modulus`	lcm of the moduli: the solution is unique modulo it

- **Method:** merge the congruences pairwise with the extended Euclidean algorithm (works for non-coprime moduli)
- **Assumes:** one residue per modulus; the system is consistent (r_i ≡ r_j modulo gcd(m_i, m_j))
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Chinese remainder theorem as inputs and results: stated residues = 2, 3, 2, moduli = 3, 5, 7; returned by discrete-math-ai999x: x = 23, modulus = 105.

```
discrete_math_ai999x.solve('crt', residues='2, 3, 2', moduli='3, 5, 7')
# -> x = 23, modulus = 105   (SOLVED)
```

```
discrete_math_ai999x.solve('crt', residues='1, 2', moduli='4, 9')
# -> x = 29, modulus = 36   (SOLVED)
```

```
discrete_math_ai999x.solve('crt', residues='3, 5', moduli='4, 6')
# -> x = 11, modulus = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve x ≡ 2 (mod 3), x ≡ 3 (mod 5), x ≡ 2 (mod 7).	SOLVED	x = 23, modulus = 105
Use the Chinese remainder theorem with residues 1, 2 and moduli 4, 9.	SOLVED	x = 29, modulus = 36
Solve x ≡ 1 (mod 4), x ≡ 2 (mod 6).	INVALID_INPUT	Chinese remainder theorem: the congruences are inconsistent: no integer satisfies all of them.
Solve 2x ≡ 1 (mod 3), x ≡ 2 (mod 5).	OUT_OF_SCOPE	Out of scope: a system with coefficients on x (each congruence must read x ≡ r (mod m)).

Discrete logarithm — `discrete_log`

The smallest $x \geq 0$ with base^x ≡ target (mod m).

$$g^x \equiv h \pmod{m}, \quad x = i\lceil\sqrt{m}\rceil + j$$

input	meaning	condition
`base`	the base g	
`target`	the value h	
`modulus`	the modulus m $(2..10^{12})$	

output	meaning
`x`	the smallest non-negative exponent with $g^x \equiv h \pmod m$

- **Method:** baby-step giant-step (keeping the smallest baby step for each value, so the first match is the smallest x)
- **Assumes:** gcd(base, m) = 1; a solution exists
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (the smallest non-negative exponent with g^x ≡ h (mod m)) against target (the value h) from 1 to 26. One curve for each value of base (the base g): 3, 5. Other inputs as in the worked example: modulus = 17. 25 points per curve computed by discrete-math-ai999x; marked: the worked example, target = 13, x = 4.

```
discrete_math_ai999x.solve('discrete_log', base=3, target=13, modulus=17)
# -> x = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('discrete_log', base=2, target=1, modulus=7)
# -> x = 0   (SOLVED)
```

```
discrete_math_ai999x.solve('discrete_log', base=5, target=8, modulus=23)
# -> x = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the smallest x with 3^x ≡ 13 (mod 17).	SOLVED	x = 4

question	outcome	result
Find the discrete logarithm of 13 to the base 3 modulo 17.	SOLVED	x = 4
Find the smallest x with 2^x ≡ 3 (mod 7).	INVALID_INPUT	Discrete logarithm: no exponent x satisfies 2^x ≡ 3 (mod 7).

Legendre symbol — `legendre_symbol`

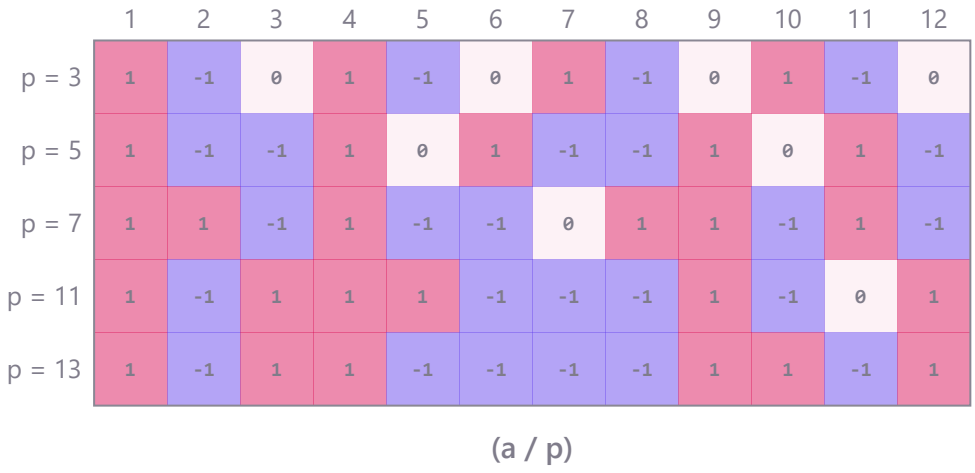
(a/p) for an odd prime p: 1 if a is a non-zero quadratic residue mod p, -1 if not, 0 if p divides a.

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

input	meaning	condition
`a`	the integer a	
`p`	an odd prime	

output	meaning
`symbol`	the Legendre symbol (a/p): -1, 0 or 1

- **Method:** quadratic reciprocity with the supplementary law for 2 (no factoring needed)
- **Assumes:** p an odd prime
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Legendre symbol (a / p) returned by the package for a = 1 … 12 (columns) and the odd primes p = 3, 5, 7, 11, 13 (rows): +1 a quadratic residue, −1 a non-residue, 0 when p divides a. The worked example is a = 5, p = 11.

```
discrete_math_ai999x.solve('legendre_symbol', a=5, p=11)
# -> symbol = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('legendre_symbol', a=2, p=7)
# -> symbol = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('legendre_symbol', a=3, p=7)
# -> symbol = -1   (SOLVED)
```

```
discrete_math_ai999x.solve('legendre_symbol', a=14, p=7)
# -> symbol = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Legendre symbol (5/11).	SOLVED	symbol = 1
Compute the Legendre symbol (-1/13).	SOLVED	symbol = 1
Compute the Legendre symbol (2/15).	INVALID_INPUT	Legendre symbol: p must be an odd prime.

Jacobi symbol — `jacobi_symbol`

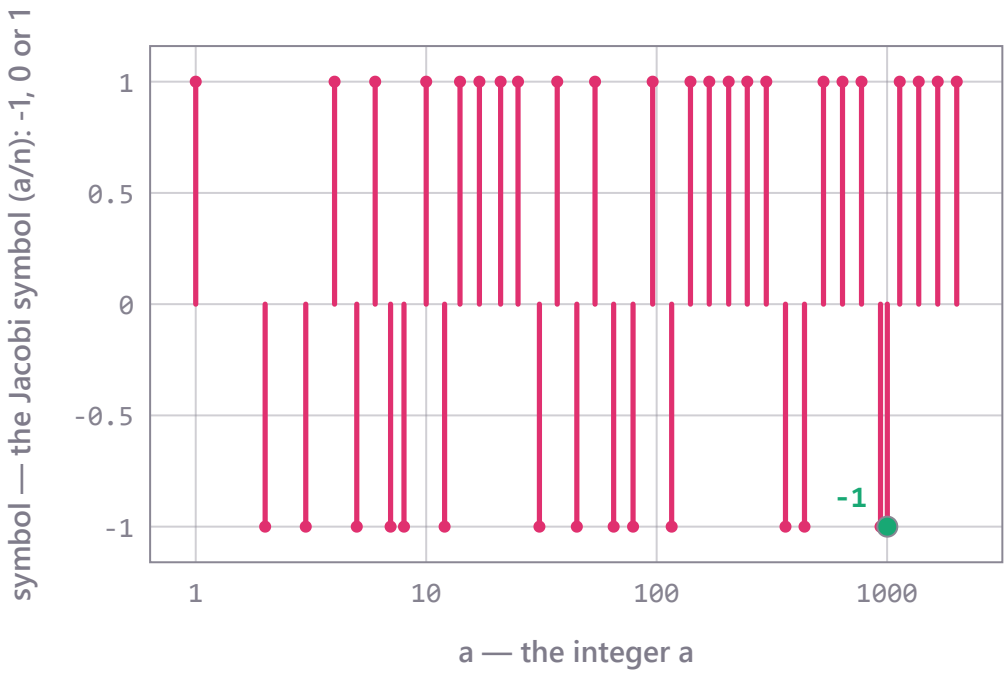
(a/n) for an odd positive n, the product of Legendre symbols over the prime factors of n.

$$\left(\frac{a}{n}\right) = \prod_i \left(\frac{a}{p_i}\right)^{e_i}, \quad n = \prod p_i^{e_i}$$

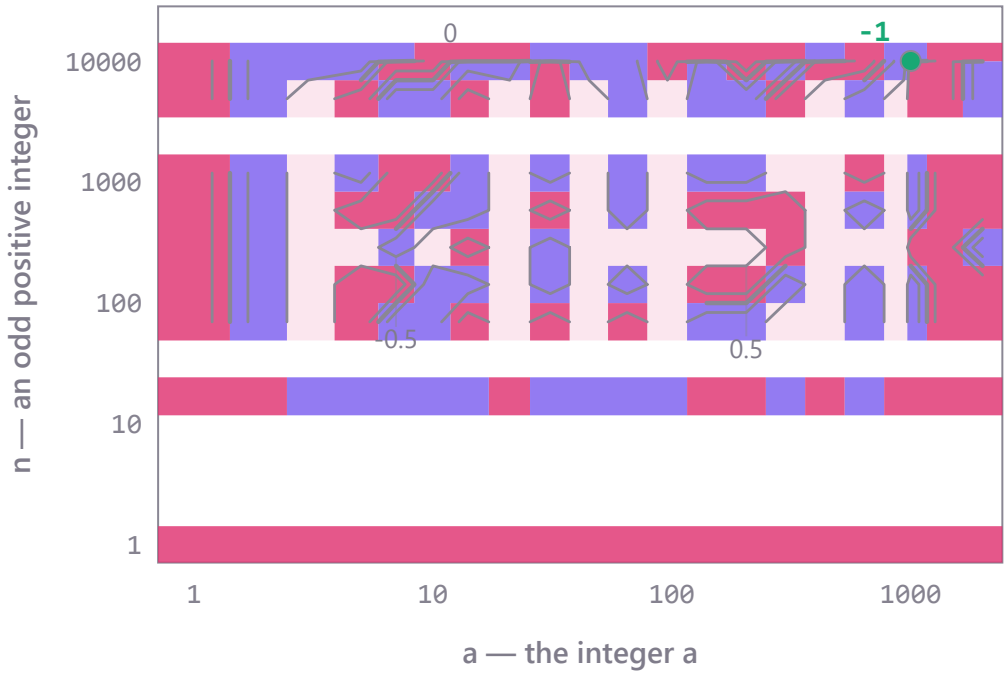
input	meaning	condition
`a`	the integer a	
`n`	an odd positive integer	

output	meaning
`symbol`	the Jacobi symbol (a/n): -1, 0 or 1

- **Method:** quadratic reciprocity with the supplementary law for 2 (no factoring needed)
- **Assumes:** n odd and positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



symbol (the Jacobi symbol (a/n): -1, 0 or 1) against a (the integer a) from 1 to 2002 on a logarithmic axis. Other inputs as in the worked example: n = 9907. 38 points computed by discrete-math-ai999x; marked: the worked example, a = 1001, symbol = -1.



symbol (the Jacobi symbol (a/n): -1, 0 or 1) over a (the integer a) from 1 to 2002 and n (an odd positive integer) from 1 to 19814 (logarithmic axes): each of the 21 × 16 cells is one run of discrete-math-ai999x, shaded by its sign and magnitude (second colour negative); iso-lines at symbol = -0.5, 0, 0.5; blank cells: inputs the capability refuses. Marked: the worked example, a = 1001, n = 9907, symbol = -1.

```
discrete_math_ai999x.solve('jacobi_symbol', a=1001, n=9907)
# -> symbol = -1   (SOLVED)
```

```
discrete_math_ai999x.solve('jacobi_symbol', a=2, n=15)
# -> symbol = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('jacobi_symbol', a=5, n=21)
# -> symbol = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('jacobi_symbol', a=3, n=9)
# -> symbol = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Jacobi symbol (1001/9907).	SOLVED	symbol = -1
Find the Jacobi symbol (2/15).	SOLVED	symbol = 1
Find the Jacobi symbol (3/10).	INVALID_INPUT	Jacobi symbol: n must be odd.

Square roots modulo a prime — `sqrt_mod`

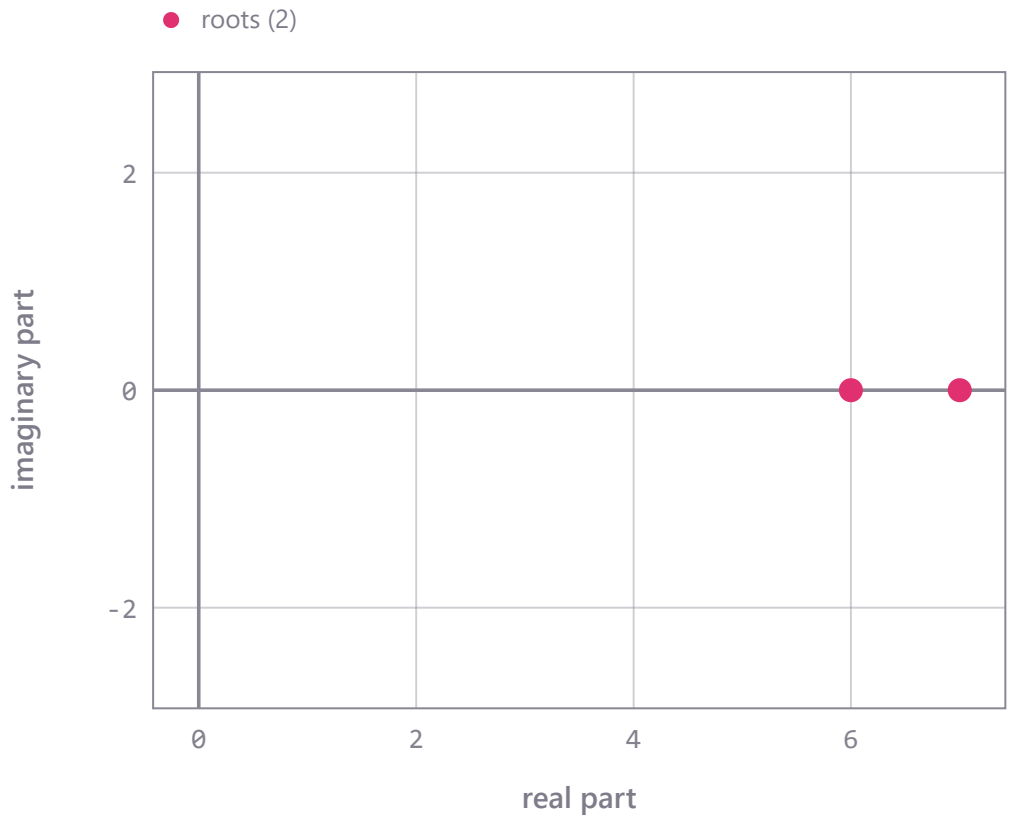
Every x in $0..p-1$ with x^2 ≡ a (mod p), for a prime p.

$$x^2 \equiv a \pmod{p}, \quad a^{(p-1)/2} \equiv 1$$

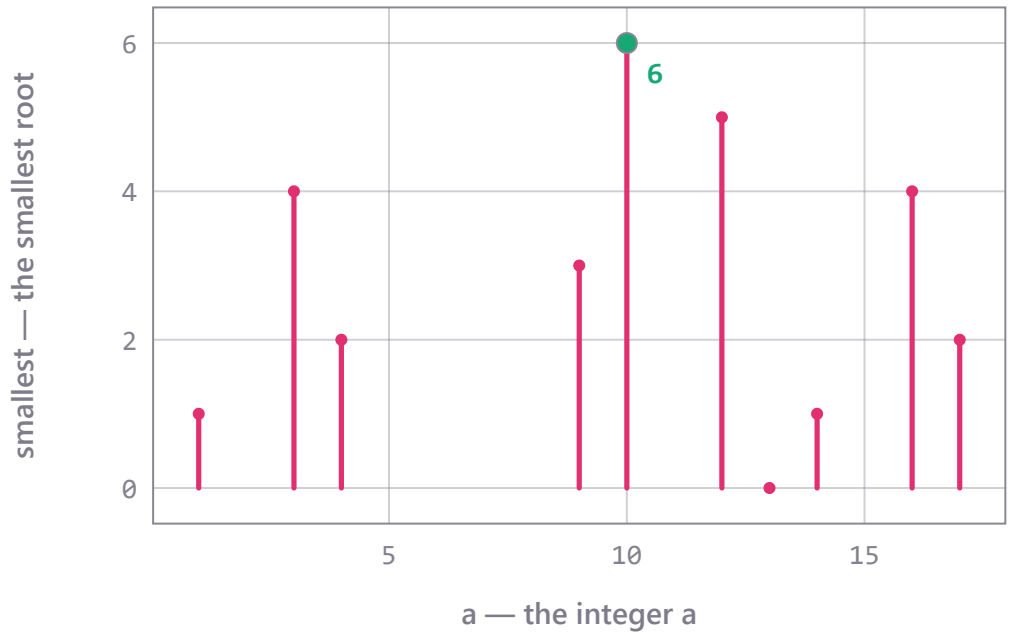
input	meaning	condition
`a`	the integer a	
`p`	a prime modulus	

output	meaning
`roots`	all square roots of a modulo p, ascending
`count`	number of roots (0, 1 or 2)
`smallest`	the smallest root (none when a is a non-residue)

- **Method:** Euler's criterion decides solvability; Tonelli-Shanks finds a root r, the other is p - r
- **Assumes:** p prime
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



all square roots of a modulo p, ascending (roots) returned by discrete-math-ai999x for the worked example (a = 10, p = 13), drawn in the complex plane: 6 + 0i, 7 + 0i.



smallest (the smallest root (none when a is a non-residue)) against a (the integer a) from 1 to 20. Other inputs as in the worked example: p = 13. 10 points computed by discrete-math-ai999x; marked: the worked example, a = 10, smallest = 6.

```
discrete_math_ai999x.solve('sqrt_mod', a=10, p=13)
# -> roots = [6, 7], count = 2, smallest = 6   (SOLVED)
```

```
discrete_math_ai999x.solve('sqrt_mod', a=2, p=7)
# -> roots = [3, 4], count = 2, smallest = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('sqrt_mod', a=3, p=7)
# -> roots = [], count = 0, smallest = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solve x^2 ≡ 10 (mod 13).	SOLVED	roots = [6, 7], count = 2, smallest = 6
Find the square roots of 2 modulo 7.	SOLVED	roots = [3, 4], count = 2, smallest = 3
Solve x^2 ≡ 3 (mod 7).	SOLVED	roots = [], count = 0, smallest = none
Solve x^2 ≡ 4 (mod 15).	INVALID_INPUT	Square roots modulo a prime: p must be prime.

Multiplicative order — `multiplicative_order`

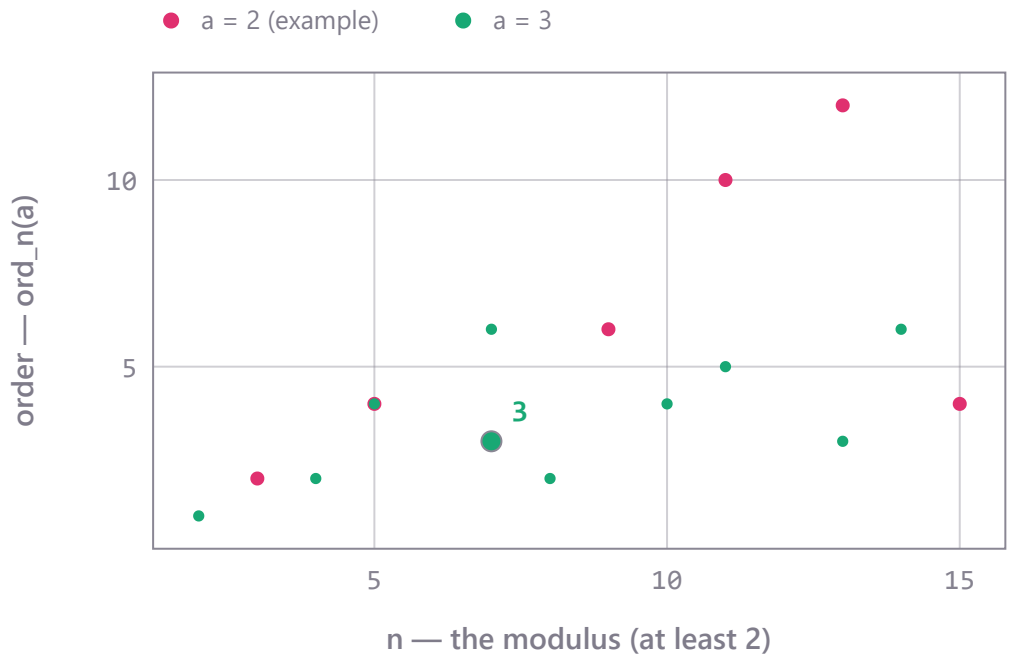
The smallest $k \geq 1$ with a^k ≡ 1 (mod n).

$$\mathrm{ord}_n(a) = \min\{k \geq 1 : a^k \equiv 1 \pmod{n}\},\ \mathrm{ord}_n(a) \mid \lambda(n)$$

input	meaning	condition
`a`	the integer a (coprime to n)	
`n`	the modulus (at least 2)	

output	meaning
`order`	ord_n(a)

- **Method:** start from the Carmichael exponent lambda(n) and divide out prime factors while $a^{k/p}$ ≡ 1
- **Assumes:** gcd(a, n) = 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



order (ord_n(a)) against n (the modulus (at least 2)) from 1 to 15. One curve for each value of a (the integer a (coprime to n)): 2, 3. 7 points per curve computed by discrete-math-ai999x; marked: the worked example, n = 7, order = 3.

```
discrete_math_ai999x.solve('multiplicative_order', a=2, n=7)
# -> order = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('multiplicative_order', a=3, n=100)
# -> order = 20   (SOLVED)
```

```
discrete_math_ai999x.solve('multiplicative_order', a=10, n=7)
# -> order = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the order of 2 modulo 7?	SOLVED	order = 3
Find the multiplicative order of 3 mod 100.	SOLVED	order = 20
Find the multiplicative order of 4 mod 10.	INVALID_INPUT	Multiplicative order: a and n must be coprime.

Smallest primitive root modulo a prime — `primitive_root`

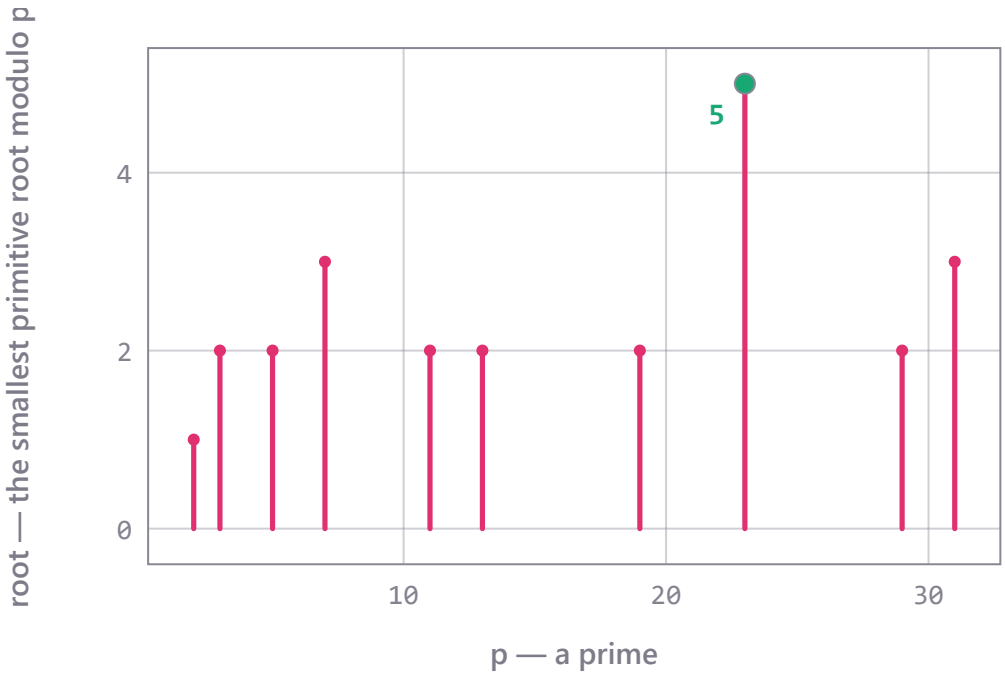
The smallest g whose powers run through every non-zero residue modulo the prime p.

$$g^{(p-1)/q} \not\equiv 1 \pmod{p} \;\; \forall q \mid p-1$$

input	meaning	condition
`p`	a prime	

output	meaning
`root`	the smallest primitive root modulo p

- **Method:** g is a primitive root iff g^((p-1)/q) ≢ 1 for every prime q dividing $p-1$; test $g = 2$, 3, ...
- **Assumes:** p prime (primitive roots also exist for 4, p^k and $2p^k$, which this capability does not take)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



root (the smallest primitive root modulo p) against p (a prime) from 1 to 46. 10 points computed by discrete-math-ai999x; marked: the worked example, p = 23, root = 5.

```
discrete_math_ai999x.solve('primitive_root', p=23)
# -> root = 5   (SOLVED)
```

```
discrete_math_ai999x.solve('primitive_root', p=7)
# -> root = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('primitive_root', p=41)
# -> root = 6   (SOLVED)
```

```
discrete_math_ai999x.solve('primitive_root', p=2)
# -> root = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the smallest primitive root modulo 23.	SOLVED	root = 5
What is the primitive root of 41?	SOLVED	root = 6
Find the primitive root modulo 15.	INVALID_INPUT	Smallest primitive root modulo a prime: p must be prime.

Prime-counting function — `prime_counting`

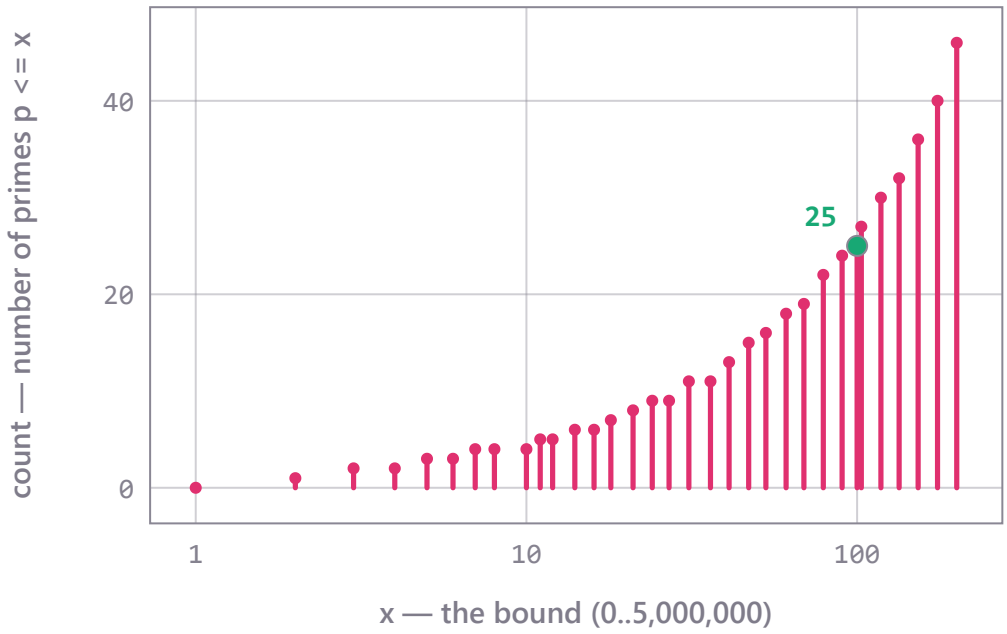
pi(x): how many primes are at most x.

$$\pi(x) = \#\{p \le x : p \text{ prime}\}$$

input	meaning	condition
`x`	the bound (0..5,000,000)	

output	meaning
`count`	number of primes $p \leq x$

- **Method:** sieve of Eratosthenes
- **Assumes:** $0 \leq x \leq 5{,}000{,}000$; 'less than N' counts primes up to $N - 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



count (number of primes p <= x) against x (the bound (0..5,000,000)) from 1 to 200 on a logarithmic axis. 33 points computed by discrete-math-ai999x; marked: the worked example, x = 100, count = 25.

```
discrete_math_ai999x.solve('prime_counting', x=100)
# -> count = 25   (SOLVED)
```

```
discrete_math_ai999x.solve('prime_counting', x=1000)
# -> count = 168   (SOLVED)
```

```
discrete_math_ai999x.solve('prime_counting', x=1)
# -> count = 0   (SOLVED)
```

```
discrete_math_ai999x.solve('prime_counting', x=1000000)
# -> count = 78498   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many primes are there less than 100?	SOLVED	count = 25
How many primes are at most 1000?	SOLVED	count = 168
Compute the prime counting function pi(10000).	SOLVED	count = 1229
How many primes are there below 10?	SOLVED	count = 4

The n-th prime — `nth_prime`

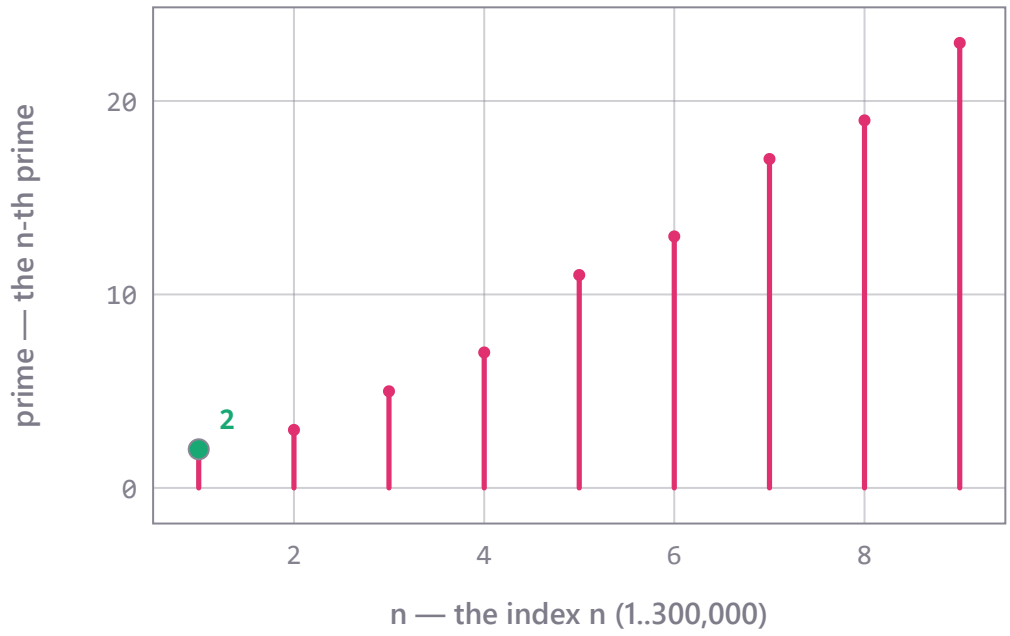
The n-th prime number p_n $(p_1 = 2)$.

$$p_n < n(\ln n + \ln \ln n) \quad (n \geq 6)$$

input	meaning	condition
`n`	the index n (1..300,000)	

output	meaning
`prime`	the n-th prime

- **Method:** sieve of Eratosthenes up to the Rosser bound n (ln n + ln ln n)
- **Assumes:** $1 \leq n \leq 300$,000; $p_1 = 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



prime (the n-th prime) against n (the index n (1..300,000)) from 1 to 9. 9 points computed by discrete-math-ai999x; marked: the worked example, n = 1, prime = 2.

```
discrete_math_ai999x.solve('nth_prime', n=1)
# -> prime = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('nth_prime', n=100)
# -> prime = 541   (SOLVED)
```

```
discrete_math_ai999x.solve('nth_prime', n=10001)
# -> prime = 104743   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 100th prime?	SOLVED	prime = 541
Find the 10001st prime number.	SOLVED	prime = 104743
What is the 400000th prime?	INVALID_INPUT	n: must be at most 300000, got 400000.

Next prime — `next_prime`

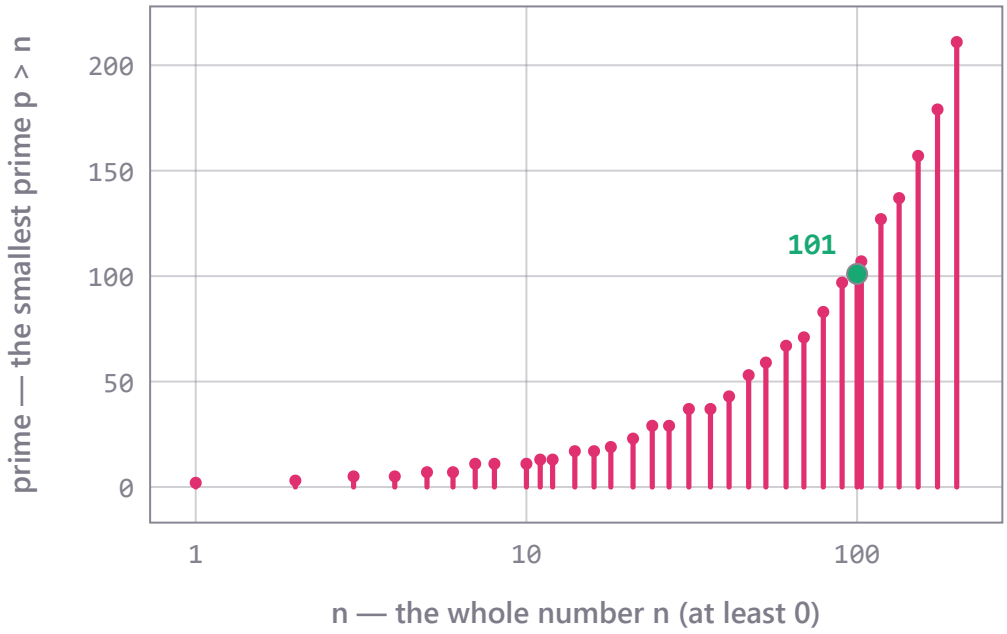
The smallest prime strictly greater than n.

$$\min\{p > n : p \text{ prime}\}$$

input	meaning	condition
`n`	the whole number n (at least 0)	

output	meaning
`prime`	the smallest prime $p > n$

- **Method:** test $n + 1$, $n + 2$, ... with deterministic Miller-Rabin
- **Assumes:** $n \geq 0$; strictly greater: the next prime after a prime p is not p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



prime (the smallest prime p > n) against n (the whole number n (at least 0)) from 1 to 200 on a logarithmic axis. 33 points computed by discrete-math-ai999x; marked: the worked example, n = 100, prime = 101.

```
discrete_math_ai999x.solve('next_prime', n=100)
# -> prime = 101   (SOLVED)
```

```
discrete_math_ai999x.solve('next_prime', n=13)
# -> prime = 17   (SOLVED)
```

```
discrete_math_ai999x.solve('next_prime', n=0)
# -> prime = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the smallest prime greater than 100?	SOLVED	prime = 101
Find the next prime after 1000.	SOLVED	prime = 1009

Previous prime — `previous_prime`

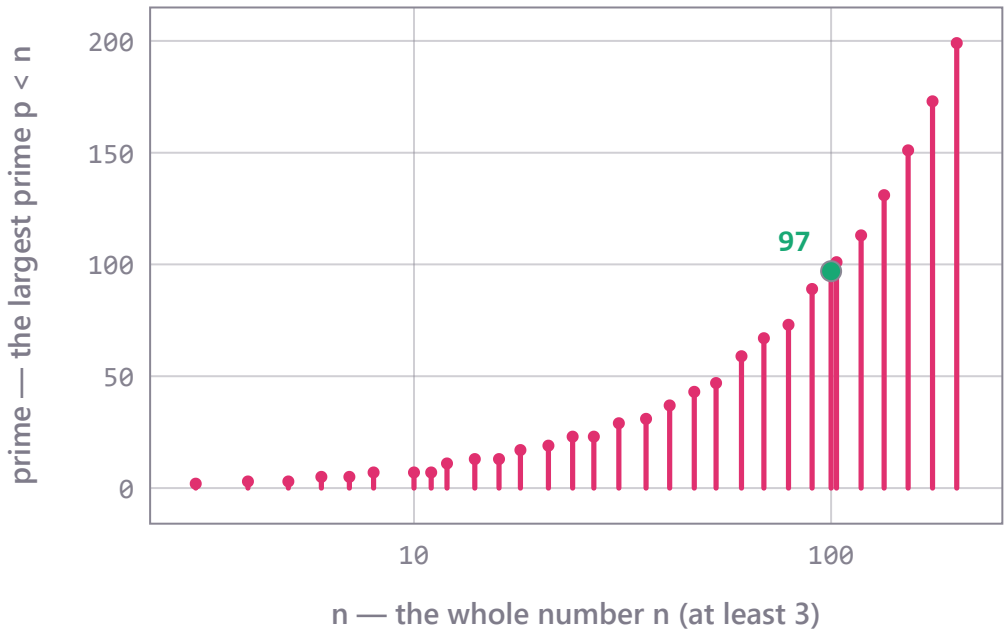
The largest prime strictly less than n.

$$\max\{p < n : p \text{ prime}\}$$

input	meaning	condition
`n`	the whole number n (at least 3)	

output	meaning
`prime`	the largest prime $p < n$

- **Method:** test $n - 1$, $n - 2$, ... with deterministic Miller-Rabin
- **Assumes:** $n \geq 3$; strictly less: the prime before a prime p is not p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



prime (the largest prime p < n) against n (the whole number n (at least 3)) from 1 to 200 on a logarithmic axis. 31 points computed by discrete-math-ai999x; marked: the worked example, n = 100, prime = 97.

```
discrete_math_ai999x.solve('previous_prime', n=100)
# -> prime = 97   (SOLVED)
```

```
discrete_math_ai999x.solve('previous_prime', n=13)
# -> prime = 11   (SOLVED)
```

```
discrete_math_ai999x.solve('previous_prime', n=3)
# -> prime = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the largest prime less than 100?	SOLVED	prime = 97
Find the largest prime below 1000.	SOLVED	prime = 997
What is the largest prime less than 2?	INVALID_INPUT	n: must be at least 3, got 2.

Carmichael function — `carmichael_lambda`

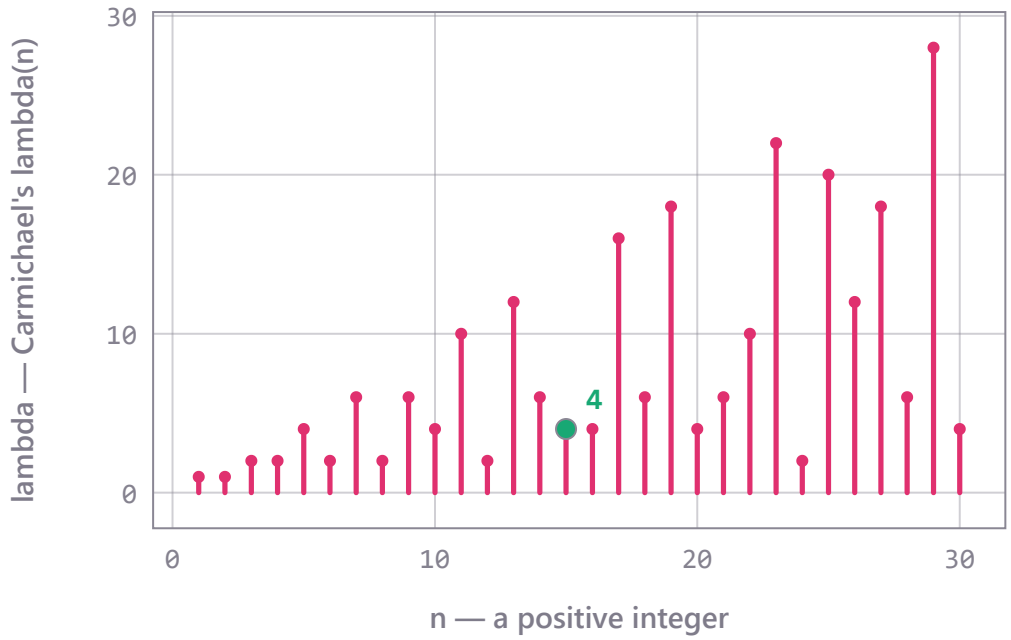
lambda(n): the smallest m with $a^m \equiv 1 \pmod n$ for every a coprime to n (the exponent of the unit group).

$$\lambda(n) = \operatorname{lcm}\left(\lambda(p_1^{e_1}), \ldots\right),\ \lambda(p^e) = p^{e-1}(p-1)$$

input	meaning	condition
`n`	a positive integer	

output	meaning
`lambda`	Carmichael's lambda(n)

- **Method:** lcm over prime powers: lambda(2^e) = 2^(e-2) for $e \geq 3$, else phi(p^e)
- **Assumes:** $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



lambda (Carmichael's lambda(n)) against n (a positive integer) from 1 to 30. 30 points computed by discrete-math-ai999x; marked: the worked example, n = 15, lambda = 4.

```
discrete_math_ai999x.solve('carmichael_lambda', n=15)
# -> lambda = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('carmichael_lambda', n=561)
# -> lambda = 80   (SOLVED)
```

```
discrete_math_ai999x.solve('carmichael_lambda', n=8)
# -> lambda = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('carmichael_lambda', n=100)
# -> lambda = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Carmichael function of 15.	SOLVED	lambda = 4
What is the reduced totient of 561?	SOLVED	lambda = 80
Find λ(100).	SOLVED	lambda = 20

Möbius function — `mobius`

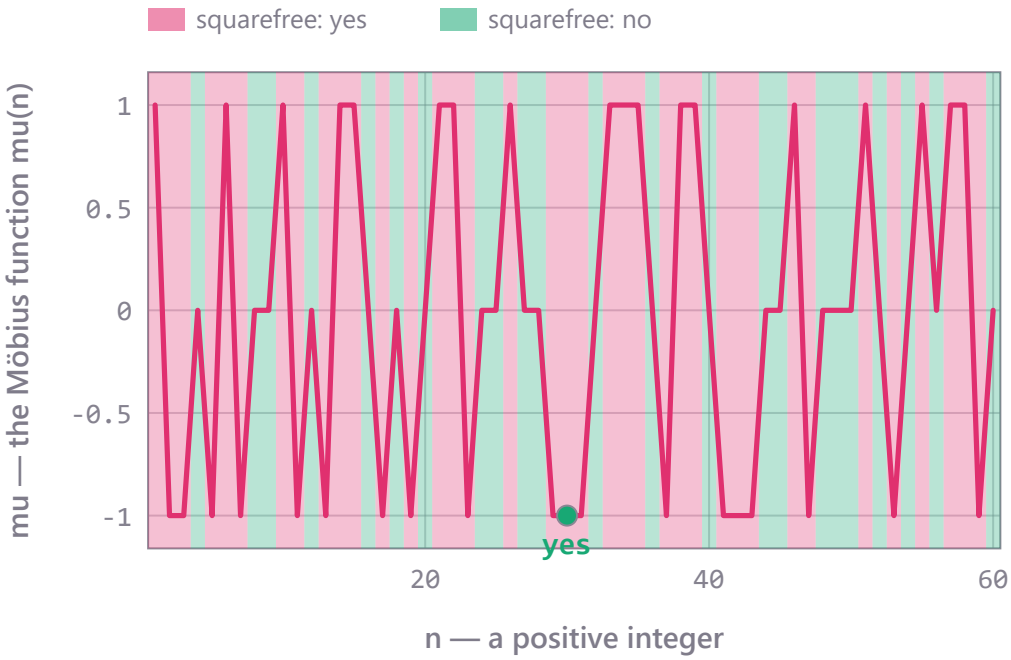
mu(n): 0 if a square divides n, otherwise (-1)^(number of prime factors).

$$\mu(n) = \begin{cases} (-1)^k & n = p_1 \cdots p_k \text{ squarefree} \\ 0 & p^2 \mid n \end{cases}$$

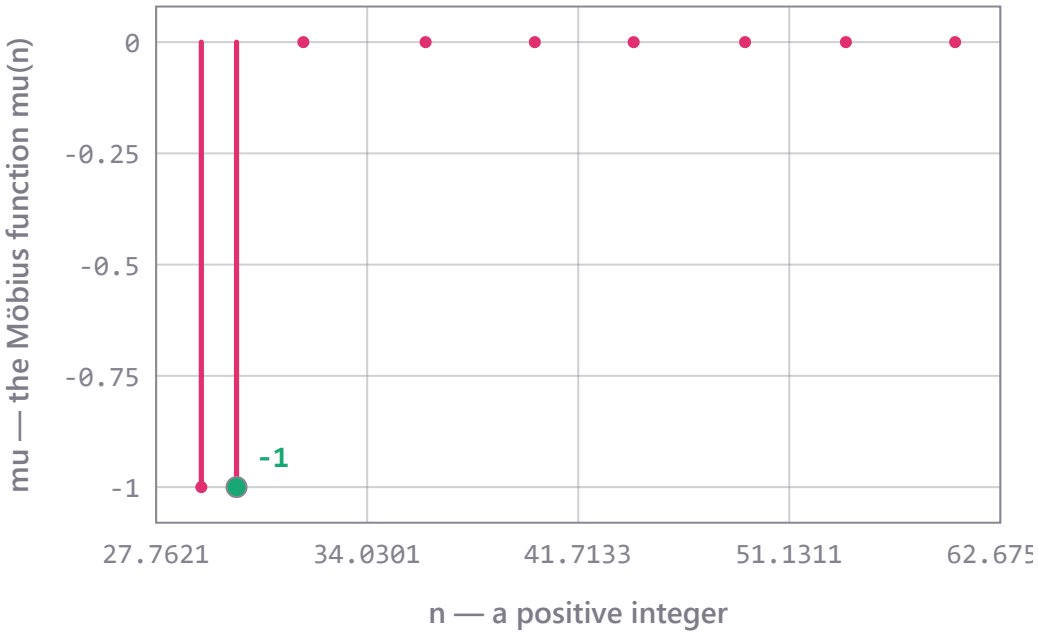
input	meaning	condition
n	a positive integer	

output	meaning
`mu`	the Möbius function mu(n)
`squarefree`	true when no square > 1 divides n

- **Method:** prime factorization
- **Assumes:** $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when no square > 1 divides n (squarefree) along n (a positive integer) from 1 to 60: 60 runs of discrete-math-ai999x, each band coloured by the verdict returned (yes, no); it changes near n = 3.5, 4.5, 7.5, 9.5; the line is mu (the Möbius function mu(n)). Marked: the worked example, n = 30 → yes.



mu (the Möbius function mu(n)) against n (a positive integer) from 29 to 60 on a logarithmic axis. 9 points computed by discrete-math-ai999x; marked: the worked example, n = 30, mu = -1.

```
discrete_math_ai999x.solve('mobius', n=30)
# -> mu = -1, squarefree = True   (SOLVED)
```

```
discrete_math_ai999x.solve('mobius', n=12)
# -> mu = 0, squarefree = False   (SOLVED)
```

```
discrete_math_ai999x.solve('mobius', n=1)
# -> mu = 1, squarefree = True   (SOLVED)
```

```
discrete_math_ai999x.solve('mobius', n=35)
# -> mu = 1, squarefree = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Möbius function of 30.	SOLVED	mu = -1, squarefree = True
What is mu(12)?	SOLVED	mu = 0, squarefree = False

Continued fraction of a rational number — `cf_rational`

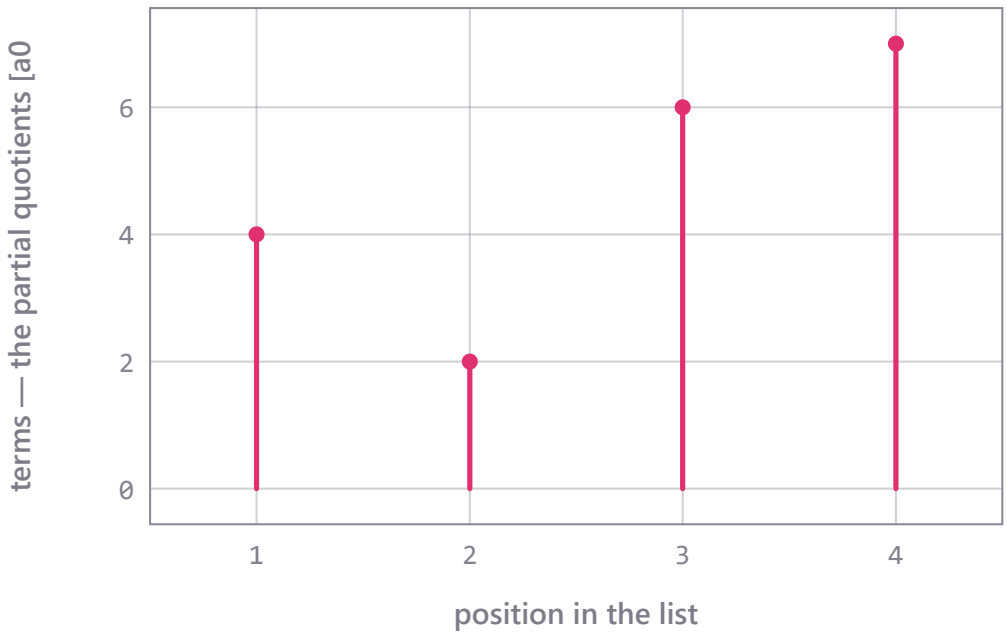
The finite simple continued fraction [a0; a1, ..., an] of p/q (canonical form: last term >= 2).

$$\frac{p}{q} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_n}}}$$

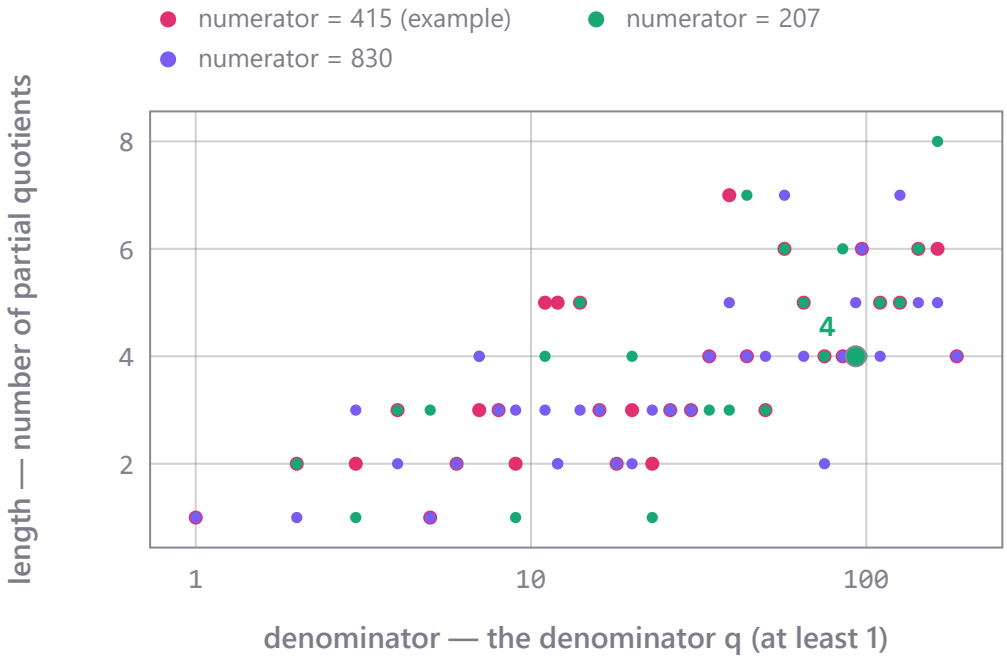
input	meaning	condition
`numerator`	the numerator p	
`denominator`	the denominator q (at least 1)	

output	meaning
`terms`	the partial quotients [a0, a1, ..., an]
`length`	number of partial quotients

- **Method:** Euclidean algorithm: a_k = floor(p/q), then (p, q) <- (q, p - a_k q)
- **Assumes:** $q \geq 1$; a0 = floor(p/q) (negative for negative p); canonical: the last term is at least 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the partial quotients [a0, a1, ..., an] (terms) returned by discrete-math-ai999x for the worked example, one value for each position in the list (4 values, first 4, last 7, largest 7). Inputs: numerator = 415, denominator = 93.



length (number of partial quotients) against denominator (the denominator q (at least 1)) from 1 to 186 on a logarithmic axis. One curve for each value of numerator (the numerator p): 415, 207, 830. 33 points per curve computed by discrete-math-ai999x; marked: the worked example, denominator = 93, length = 4.

```
discrete_math_ai999x.solve('cf_rational', numerator=415, denominator=93)
# -> terms = [4, 2, 6, 7], length = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('cf_rational', numerator=355, denominator=113)
# -> terms = [3, 7, 16], length = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('cf_rational', numerator=-7, denominator=3)
# -> terms = [-3, 1, 2], length = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the continued fraction of 415/93.	SOLVED	terms = [4, 2, 6, 7], length = 4
What is the continued fraction expansion of 355/113?	SOLVED	terms = [3, 7, 16], length = 3
Find the continued fraction of 5/0.	INVALID_INPUT	denominator: must be at least 1, got 0.

Continued fraction of a square root — `cf_sqrt`

The periodic continued fraction sqrt(d) = [a0; (a1, ..., ak)] of a non-square d, and its period length.

$$\sqrt{d} = [a_0; \overline{a_1, \ldots, a_{k-1}, 2a_0}]$$

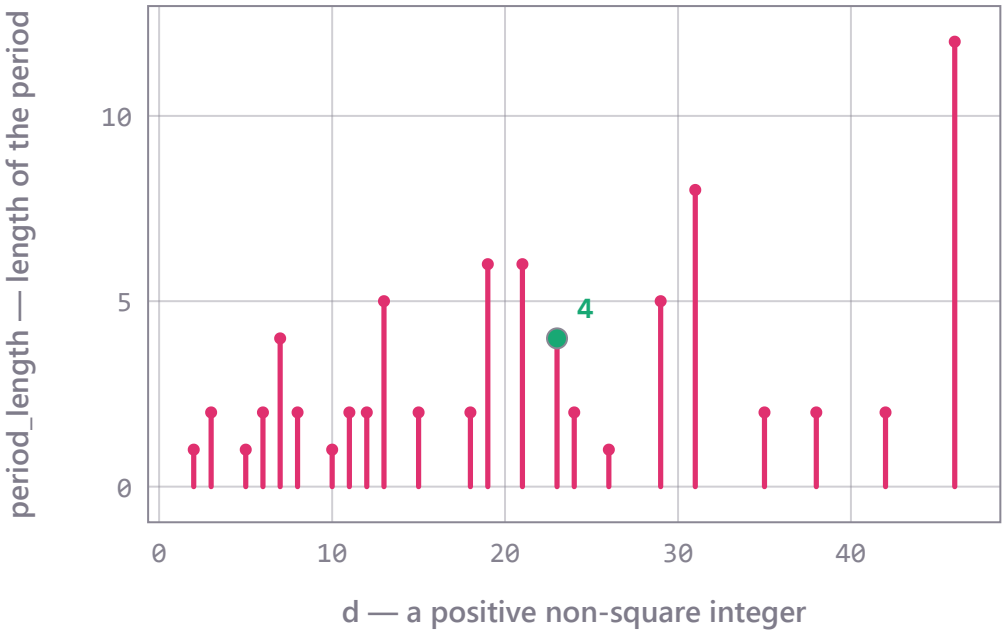
input	meaning	condition
`d`	a positive non-square integer	

output	meaning
`a0`	integer part floor(sqrt(d))
`period`	the repeating block (ends with 2*a0)
`period_length`	length of the period

- **Method:** the exact recurrence m' = den*a - m, den' = (d - m'^2)/den, a' = floor((a0 + m')/den') until a' = $2a_0$
- **Assumes:** d not a perfect square
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the repeating block (ends with 2*a0) (period) returned by discrete-math-ai999x for the worked example, one value for each position in the list (4 values, first 1, last 8, largest 8). Inputs: d = 23.



period_length (length of the period) against d (a positive non-square integer) from 1 to 46. 23 points computed by discrete-math-ai999x; marked: the worked example, d = 23, period_length = 4.

```
discrete_math_ai999x.solve('cf_sqrt', d=23)
# -> a0 = 4, period = [1, 3, 1, 8], period_length = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('cf_sqrt', d=2)
# -> a0 = 1, period = [2], period_length = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('cf_sqrt', d=14)
# -> a0 = 3, period = [1, 2, 1, 6], period_length = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the continued fraction of sqrt(23).	SOLVED	a0 = 4, period = [1, 3, 1, 8], period_length = 4
What is the continued fraction expansion of √14?	SOLVED	a0 = 3, period = [1, 2, 1, 6], period_length = 4
Find the continued fraction of sqrt(16).	INVALID_INPUT	Continued fraction of a square root: d must not be a perfect square (its square root is an integer).

Pell's equation (fundamental solution) — `pell_equation`

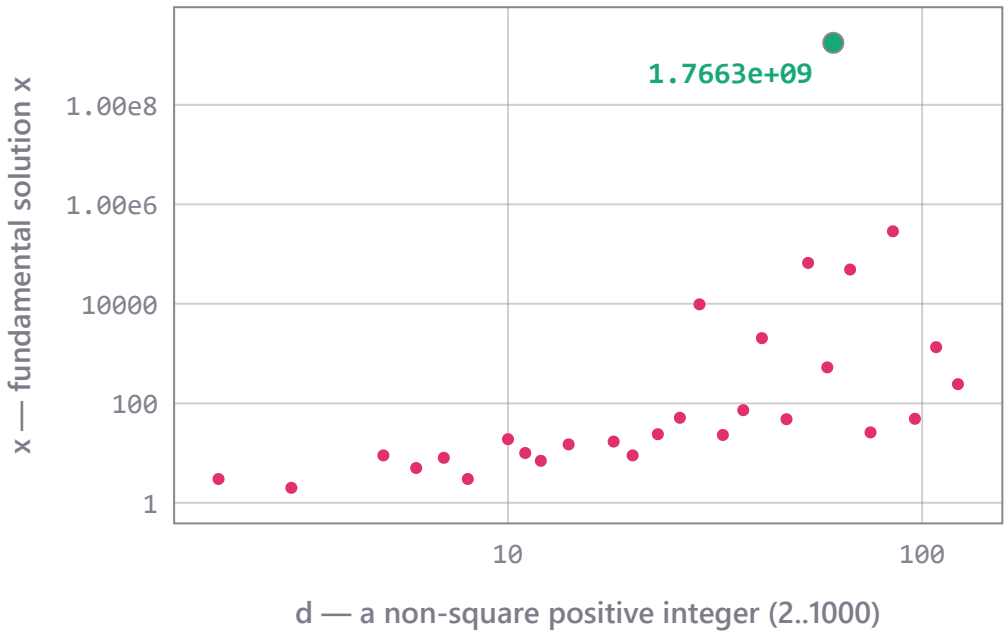
The smallest positive solution (x, y) of $x^2 - d\,y^2 = 1$ for a non-square d.

$$x^2 - d\,y^2 = 1, \quad \frac{x}{y} = [a_0; a_1, \ldots, a_{k-1}]$$

input	meaning	condition
`d`	a non-square positive integer (2..1000)	

output	meaning
`x`	fundamental solution x
`y`	fundamental solution y

- **Method:** convergent of the continued fraction of sqrt(d) at the end of the first period (squared when the period is odd)
- **Assumes:** d not a perfect square; $2 \leq d \leq 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (fundamental solution x) against d (a non-square positive integer (2..1000)) from 1 to 122 on a logarithmic axis, logarithmic output axis. 28 points computed by discrete-math-ai999x; marked: the worked example, d = 61, x = 1.7663e+09.

```
discrete_math_ai999x.solve('pell_equation', d=61)
# -> x = 1.76632e+09, y = 2.26154e+08   (SOLVED)
```

```
discrete_math_ai999x.solve('pell_equation', d=2)
# -> x = 3, y = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('pell_equation', d=7)
# -> x = 8, y = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('pell_equation', d=13)
# -> x = 649, y = 180   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the fundamental solution of x^2 - 61y^2 = 1.	SOLVED	x = 1.76632e+09, y = 2.26154e+08
Solve Pell's equation for d = 7.	SOLVED	x = 8, y = 3

question	outcome	result
Find the fundamental solution of x^2 - 9y^2 = 1.	INVALID_INPUT	Pell's equation (fundamental solution): d must not be a perfect square (its square root is an integer).

Sums of two squares — `two_squares`

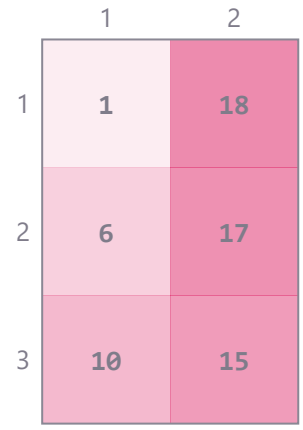
Every way to write $n = x^2 + y^2$ with $0 \le x \le y$, and r2(n), the number of ordered signed representations.

$$r_2(n) = 4\big(d_1(n) - d_3(n)\big)$$

input	meaning	condition
`n`	a whole number $(0..10^{12})$	

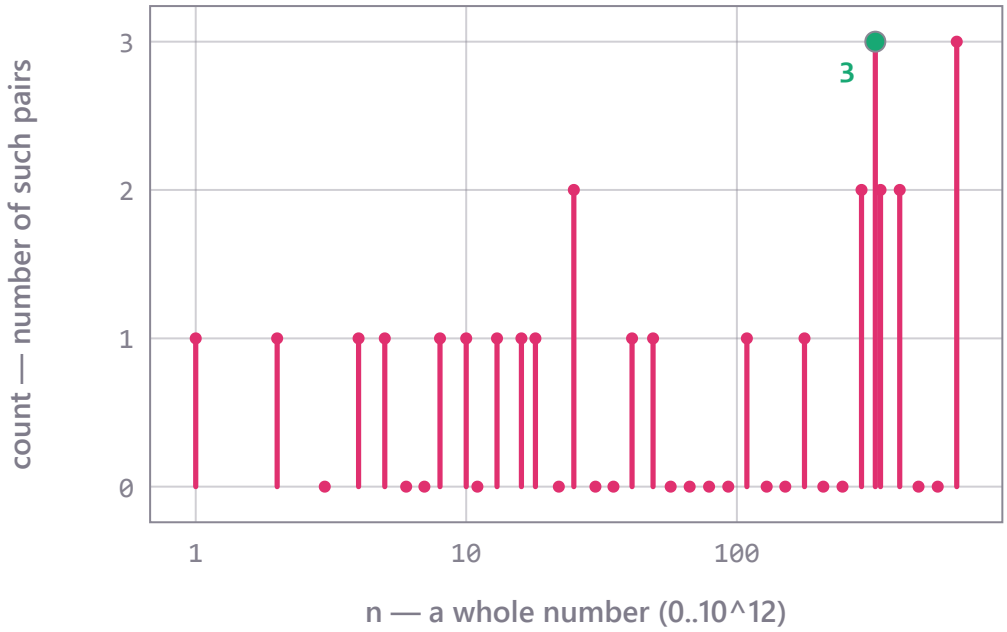
output	meaning
`representations`	pairs [x, y] with $0 \le x \le y$ and $x^2 + y^2 = n$
`count`	number of such pairs
`r2`	r2(n): ordered representations with signs

- **Method:** search x up to sqrt(n/2) with an exact integer square root; r2(n) = 4 (d1(n) - d3(n)) from the divisors
- **Assumes:** $0 \le n \le 10^{12}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



representations (returned)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): representations (returned). Returned with them by discrete-math-ai999x: count = 3, r2 = 24. Other inputs: n = 325.



count (number of such pairs) against n (a whole number (0..10^12)) from 1 to 650 on a logarithmic axis. 36 points computed by discrete-math-ai999x; marked: the worked example, n = 325, count = 3.

```
discrete_math_ai999x.solve('two_squares', n=325)
# -> representations = [[1, 18], [6, 17], [10, 15]], count = 3, r2 = 24   (SOLVED)
```

```
discrete_math_ai999x.solve('two_squares', n=25)
# -> representations = [[0, 5], [3, 4]], count = 2, r2 = 12   (SOLVED)
```

```
discrete_math_ai999x.solve('two_squares', n=3)
# -> representations = [], count = 0, r2 = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Write 325 as a sum of two squares.	SOLVED	representations = [[1, 18], [6, 17], [10, 15]], count = 3, r2 = 24
In how many ways is 25 a sum of two squares?	SOLVED	representations = [[0, 5], [3, 4]], count = 2, r2 = 12

Integer square root and perfect squares — `integer_sqrt`

floor(sqrt(n)) computed exactly, the remainder n - floor(sqrt(n))^2, and whether n is a perfect square.

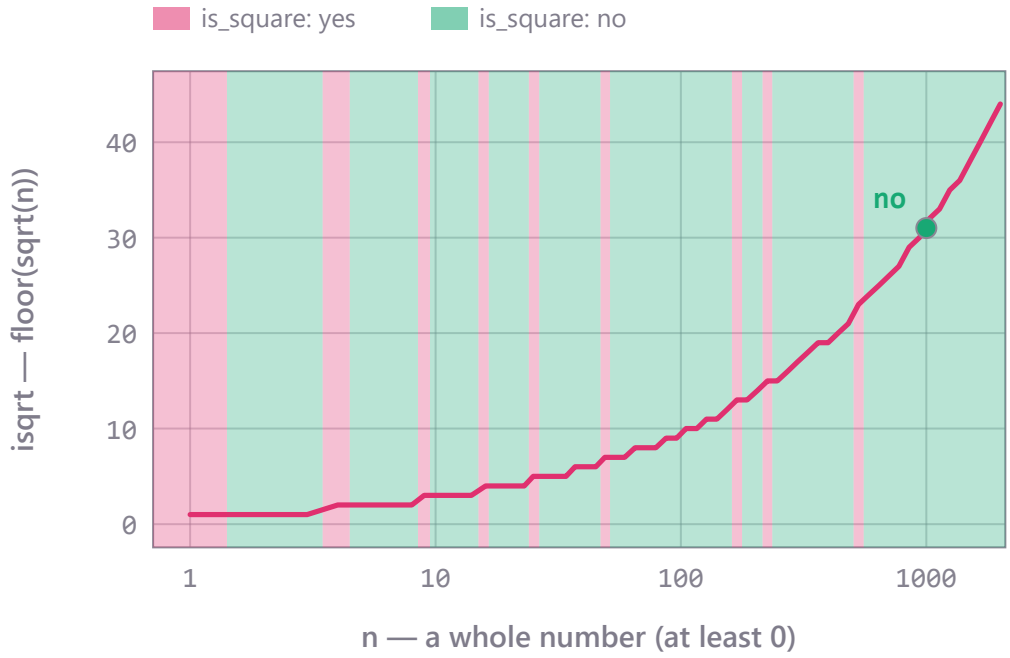
$$r = \lfloor \sqrt{n} \rfloor \iff r^2 \le n < (r+1)^2$$

input	meaning	condition
`n`	a whole number (at least 0)	

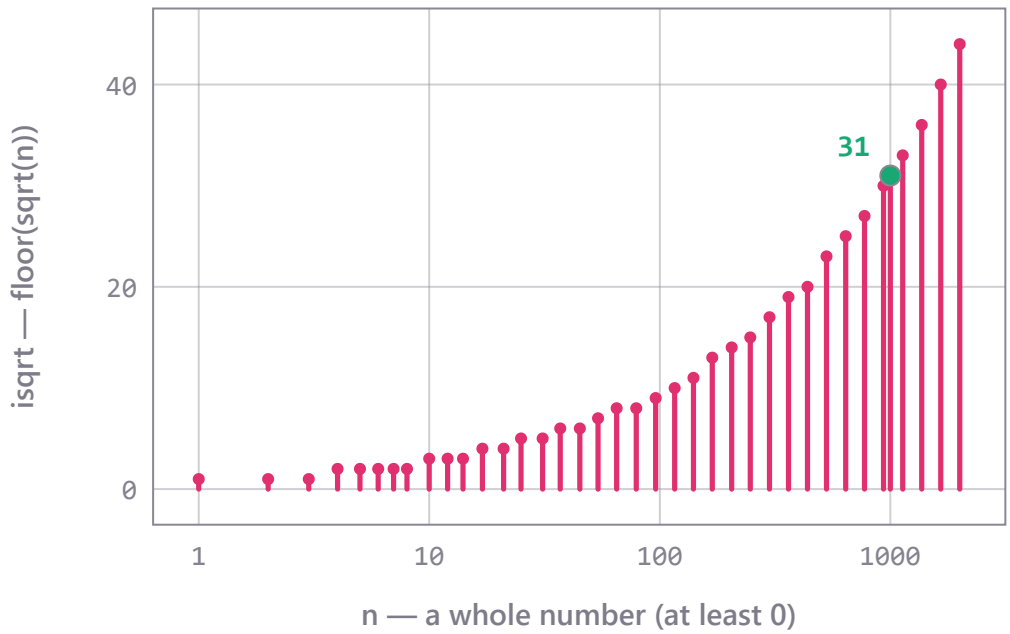
output	meaning
`isqrt`	floor(sqrt(n))
`remainder`	n - isqrt^2

output	meaning
`is_square`	true when n is a perfect square

- **Method:** exact integer Newton iteration (no floating point)
- **Assumes:** $n \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when n is a perfect square (is_square) along n (a whole number (at least 0)) from 1 to 2000 (logarithmic axis): 67 runs of discrete-math-ai999x, each band coloured by the verdict returned (yes, no); it changes near n = 1.41421, 3.4641, 4.47214, 8.48528; the line is isqrt (floor(sqrt(n))). Marked: the worked example, n = 1000 → no.



isqrt (floor(sqrt(n))) against n (a whole number (at least 0)) from 1 to 2000 on a logarithmic axis. 38 points computed by discrete-math-ai999x; marked: the worked example, n = 1000, isqrt = 31.

```
discrete_math_ai999x.solve('integer_sqrt', n=1000)
# -> isqrt = 31, remainder = 39, is_square = False   (SOLVED)
```

```
discrete_math_ai999x.solve('integer_sqrt', n=144)
# -> isqrt = 12, remainder = 0, is_square = True   (SOLVED)
```

```
discrete_math_ai999x.solve('integer_sqrt', n=0)
# -> isqrt = 0, remainder = 0, is_square = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the integer square root of 1000?	SOLVED	isqrt = 31, remainder = 39, is_square = False
Is 144 a perfect square?	SOLVED	isqrt = 12, remainder = 0, is_square = True
Is 145 a perfect square?	SOLVED	isqrt = 12, remainder = 1, is_square = False

Base (radix) conversion — `base_convert`

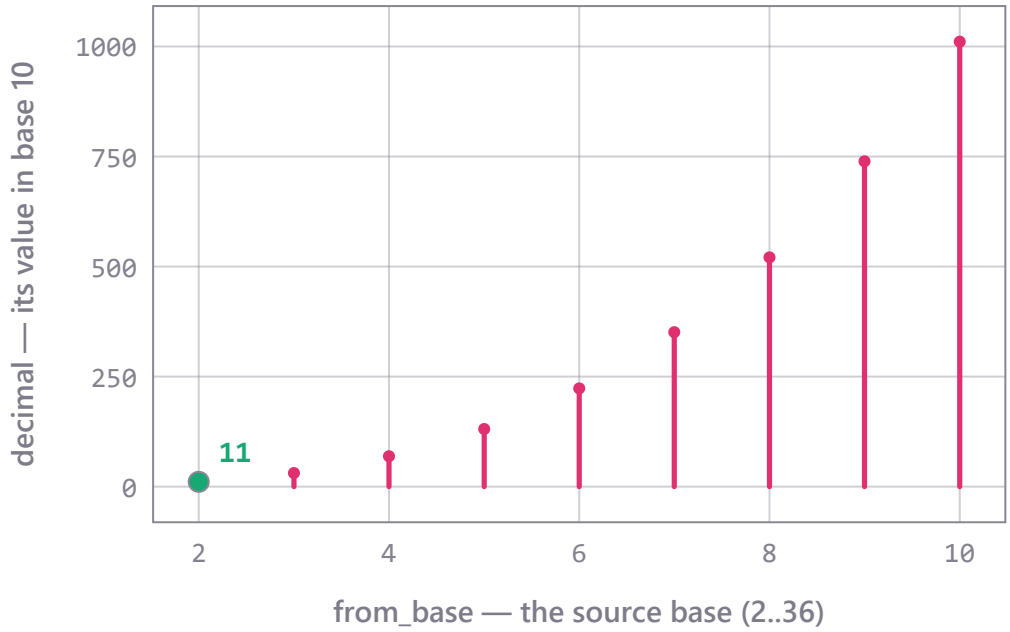
Rewrite an integer from one base (2..36) to another, with its decimal value.

$$(d_k \dots d_0)_b = \sum_i d_i \, b^i$$

input	meaning	condition
`digits`	the number written in the source base (digits $0-9$ then A-Z)	
`from_base`	the source base (2..36)	
`to_base`	the target base (2..36)	

output	meaning
`result`	the number written in the target base (upper-case digits)
`decimal`	its value in base 10

- **Method:** Horner evaluation in the source base, then repeated division by the target base
- **Assumes:** 2 <= bases <= 36; every digit is valid in the source base; a leading minus sign is kept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



decimal (its value in base 10) against from_base (the source base (2..36)) from 1 to 10. Other inputs as in the worked example: digits = 1011, to_base = 10. 9 points computed by discrete-math-ai999x; marked: the worked example, from_base = 2, decimal = 11.

```
discrete_math_ai999x.solve('base_convert', digits='1011', from_base=2, to_base=10)
# -> result = 11, decimal = 11   (SOLVED)
```

```
discrete_math_ai999x.solve('base_convert', digits='255', from_base=10, to_base=16)
# -> result = FF, decimal = 255   (SOLVED)
```

```
discrete_math_ai999x.solve('base_convert', digits='ff', from_base=16, to_base=2)
# -> result = 11111111, decimal = 255   (SOLVED)
```

```
discrete_math_ai999x.solve('base_convert', digits='-100', from_base=10, to_base=7)
# -> result = -202, decimal = -100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 1011 from base 2 to base 10.	SOLVED	result = 11, decimal = 11
Convert 255 to hexadecimal.	SOLVED	result = FF, decimal = 255
Convert FF from hexadecimal to binary.	SOLVED	result = 11111111, decimal = 255
Convert 1234 from base 10 to base 7.	SOLVED	result = 3412, decimal = 1234
Convert 129 from base 2 to base 10.	INVALID_INPUT	Base (radix) conversion: '2' is not a digit in base 2.

Two's complement representation — `twos_complement`

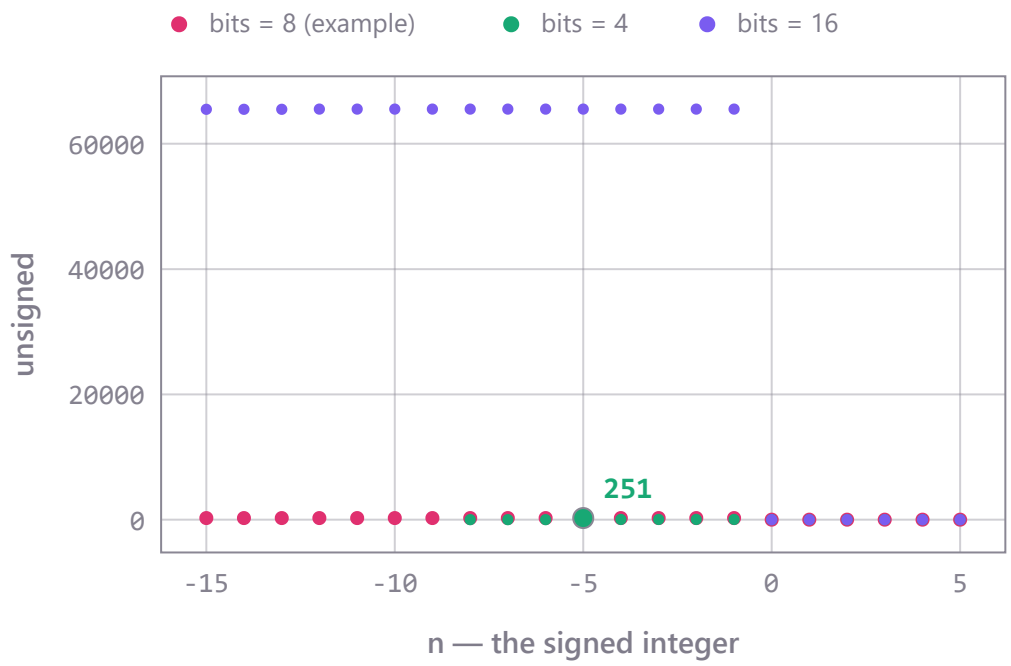
The b-bit two's complement bit pattern of a signed integer, and that pattern read as an unsigned number.

$$\text{pattern}(n) = n \bmod 2^b$$

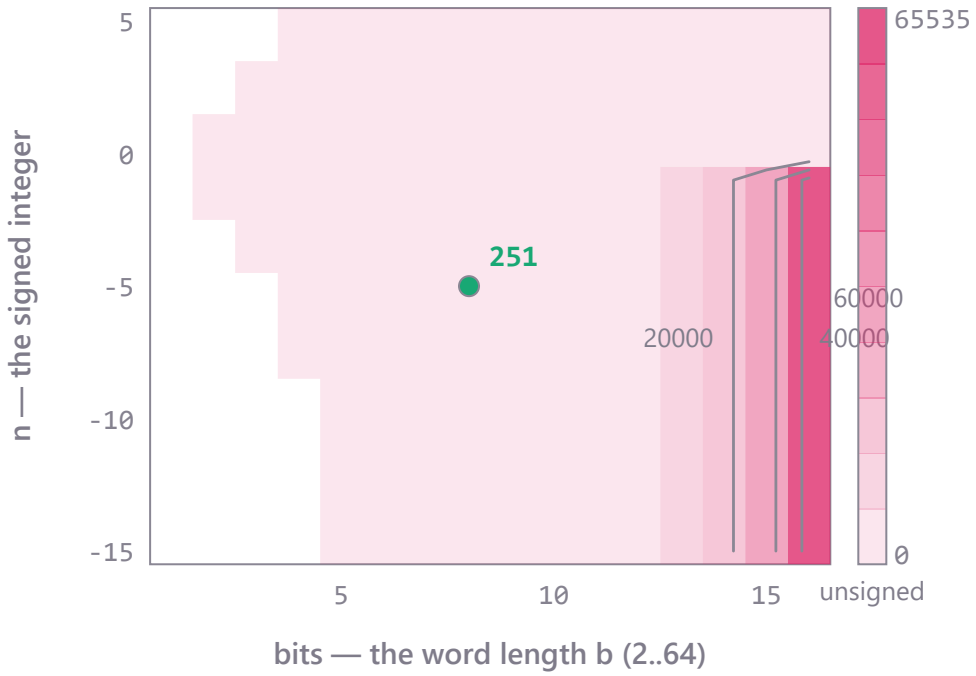
input	meaning	condition
`n`	the signed integer	
`bits`	the word length b (2..64)	

output	meaning
`bits_text`	the b-bit pattern
`unsigned`	the pattern read as an unsigned integer (n mod 2^b)

- **Method:** non-negative: binary padded to b bits; negative: invert the bits of |n| and add 1
- **Assumes:** $-2^{b-1} \leq n < 2^{b-1}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



unsigned (the pattern read as an unsigned integer (n mod 2^b)) against n (the signed integer) from -15 to 5. One curve for each value of bits (the word length b (2..64)): 8, 4, 16. 21 points per curve computed by discrete-math-ai999x; marked: the worked example, n = -5, unsigned = 251.



unsigned (the pattern read as an unsigned integer (n mod 2^b)) over bits (the word length b (2..64)) from 1 to 16 and n (the signed integer) from -15 to 5: each of the 16 × 21 cells is one run of discrete-math-ai999x, shaded by its value (light 0 to dark 65535); iso-lines at unsigned = 20000, 40000, 60000; blank cells: inputs the capability refuses. Marked: the worked example, bits = 8, n = -5, unsigned = 251.

```
discrete_math_ai999x.solve('twos_complement', n=-5, bits=8)
# -> bits_text = 11111011, unsigned = 251   (SOLVED)
```

```
discrete_math_ai999x.solve('twos_complement', n=-128, bits=8)
# -> bits_text = 10000000, unsigned = 128   (SOLVED)
```

```
discrete_math_ai999x.solve('twos_complement', n=5, bits=4)
# -> bits_text = 0101, unsigned = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 8-bit two's complement of -5?	SOLVED	bits_text = 11111011, unsigned = 251
Write -128 in 8-bit two's complement.	SOLVED	bits_text = 10000000, unsigned = 128
What is the 8-bit two's complement of -200?	INVALID_INPUT	Two's complement representation: -200 does not fit in 8-bit two's complement (range -128 to 127).

Combinations C(n, k) — `combinations`

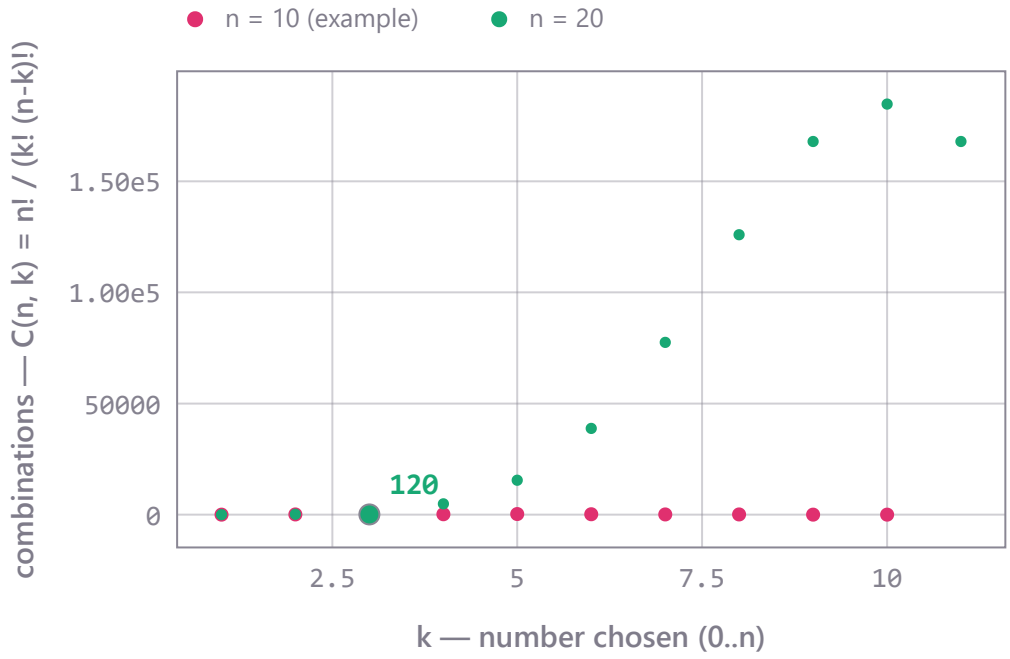
The number of ways to choose k of n distinct items when order does not matter (exact integer).

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

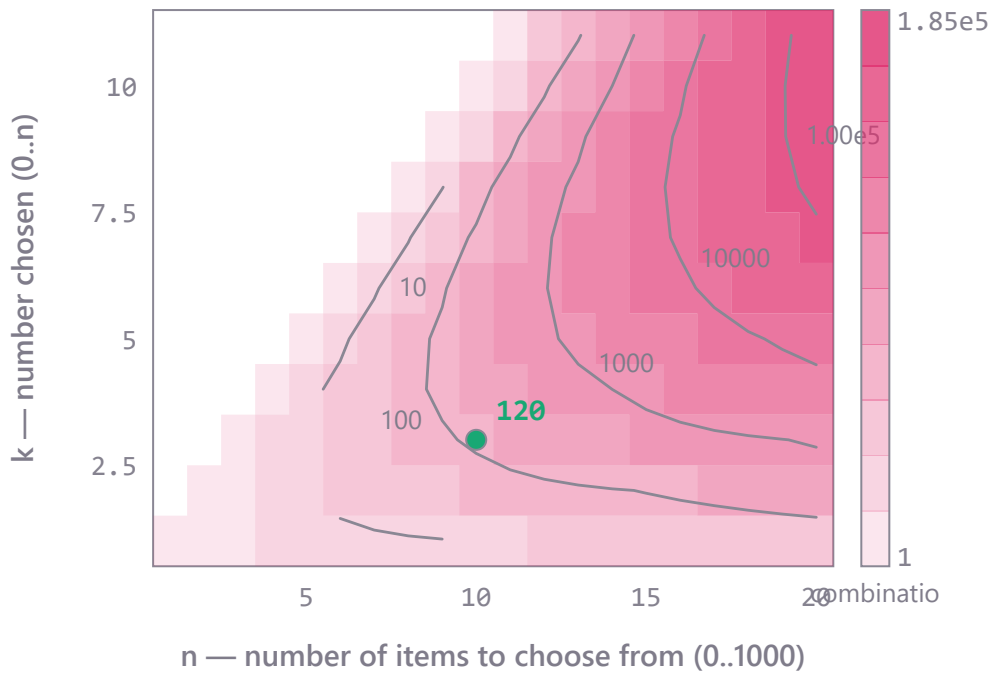
input	meaning	condition
`n`	number of items to choose from (0..1000)	
`k`	number chosen (0..n)	

output	meaning
`combinations`	C(n, k) = n! / (k! (n-k)!)

- **Method:** multiplicative formula with exact integer division at every step
- **Assumes:** $0 \le k \le n \le 1000$; result at most 10^{300}
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



combinations (C(n, k) = n! / (k! (n-k)!)) against k (number chosen (0..n)) from 1 to 11. One curve for each value of n (number of items to choose from (0..1000)): 10, 20. 10 points per curve computed by discrete-math-ai999x; marked: the worked example, k = 3, combinations = 120.



combinations (C(n, k) = n! / (k! (n-k)!)) over n (number of items to choose from (0..1000)) from 1 to 20 and k (number chosen (0..n)) from 1 to 11: each of the 20 × 11 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 184756, logarithmic shading); iso-lines at combinations = 10, 100, 1000, 10000, 1.00e5; blank cells: inputs the capability refuses. Marked: the worked example, n = 10, k = 3, combinations = 120.

```
discrete_math_ai999x.solve('combinations', n=10, k=3)
# -> combinations = 120   (SOLVED)
```

```
discrete_math_ai999x.solve('combinations', n=52, k=5)
# -> combinations = 2.59896e+06   (SOLVED)
```

```
discrete_math_ai999x.solve('combinations', n=0, k=0)
# -> combinations = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many ways are there to choose 3 books from 10?	SOLVED	combinations = 120
Compute C(52, 5).	SOLVED	combinations = 2.59896e+06
What is 10 choose 3?	SOLVED	combinations = 120
Compute C(4, 7).	INVALID_INPUT	Combinations C(n, k): k must not exceed n.

Permutations P(n, k) — `permutations`

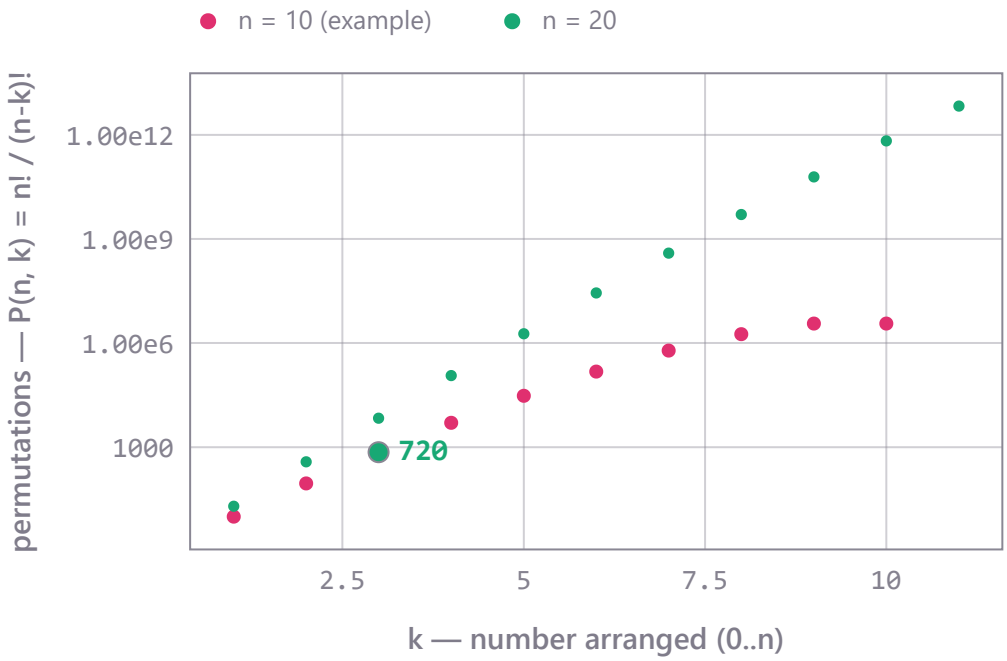
The number of ordered arrangements of k of n distinct items (exact integer); P(n, n) = n!.

$$P(n,k) = \frac{n!}{(n-k)!}$$

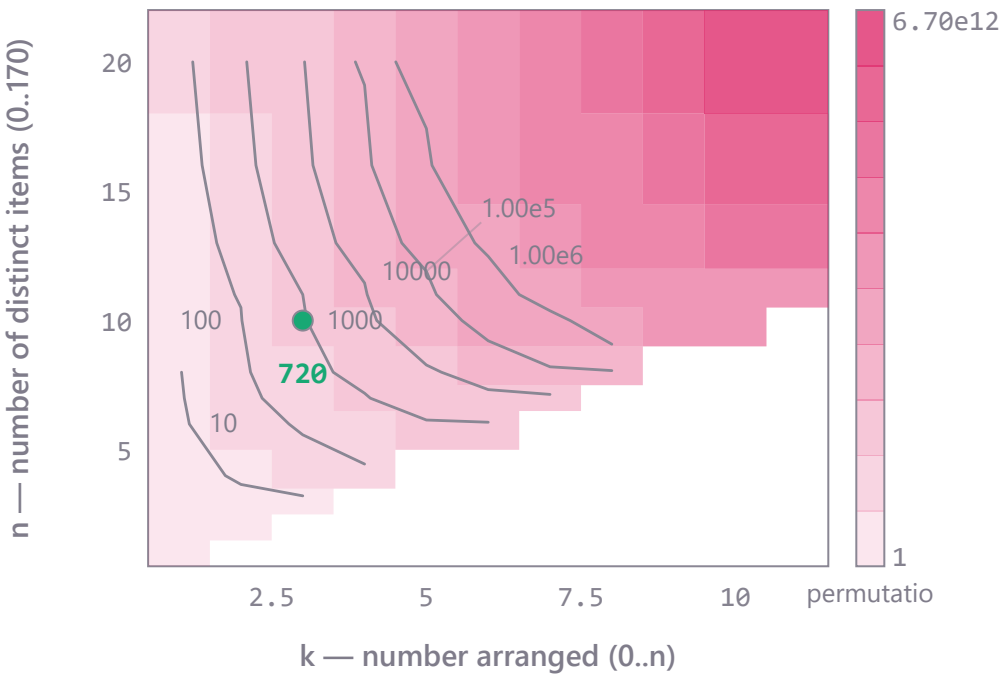
input	meaning	condition
`n`	number of distinct items (0..170)	
`k`	number arranged (0..n)	

output	meaning
`permutations`	P(n, k) = n! / (n-k)!

- **Method:** falling factorial n $(n-1)$ … $(n-k+1)$
- **Assumes:** $0 \le k \le n \le 170$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



permutations (P(n, k) = n! / (n-k)!) against k (number arranged (0..n)) from 1 to 11, logarithmic output axis. One curve for each value of n (number of distinct items (0..170)): 10, 20. 10 points per curve computed by discrete-math-ai999x; marked: the worked example, k = 3, permutations = 720.



permutations (P(n, k) = n! / (n-k)!) over k (number arranged (0..n)) from 1 to 11 and n (number of distinct items (0..170)) from 1 to 20: each of the 11 × 12 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 6.7044e+12, logarithmic shading); iso-lines at permutations = 10, 100, 1000, 10000, 1.00e5, 1.00e6; blank cells: inputs the capability refuses. Marked: the worked example, k = 3, n = 10, permutations = 720.

```
discrete_math_ai999x.solve('permutations', n=10, k=3)
# -> permutations = 720   (SOLVED)
```

```
discrete_math_ai999x.solve('permutations', n=5, k=5)
# -> permutations = 120   (SOLVED)
```

```
discrete_math_ai999x.solve('permutations', n=7, k=0)
# -> permutations = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute P(10, 3).	SOLVED	permutations = 720
In how many ways can we arrange 3 books out of 10?	SOLVED	permutations = 720
In how many ways can 5 people be arranged in a row?	SOLVED	permutations = 120
Compute P(3, 5).	INVALID_INPUT	Permutations P(n, k): k must not exceed n.

Sequences with repetition (n^k) — `sequences_with_repetition`

The number of length-k sequences over n symbols when symbols may repeat.

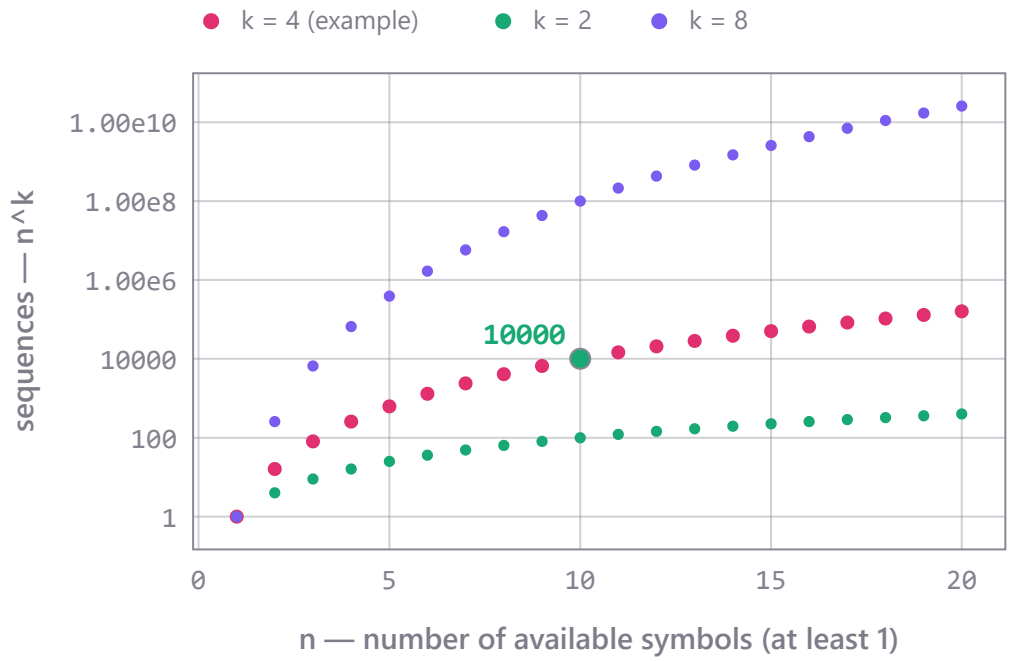
$$n^k$$

input	meaning	condition
n	number of available symbols (at least 1)	

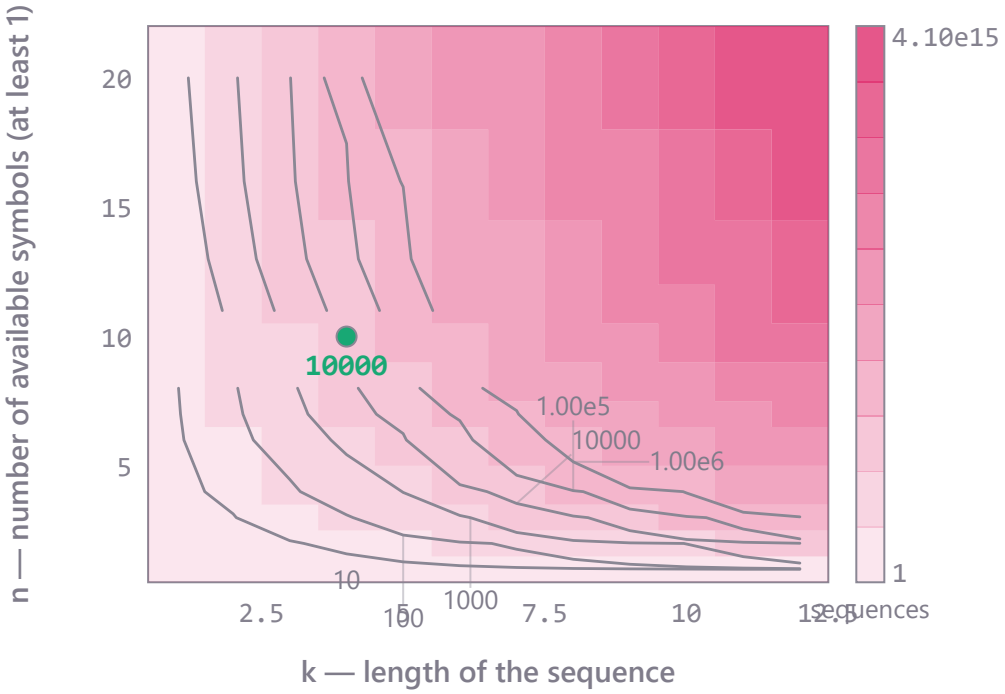
input	meaning	condition
`k`	length of the sequence	

output	meaning
`sequences`	n^k

- **Method:** multiplication principle
- **Assumes:** $n \geq 1$; result at most 10^{300}
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sequences (n^k) against n (number of available symbols (at least 1)) from 1 to 20, logarithmic output axis. One curve for each value of k (length of the sequence): 4, 2, 8. 20 points per curve computed by discrete-math-ai999x; marked: the worked example, n = 10, sequences = 10000.



sequences (n^k) over k (length of the sequence) from 1 to 12 and n (number of available symbols (at least 1)) from 1 to 20: each of the 12 × 12 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 4.0960e+15, logarithmic shading); iso-lines at sequences = 10, 100, 1000, 10000, 1.00e5, 1.00e6. Marked: the worked example, k = 4, n = 10, sequences = 10000.

```
discrete_math_ai999x.solve('sequences_with_repetition', n=10, k=4)
# -> sequences = 10000   (SOLVED)
```

```
discrete_math_ai999x.solve('sequences_with_repetition', n=26, k=5)
# -> sequences = 1.18814e+07   (SOLVED)
```

```
discrete_math_ai999x.solve('sequences_with_repetition', n=2, k=10)
# -> sequences = 1024   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many sequences of length 4 can be formed from 10 digits with repetition allowed?	SOLVED	sequences = 10000
How many 5-letter strings can be made from 26 letters with repetition?	SOLVED	sequences = 1.18814e+07

Combinations with repetition (multisets) — `multisets`

The number of ways to choose k items from n types when types may repeat and order does not matter.

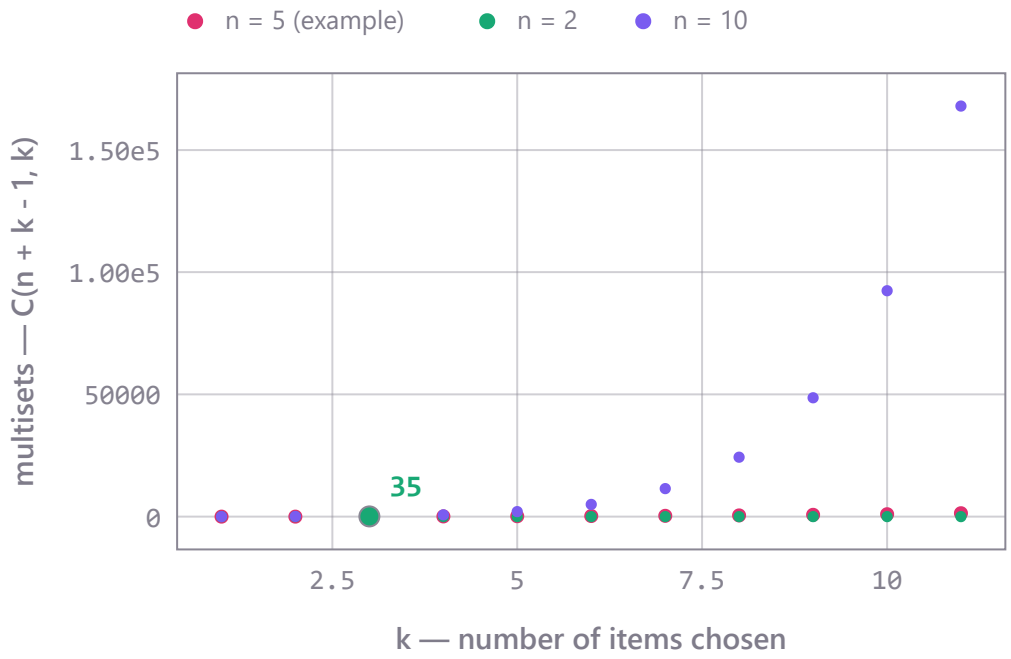
$$\left(\binom{n}{k}\right) = \binom{n+k-1}{k}$$

input	meaning	condition
n	number of types (at least 1)	

input	meaning	condition
`k`	number of items chosen	

output	meaning
`multisets`	C($n + k - 1$, k)

- **Method:** stars and bars
- **Assumes:** $n \geq 1$; result at most 10^{300}
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



multisets (C(n + k - 1, k)) against k (number of items chosen) from 1 to 11. One curve for each value of n (number of types (at least 1)): 5, 2, 10. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, k = 3, multisets = 35.



multisets (C(n + k - 1, k)) over k (number of items chosen) from 1 to 11 and n (number of types (at least 1)) from 1 to 13: each of the 11 × 13 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 1.35208e+06, logarithmic shading); iso-lines at multisets = 10, 100, 1000, 10000, 1.00e5. Marked: the worked example, k = 3, n = 5, multisets = 35.

```
discrete_math_ai999x.solve('multisets', n=5, k=3)
# -> multisets = 35   (SOLVED)
```

```
discrete_math_ai999x.solve('multisets', n=6, k=4)
# -> multisets = 126   (SOLVED)
```

```
discrete_math_ai999x.solve('multisets', n=1, k=7)
# -> multisets = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many ways can we choose 3 scoops from 5 flavours with repetition allowed?	SOLVED	multisets = 35
How many multisets of size 4 from 6 types are there?	SOLVED	multisets = 126

Multinomial coefficient — `multinomial`

n! / (k1! k2! ... km!): ways to split n distinct items into labelled groups of the given sizes.

$$\binom{n}{k_1, \ldots, k_m} = \frac{n!}{k_1! \cdots k_m!}$$

input	meaning	condition
`counts`	the group sizes k1, ..., km (whole numbers)	
`total`	the stated total n (optional; must equal the sum of the sizes)	

output	meaning
`coefficient`	the multinomial coefficient
`total`	$n = k_1 + \ldots + km$

- **Method:** n! divided by each k_i! (exact integers)
- **Assumes:** sizes are whole numbers >= 0 adding up to at most 170; a stated total must equal their sum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Multinomial coefficient as inputs and results: stated counts = 2, 3, 5; returned by discrete-math-ai999x: coefficient = 2520, total = 10.

```
discrete_math_ai999x.solve('multinomial', counts='2, 3, 5')
# -> coefficient = 2520, total = 10   (SOLVED)
```

```
discrete_math_ai999x.solve('multinomial', counts='1, 4, 4, 2')
# -> coefficient = 34650, total = 11   (SOLVED)
```

```
discrete_math_ai999x.solve('multinomial', counts=[3, 3], total=6)
# -> coefficient = 20, total = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the multinomial coefficient for 2, 3, 5.	SOLVED	coefficient = 2520, total = 10
In how many ways can 10 people be split into groups of sizes 2, 3 and 5?	SOLVED	coefficient = 2520, total = 10
In how many ways can 12 people be split into groups of sizes 2, 3 and 5?	INVALID_INPUT	Multinomial coefficient: the group sizes add up to 10, not to the stated total 12.

Distinct arrangements of the letters of a word — `word_arrangements`

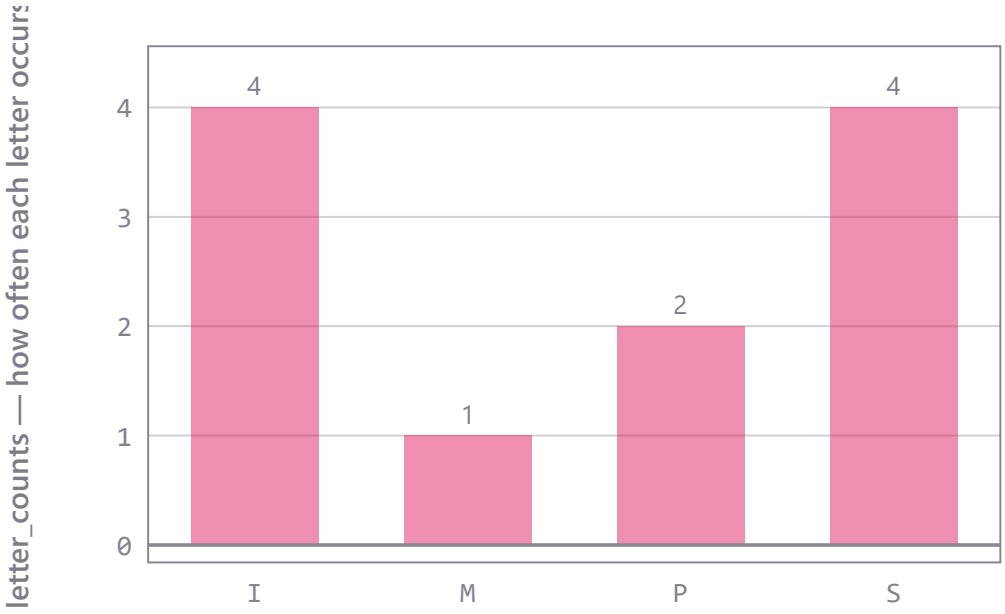
The number of distinct orderings of a word's letters, repeated letters counted once (multiset permutations).

$$\frac{n!}{n_1!\,n_2!\cdots n_r!}$$

input	meaning	condition
`word`	the word (letters A-Z; case is ignored)	

output	meaning
`arrangements`	n! / (n1! n2! ...) over the letter multiplicities
`length`	number of letters
`letter_counts`	how often each letter occurs

- **Method:** product of binomial coefficients, placing one letter type at a time
- **Assumes:** letters A-Z only (upper and lower case are the same letter); at most 170 letters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



how often each letter occurs (letter_counts) returned by discrete-math-ai999x for the worked example (word = MISSISSIPPI), one bar per entry: I = 4, M = 1, P = 2, S = 4.

```
discrete_math_ai999x.solve('word_arrangements', word='MISSISSIPPI')
# -> arrangements = 34650, length = 11, letter_counts = {'I': 4, 'M': 1, 'P': 2, 'S': 4}   (SOLVED)
```

```
discrete_math_ai999x.solve('word_arrangements', word='banana')
# -> arrangements = 60, length = 6, letter_counts = {'A': 3, 'B': 1, 'N': 2}   (SOLVED)
```

```
discrete_math_ai999x.solve('word_arrangements', word='ABC')
# -> arrangements = 6, length = 3, letter_counts = {'A': 1, 'B': 1, 'C': 1}   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many distinct arrangements of the letters of MISSISSIPPI are there?	SOLVED	arrangements = 34650, length = 11, letter_counts = {'I': 4, 'M': 1, 'P': 2, 'S': 4}
How many anagrams of BANANA are there?	SOLVED	arrangements = 60, length = 6, letter_counts = {'A': 3, 'B': 1, 'N': 2}
In how many ways can the letters of the word STATISTICS be arranged?	SOLVED	arrangements = 50400, length = 10, letter_counts = {'A': 1, 'C': 1, 'I': 2, 'S': 3, 'T': 3}

Stirling numbers of the second kind — `stirling2`

S(n, k): ways to partition an n-element set into k non-empty unlabelled blocks; k! S(n, k) onto functions.

$$S(n,k) = k\,S(n-1,k) + S(n-1,k-1) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^j\binom{k}{j}(k-j)^n$$

input	meaning	condition
`n`	size of the set (0..100)	
`k`	number of blocks (0..100)	

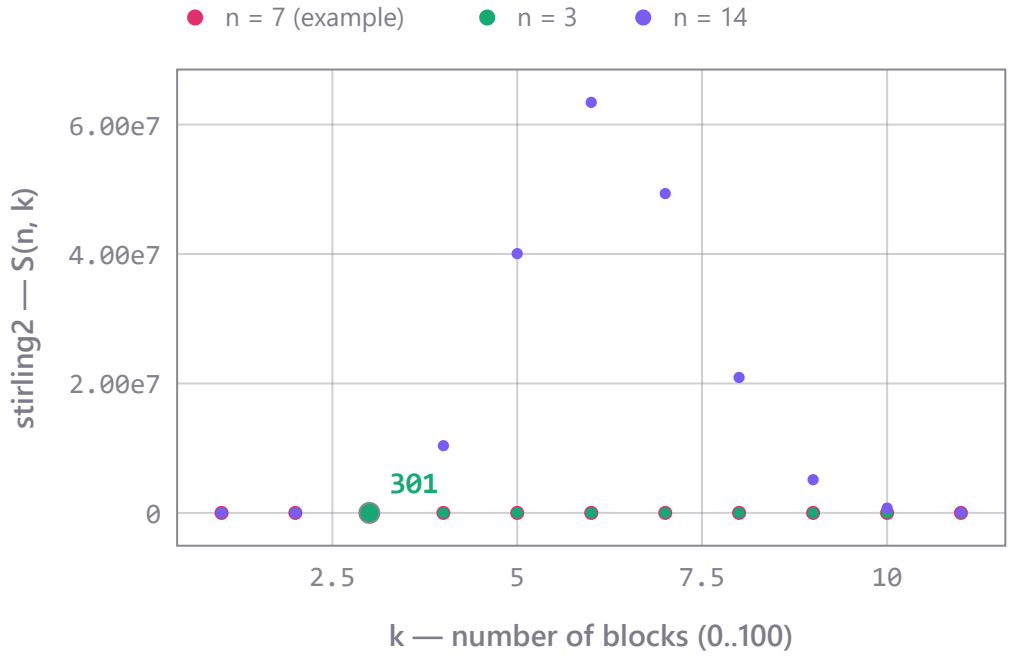
output	meaning
`stirling2`	S(n, k)
`surjections`	k! S(n, k): onto functions from an n-set to a k-set

- **Method:** recurrence S(n, k) = k S($n-1$, k) + S($n-1$, k-1)
- **Assumes:** $0 \le n, k \le 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	1	2	3	4	5	6	7
1	1	0	0	0	0	0	0
2	1	1	0	0	0	0	0
3	1	3	1	0	0	0	0
4	1	7	6	1	0	0	0
5	1	15	25	10	1	0	0
6	1	31	90	65	15	1	0
7	1	63	301	350	140	21	1

S(n, k): rows n = 1 … 7, columns k = 1 … 7

Stirling numbers of the second kind S(n, k) for n, k = 1 … 7 (row n, column k), each computed by the package: the ways to split n labelled objects into k non-empty groups; row 7 sums to the Bell number B(7) = 877, also computed by the package.



stirling2 (S(n, k)) against k (number of blocks (0..100)) from 1 to 11. One curve for each value of n (size of the set (0..100)): 7, 3, 14. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, k = 3, stirling2 = 301.

```
discrete_math_ai999x.solve('stirling2', n=7, k=3)
# -> stirling2 = 301, surjections = 1806   (SOLVED)
```

```
discrete_math_ai999x.solve('stirling2', n=5, k=3)
# -> stirling2 = 25, surjections = 150   (SOLVED)
```

```
discrete_math_ai999x.solve('stirling2', n=0, k=0)
# -> stirling2 = 1, surjections = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('stirling2', n=3, k=5)
# -> stirling2 = 0, surjections = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Stirling number of the second kind S(7, 3).	SOLVED	stirling2 = 301, surjections = 1806
In how many ways can we partition a set of 7 elements into 3 non-empty subsets?	SOLVED	stirling2 = 301, surjections = 1806
How many onto functions are there from a 5-element set to a 3-element set?	SOLVED	stirling2 = 25, surjections = 150

Stirling numbers of the first kind — `stirling1`

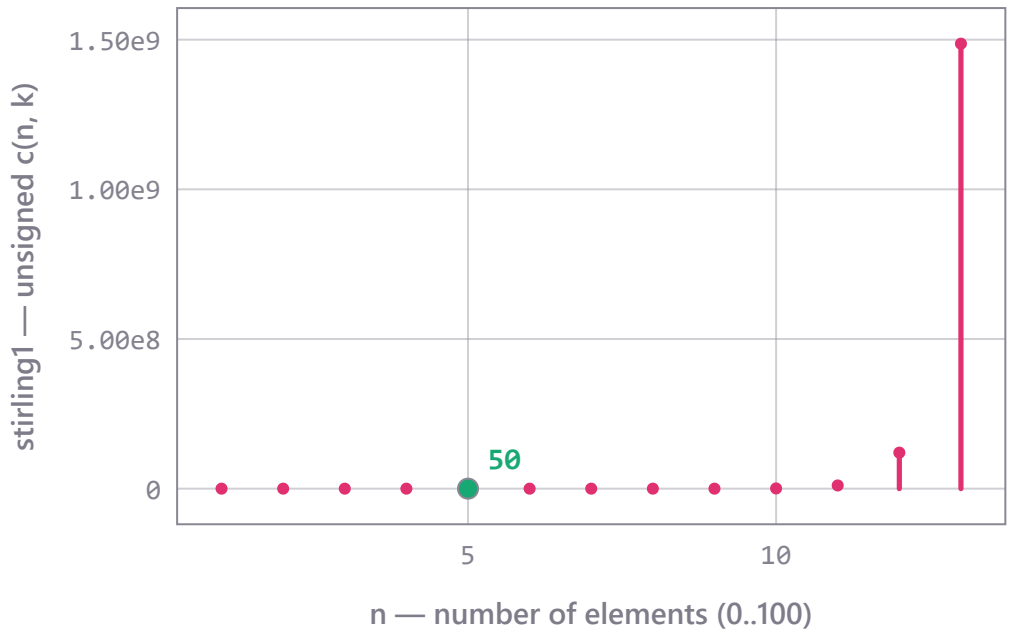
c(n, k): permutations of n elements with exactly k cycles (unsigned), and the signed s(n, k).

$$c(n,k) = (n-1)\,c(n-1,k) + c(n-1,k-1), \quad x^{\overline{n}} = \sum_{k} c(n,k)\,x^k$$

input	meaning	condition
`n`	number of elements (0..100)	
`k`	number of cycles (0..100)	

output	meaning
`stirling1`	unsigned c(n, k)
`signed`	signed s(n, k) = (-1)^(n-k) c(n, k)

- **Method:** recurrence c(n, k) = (n-1) c($n-1$, k) + c($n-1$, k-1)
- **Assumes:** $0 \le n, k \le 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



stirling1 (unsigned c(n, k)) against n (number of elements (0..100)) from 1 to 13. Other inputs as in the worked example: k = 2. 13 points computed by discrete-math-ai999x; marked: the worked example, n = 5, stirling1 = 50.

```
discrete_math_ai999x.solve('stirling1', n=5, k=2)
# -> stirling1 = 50, signed = -50   (SOLVED)
```

```
discrete_math_ai999x.solve('stirling1', n=6, k=2)
# -> stirling1 = 274, signed = 274   (SOLVED)
```

```
discrete_math_ai999x.solve('stirling1', n=4, k=4)
# -> stirling1 = 1, signed = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the Stirling number of the first kind c(5, 2).	SOLVED	stirling1 = 50, signed = -50
How many permutations of 6 elements have exactly 2 cycles?	SOLVED	stirling1 = 274, signed = 274

Bell numbers — `bell`

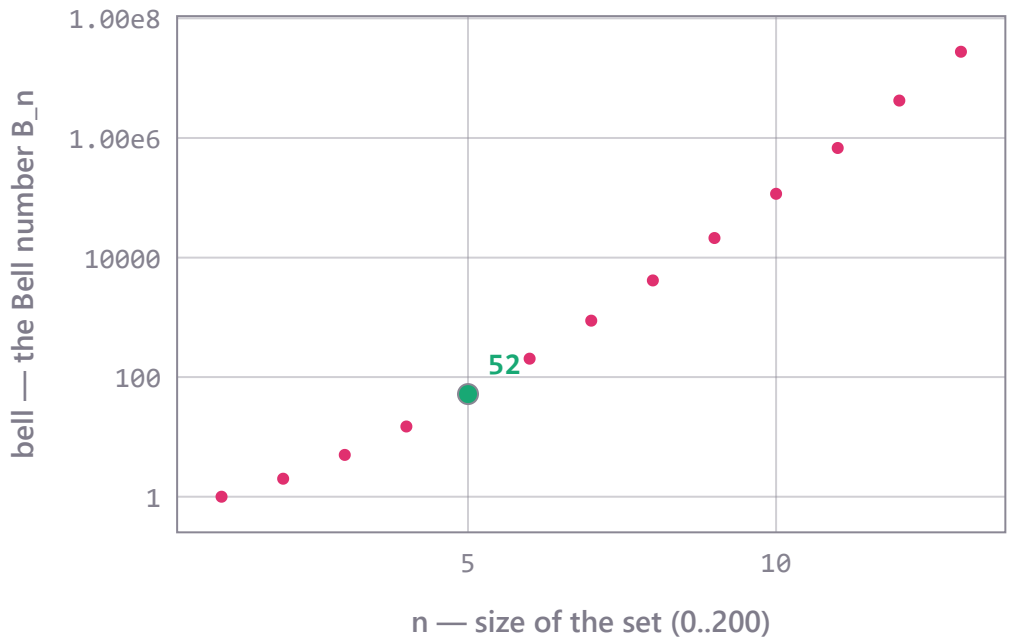
B_n: the number of partitions of an n-element set into any number of non-empty blocks.

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$$

input	meaning	condition
`n`	size of the set (0..200)	

output	meaning
`bell`	the Bell number B_n

- **Method:** Bell triangle (Aitken's array)
- **Assumes:** $0 \leq n \leq 200$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



bell (the Bell number B_n) against n (size of the set (0..200)) from 1 to 13, logarithmic output axis. 13 points computed by discrete-math-ai999x; marked: the worked example, n = 5, bell = 52.

```
discrete_math_ai999x.solve('bell', n=5)
# -> bell = 52   (SOLVED)
```

```
discrete_math_ai999x.solve('bell', n=0)
# -> bell = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('bell', n=10)
# -> bell = 115975   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 5th Bell number?	SOLVED	bell = 52
In how many ways can we partition a set of 4 elements?	SOLVED	bell = 15

Catalan numbers — `catalan`

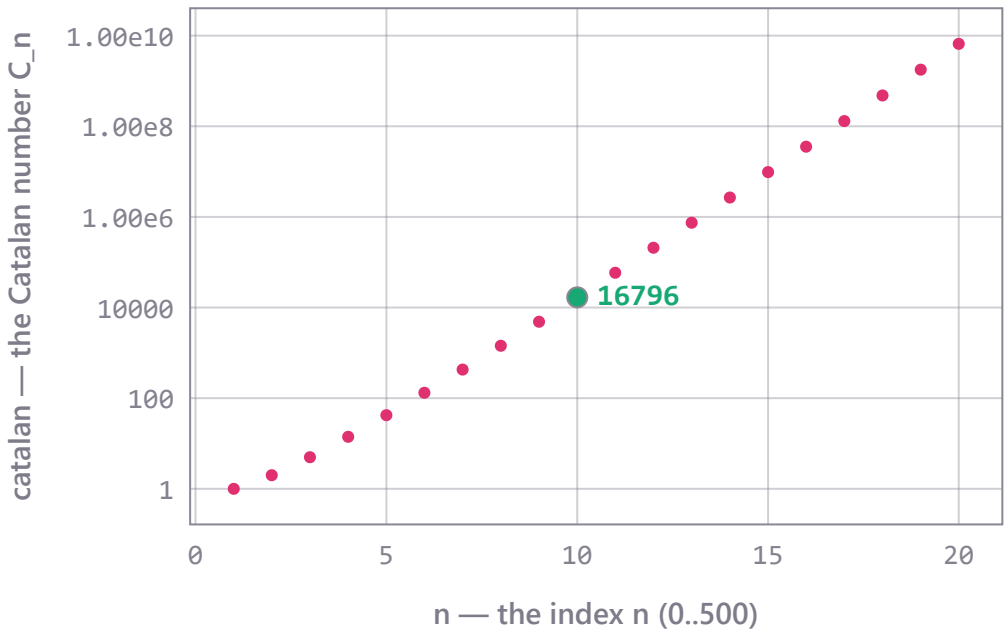
C_n: balanced parenthesizations with n pairs, triangulations of an (n+2)-gon, full binary trees with $n+1$ leaves.

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

input	meaning	condition
`n`	the index n (0..500)	

output	meaning
`catalan`	the Catalan number C_n

- **Method:** closed form binomial(2n, n)/(n + 1)
- **Assumes:** $0 \leq n \leq 500$; a convex polygon with s sides has C_(s-2) triangulations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



catalan (the Catalan number C_n) against n (the index n (0..500)) from 1 to 20, logarithmic output axis. 20 points computed by discrete-math-ai999x; marked: the worked example, n = 10, catalan = 16796.

```
discrete_math_ai999x.solve('catalan', n=10)
# -> catalan = 16796   (SOLVED)
```

```
discrete_math_ai999x.solve('catalan', n=4)
# -> catalan = 14   (SOLVED)
```

```
discrete_math_ai999x.solve('catalan', n=0)
# -> catalan = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 10th Catalan number?	SOLVED	catalan = 16796
How many balanced strings of parentheses with 4 pairs are there?	SOLVED	catalan = 14
How many triangulations of a convex polygon with 8 sides are there?	SOLVED	catalan = 132

Fibonacci numbers — `fibonacci`

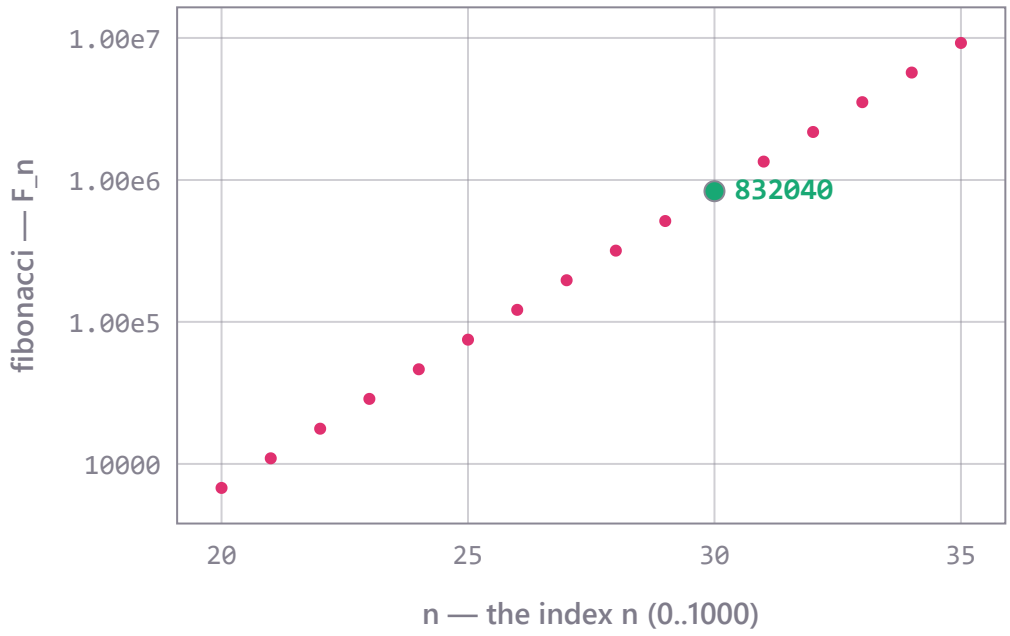
F_n with $F_0 = 0$, $F_1 = 1$, Missing superscript or subscript argument(n-1) + F_(n-2) (exact integer).

$$F_n = F_{n-1} + F_{n-2},\ F_0 = 0,\ F_1 = 1$$

input	meaning	condition
`n`	the index n (0..1000)	

output	meaning
`fibonacci`	F_n

- **Method:** iteration of the recurrence
- **Assumes:** $0 \le n \le 1000$; $F_0 = 0$, $F_1 = 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fibonacci (F_n) against n (the index n (0..1000)) from 20 to 35, logarithmic output axis. 16 points computed by discrete-math-ai999x; marked: the worked example, n = 30, fibonacci = 832040.

```
discrete_math_ai999x.solve('fibonacci', n=30)
# -> fibonacci = 832040   (SOLVED)
```

```
discrete_math_ai999x.solve('fibonacci', n=0)
# -> fibonacci = 0   (SOLVED)
```

```
discrete_math_ai999x.solve('fibonacci', n=100)
# -> fibonacci = 3.54225e+20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 30th Fibonacci number?	SOLVED	fibonacci = 832040
Compute the Fibonacci number F(100).	SOLVED	fibonacci = 3.54225e+20
Compute the Fibonacci number F(5000).	INVALID_INPUT	n: must be at most 1000, got 5000.

Lucas numbers — `lucas`

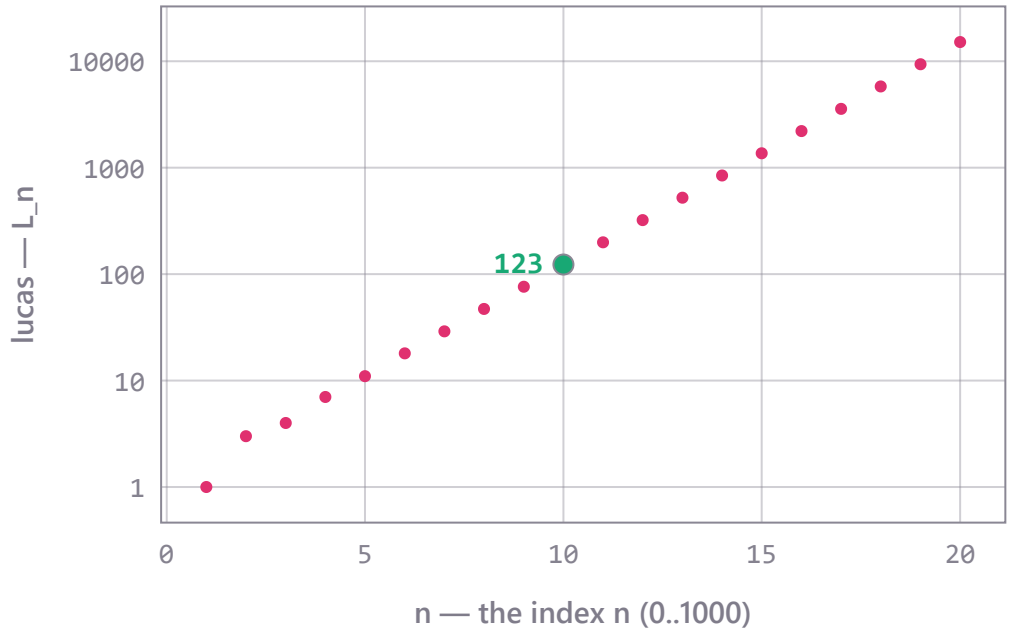
L_n with $L_0 = 2$, $L_1 = 1$, Missing superscript or subscript argument(n-1) + L_(n-2) (exact integer).

$$L_n = L_{n-1} + L_{n-2},\ L_0 = 2,\ L_1 = 1$$

input	meaning	condition
`n`	the index n (0..1000)	

output	meaning
`lucas`	L_n

- **Method:** iteration of the recurrence
- **Assumes:** $0 \le n \le 1000$; $L_0 = 2$, $L_1 = 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



lucas (L_n) against n (the index n (0..1000)) from 1 to 20, logarithmic output axis. 20 points computed by discrete-math-ai999x; marked: the worked example, n = 10, lucas = 123.

```
discrete_math_ai999x.solve('lucas', n=10)
# -> lucas = 123   (SOLVED)
```

```
discrete_math_ai999x.solve('lucas', n=0)
# -> lucas = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('lucas', n=20)
# -> lucas = 15127   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the 10th Lucas number?	SOLVED	lucas = 123
Compute the Lucas number L(20).	SOLVED	lucas = 15127

Derangements — `derangements`

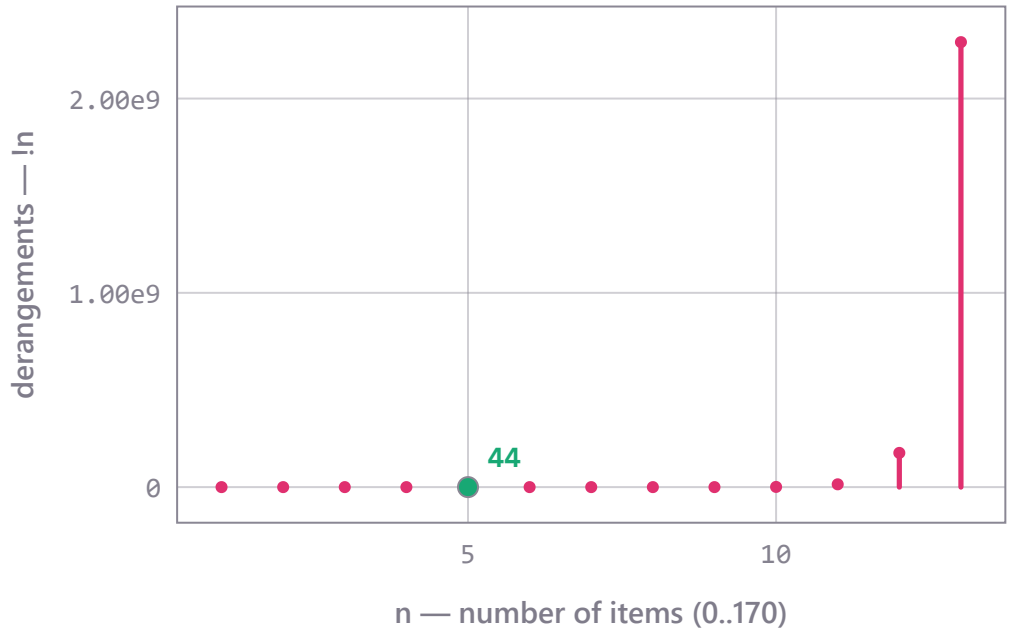
!n: permutations of n items with no item in its original place, and the probability !n/n!.

$$!n = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!} = (n-1)\big(!(n-1)+!(n-2)\big)$$

input	meaning	condition
`n`	number of items (0..170)	

output	meaning
`derangements`	!n
`probability`	!n / n! (tends to 1/e)

- **Method:** recurrence D(n) = (n-1)(D(n-1) + D(n-2))
- **Assumes:** $0 \le n \le 170$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



derangements (!n) against n (number of items (0..170)) from 1 to 13. 13 points computed by discrete-math-ai999x; marked: the worked example, n = 5, derangements = 44.

```
discrete_math_ai999x.solve('derangements', n=5)
# -> derangements = 44, probability = 0.366667   (SOLVED)
```

```
discrete_math_ai999x.solve('derangements', n=0)
# -> derangements = 1, probability = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('derangements', n=1)
# -> derangements = 0, probability = 0   (SOLVED)
```

```
discrete_math_ai999x.solve('derangements', n=6)
# -> derangements = 265, probability = 0.368056   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many derangements of 5 objects are there?	SOLVED	derangements = 44, probability = 0.366667
In how many ways can 6 letters be put into envelopes so that no letter is in its own envelope?	SOLVED	derangements = 265, probability = 0.368056

Integer partitions p(n) — `integer_partitions`

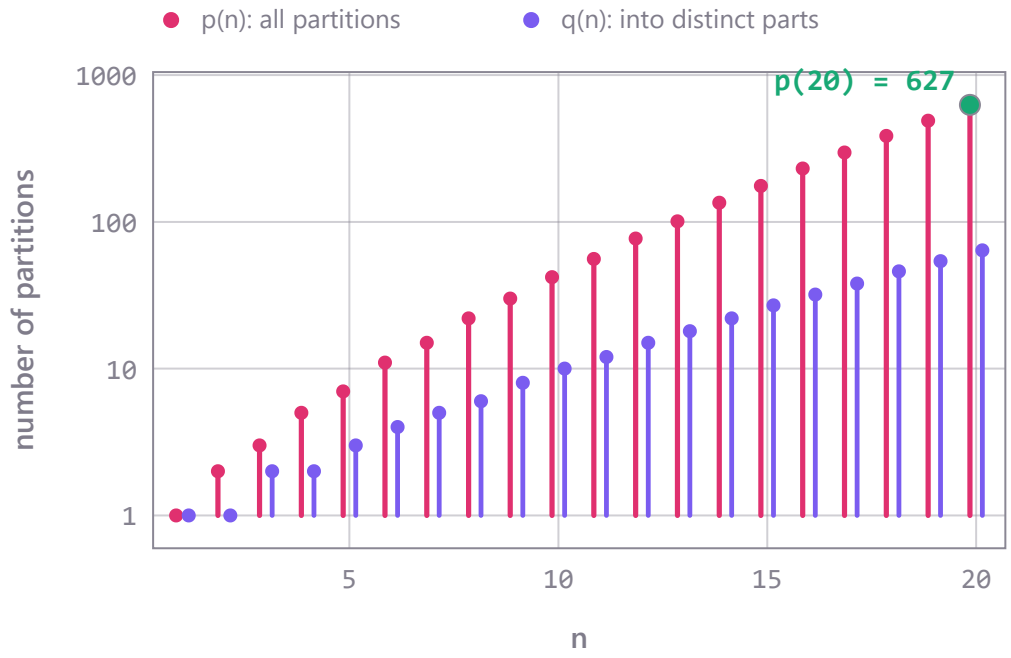
p(n): the number of ways to write n as a sum of positive integers, order ignored.

$$p(n) = \sum_{k \neq 0} (-1)^{k+1} p\left(n - \frac{k(3k-1)}{2}\right)$$

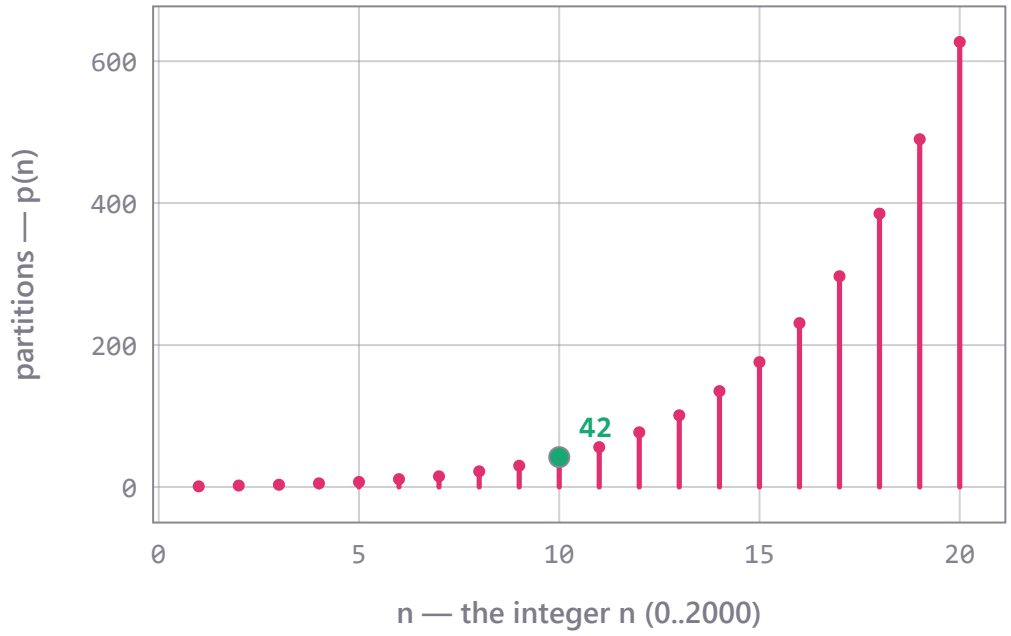
input	meaning	condition
`n`	the integer n (0..2000)	

output	meaning
`partitions`	p(n)

- **Method:** Euler's pentagonal number recurrence
- **Assumes:** $0 \leq n \leq 2000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The number of ways to write n as a sum of positive integers, p(n), and as a sum of distinct ones, q(n), for n = 1 … 20, each counted by the package (logarithmic scale): p(10) = 42, q(10) = 10; marked: p(20) = 627 (q(20) = 64).



partitions (p(n)) against n (the integer n (0..2000)) from 1 to 20. 20 points computed by discrete-math-ai999x; marked: the worked example, n = 10, partitions = 42.

```
discrete_math_ai999x.solve('integer_partitions', n=10)
# -> partitions = 42   (SOLVED)
```

```
discrete_math_ai999x.solve('integer_partitions', n=100)
# -> partitions = 1.90569e+08   (SOLVED)
```

```
discrete_math_ai999x.solve('integer_partitions', n=0)
# -> partitions = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many partitions of 10 are there?	SOLVED	partitions = 42
Compute the number of integer partitions of 100.	SOLVED	partitions = 1.90569e+08
In how many ways can we write 5 as a sum of positive integers?	SOLVED	partitions = 7

Partitions into exactly k parts — `partitions_k`

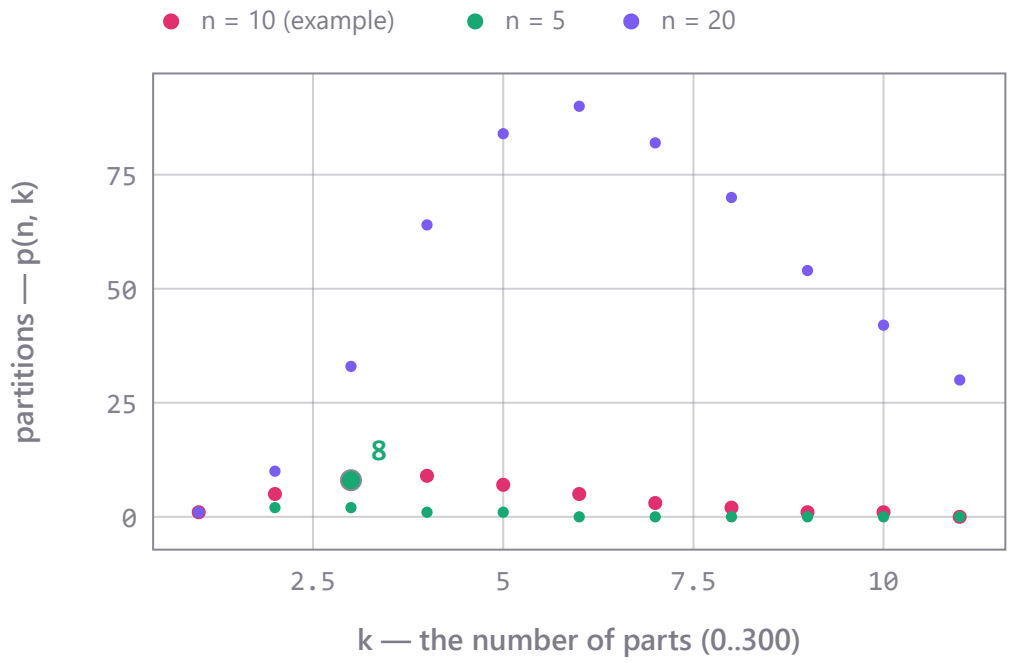
p(n, k): the number of ways to write n as a sum of exactly k positive integers, order ignored.

$$p(n,k) = p(n-1,k-1) + p(n-k,k)$$

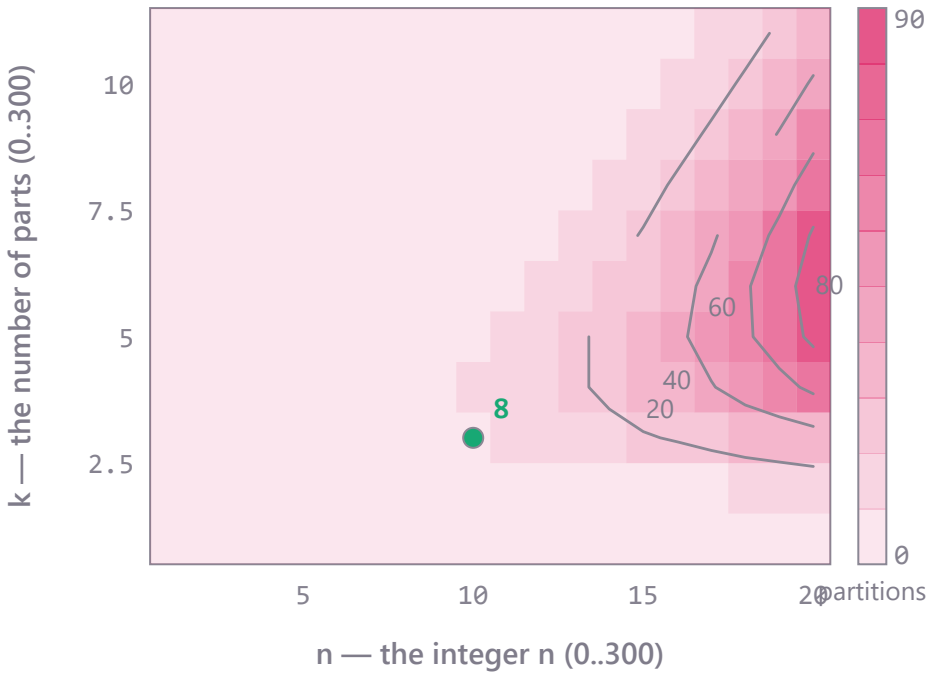
input	meaning	condition
`n`	the integer n (0..300)	
`k`	the number of parts (0..300)	

output	meaning
`partitions`	p(n, k)

- **Method:** recurrence p(n, k) = p($n-1$, k-1) + p(n-k, k)
- **Assumes:** $0 \le n, k \le 300$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



partitions (p(n, k)) against k (the number of parts (0..300)) from 1 to 11. One curve for each value of n (the integer n (0..300)): 10, 5, 20. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, k = 3, partitions = 8.



partitions (p(n, k)) over n (the integer n (0..300)) from 1 to 20 and k (the number of parts (0..300)) from 1 to 11: each of the 20 × 11 cells is one run of discrete-math-ai999x, shaded by its value (light 0 to dark 90); iso-lines at partitions = 20, 40, 60, 80. Marked: the worked example, n = 10, k = 3, partitions = 8.

```
discrete_math_ai999x.solve('partitions_k', n=10, k=3)
# -> partitions = 8   (SOLVED)
```

```
discrete_math_ai999x.solve('partitions_k', n=8, k=4)
# -> partitions = 5   (SOLVED)
```

```
discrete_math_ai999x.solve('partitions_k', n=5, k=7)
# -> partitions = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many partitions of 10 into exactly 3 parts are there?	SOLVED	partitions = 8
In how many ways can we write 8 as a sum of exactly 4 positive integers?	SOLVED	partitions = 5

Partitions into distinct parts — `partitions_distinct`

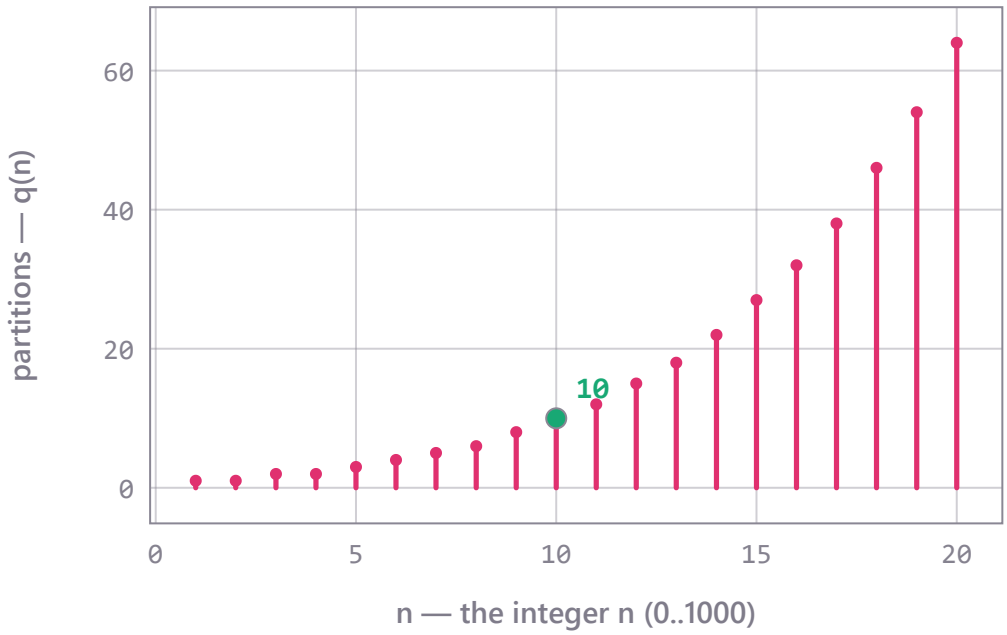
q(n): the number of ways to write n as a sum of distinct positive integers (equal to partitions into odd parts).

$$\sum_{n} q(n)x^n = \prod_{k\geq 1}(1+x^k) = \prod_{k\geq 1}\frac{1}{1-x^{2k-1}}$$

input	meaning	condition
`n`	the integer n (0..1000)	

output	meaning
`partitions`	q(n)

- **Method:** $0/1$ knapsack counting over the parts 1..n
- **Assumes:** $0 \leq n \leq 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



partitions (q(n)) against n (the integer n (0..1000)) from 1 to 20. 20 points computed by discrete-math-ai999x; marked: the worked example, n = 10, partitions = 10.

```
discrete_math_ai999x.solve('partitions_distinct', n=10)
# -> partitions = 10   (SOLVED)
```

```
discrete_math_ai999x.solve('partitions_distinct', n=15)
# -> partitions = 27   (SOLVED)
```

```
discrete_math_ai999x.solve('partitions_distinct', n=0)
# -> partitions = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many partitions of 10 into distinct parts are there?	SOLVED	partitions = 10
In how many ways can we write 15 as a sum of distinct positive integers?	SOLVED	partitions = 27

Integer solutions of x1 + ... + xk = n (stars and bars) — `stars_and_bars`

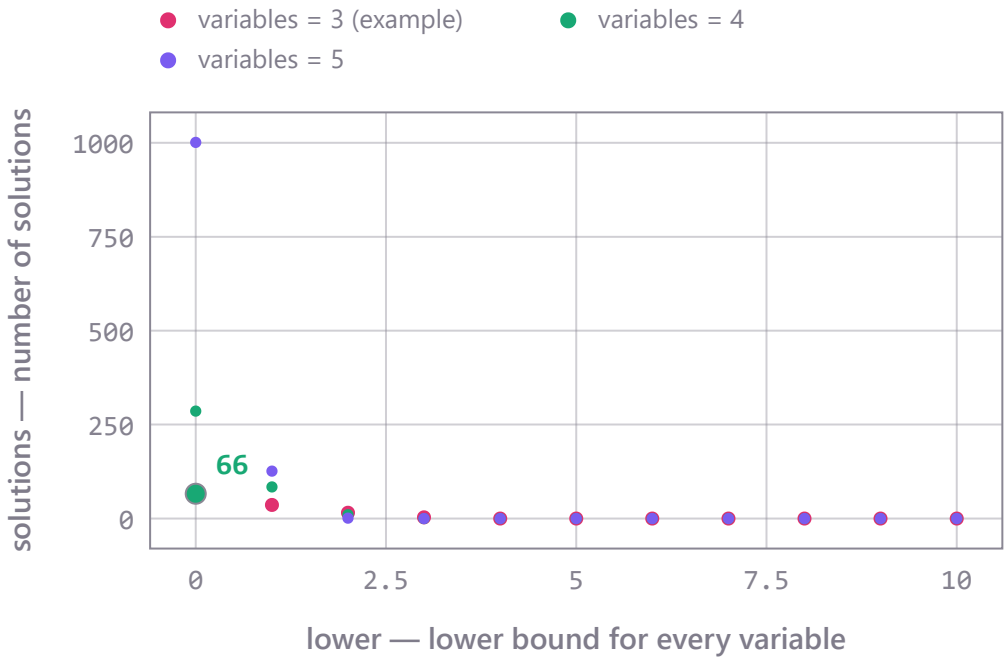
How many integer solutions $x_1 + \ldots + xk = n$ have every xi between a lower and an optional upper bound (identical objects into distinct boxes).

$$\#\{x_i \ge 0, \sum x_i = n\} = \binom{n+k-1}{k-1}, \quad \sum_j (-1)^j \binom{k}{j} \binom{n-j(c+1)+k-1}{k-1}$$

input	meaning	condition
`total`	the total n (0..2000)	
`variables`	the number of variables or boxes k (1..50)	
`lower`	lower bound for every variable (0 = non-negative, 1 = positive)	
`upper`	optional upper bound for every variable	

output	meaning
`solutions`	number of solutions

- **Method:** stars and bars after shifting by the lower bound; inclusion-exclusion over variables exceeding the upper bound
- **Assumes:** the bounds apply to every variable; a question must say non-negative or positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



solutions (number of solutions) against lower (lower bound for every variable (0 = non-negative, 1 = positive)) from -10 to 10. One curve for each value of variables (the number of variables or boxes k (1..50)): 3, 4, 5. Other inputs as in the worked example: total = 10. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, lower = 0, solutions = 66.

```
discrete_math_ai999x.solve('stars_and_bars', total=10, variables=3, lower=0)
# -> solutions = 66   (SOLVED)
```

```
discrete_math_ai999x.solve('stars_and_bars', total=12, variables=4, lower=1)
# -> solutions = 165   (SOLVED)
```

```
discrete_math_ai999x.solve('stars_and_bars', total=12, variables=4, lower=0, upper=5)
# -> solutions = 125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many non-negative integer solutions does x1 + x2 + x3 = 10 have?	SOLVED	solutions = 66
How many positive integer solutions does x + y + z + w = 12 have?	SOLVED	solutions = 165
In how many ways can 10 identical balls be placed into 4 distinct boxes?	SOLVED	solutions = 286
In how many ways can 10 identical balls be placed into 4 distinct boxes so that each box gets at least one?	SOLVED	solutions = 84
How many non-negative integer solutions does x1 + x2 + x3 + x4 = 12 have with each variable at most 5?	SOLVED	solutions = 125

Counting multiples by inclusion-exclusion — `inclusion_exclusion`

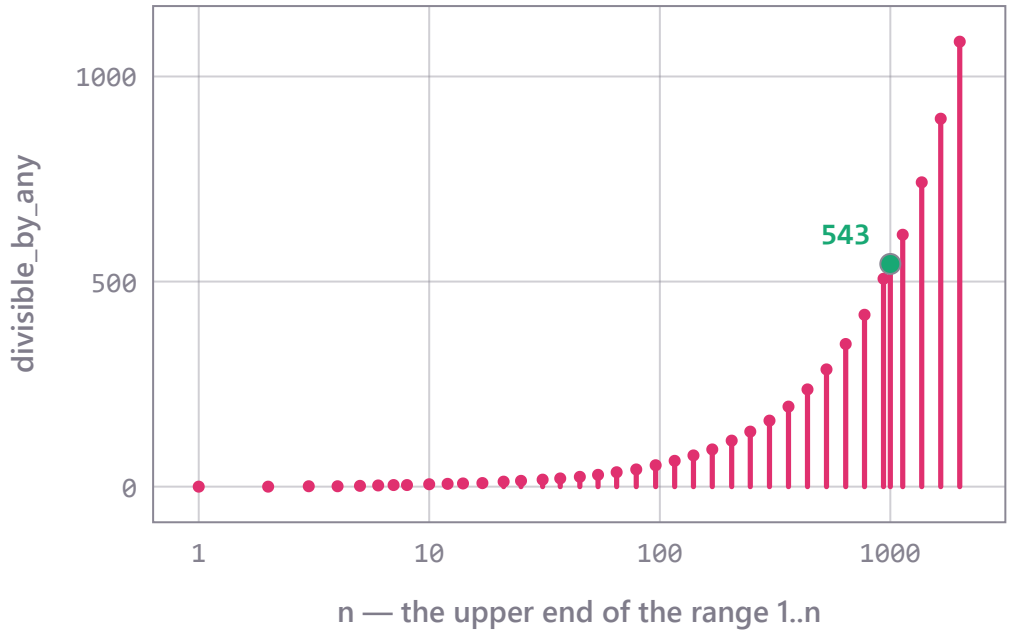
How many integers in 1..n are divisible by at least one of the given numbers (and how many by none).

$$\left|\bigcup_i A_i\right| = \sum_{\emptyset \neq S} (-1)^{|S|+1} \left\lfloor \frac{n}{\operatorname{lcm}(S)} \right\rfloor$$

input	meaning	condition
`n`	the upper end of the range 1..n	
`divisors`	the divisors (whole numbers >= 1)	

output	meaning
`divisible_by_any`	count divisible by at least one divisor
`divisible_by_none`	count divisible by none

- **Method:** inclusion-exclusion over subsets of the divisors, using floor(n / lcm(subset))
- **Assumes:** $n \geq 1$; 1 to 12 divisors, each >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



divisible_by_any (count divisible by at least one divisor) against n (the upper end of the range 1..n) from 1 to 2000 on a logarithmic axis. Other inputs as in the worked example: divisors = 3, 5, 7. 38 points computed by discrete-math-ai999x; marked: the worked example, n = 1000, divisible_by_any = 543.

```
discrete_math_ai999x.solve('inclusion_exclusion', n=1000, divisors='3, 5, 7')
# -> divisible_by_any = 543, divisible_by_none = 457   (SOLVED)
```

```
discrete_math_ai999x.solve('inclusion_exclusion', n=100, divisors='2, 3')
# -> divisible_by_any = 67, divisible_by_none = 33   (SOLVED)
```

```
discrete_math_ai999x.solve('inclusion_exclusion', n=30, divisors='4, 6')
# -> divisible_by_any = 10, divisible_by_none = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many integers from 1 to 1000 are divisible by 3, 5 or 7?	SOLVED	divisible_by_any = 543, divisible_by_none = 457
How many integers from 1 to 100 are divisible by 2 or 3?	SOLVED	divisible_by_any = 67, divisible_by_none = 33

Pigeonhole principle: guaranteed load — `pigeonhole_max`

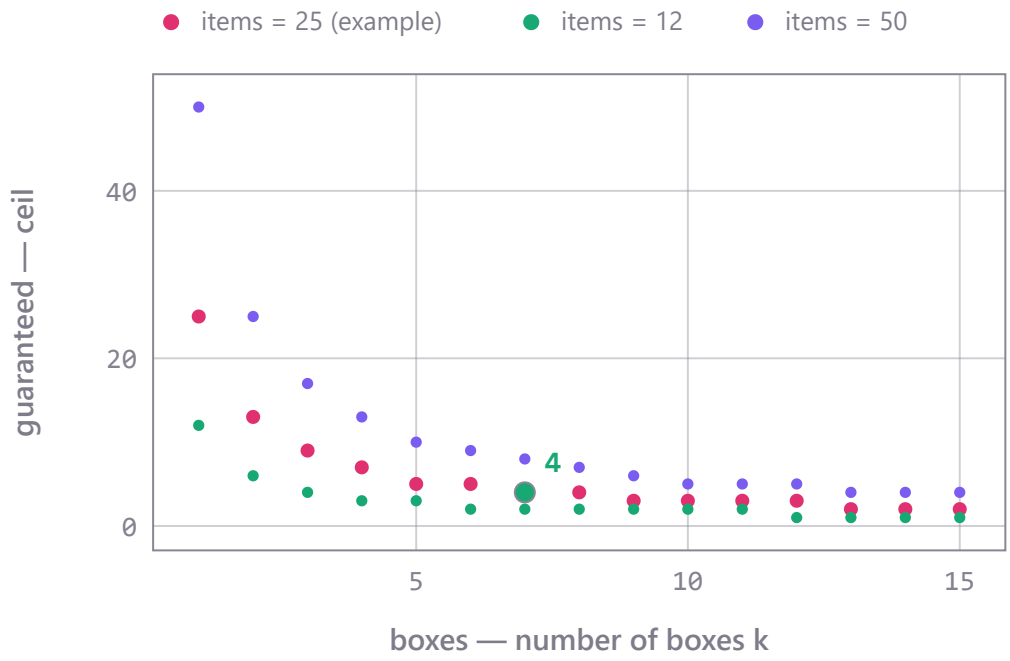
Placing n objects into k boxes, some box must hold at least ceil(n / k) objects (and this is the best guarantee).

$$\max_i x_i \geq \left\lceil \frac{n}{k} \right\rceil$$

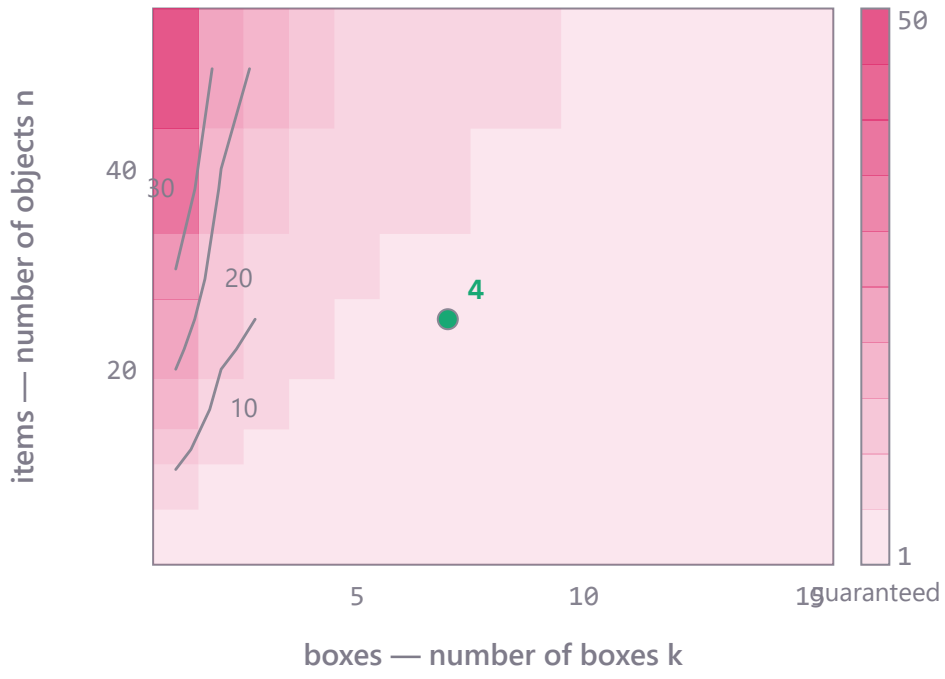
input	meaning	condition
`items`	number of objects n	
`boxes`	number of boxes k	

output	meaning
`guaranteed`	ceil(n / k): the largest load every placement must reach in some box

- **Method:** generalized pigeonhole principle
- **Assumes:** n, $k \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



guaranteed (ceil(n / k): the largest load every placement must reach in some box) against boxes (number of boxes k) from 1 to 15. One curve for each value of items (number of objects n): 25, 12, 50. 15 points per curve computed by discrete-math-ai999x; marked: the worked example, boxes = 7, guaranteed = 4.



guaranteed (ceil(n / k): the largest load every placement must reach in some box) over boxes (number of boxes k) from 1 to 15 and items (number of objects n) from 1 to 50: each of the 15 × 14 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 50); iso-lines at guaranteed = 10, 20, 30. Marked: the worked example, boxes = 7, items = 25, guaranteed = 4.

```
discrete_math_ai999x.solve('pigeonhole_max', items=25, boxes=7)
# -> guaranteed = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('pigeonhole_max', items=100, boxes=9)
# -> guaranteed = 12   (SOLVED)
```

```
discrete_math_ai999x.solve('pigeonhole_max', items=12, boxes=4)
# -> guaranteed = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
If 25 pigeons are placed into 7 holes, at least how many pigeons must share a hole?	SOLVED	guaranteed = 4
100 letters are put into 9 mailboxes; what is the least number some mailbox must contain?	SOLVED	guaranteed = 12

Pigeonhole principle: objects needed for a guarantee — `pigeonhole_needed`

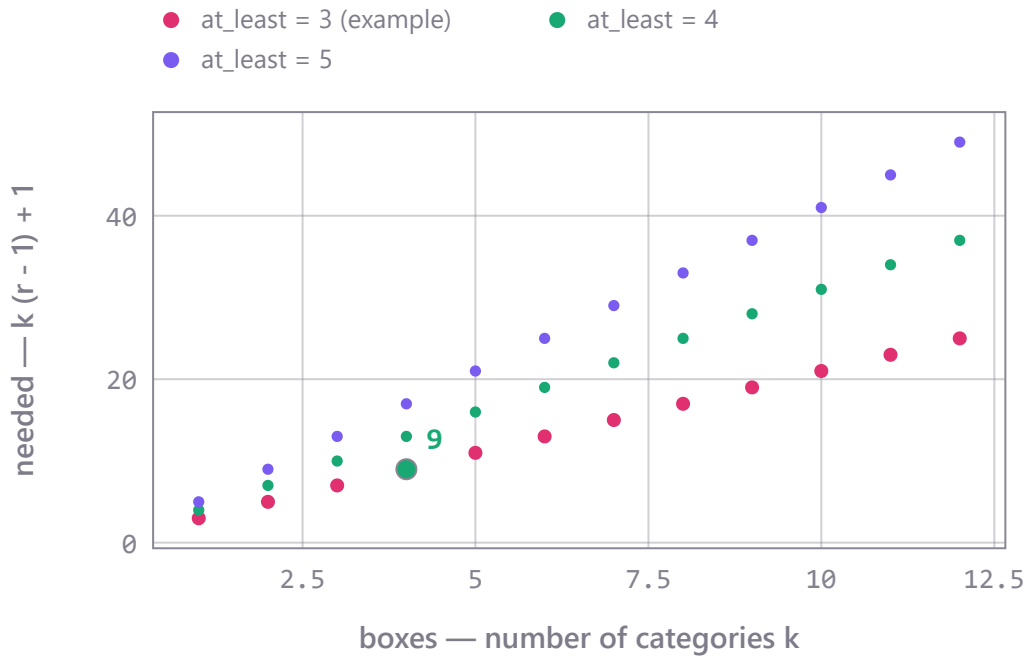
How many objects drawn from k categories guarantee at least r of one category: k $(r-1)+1$.

$$N = k(r-1)+1$$

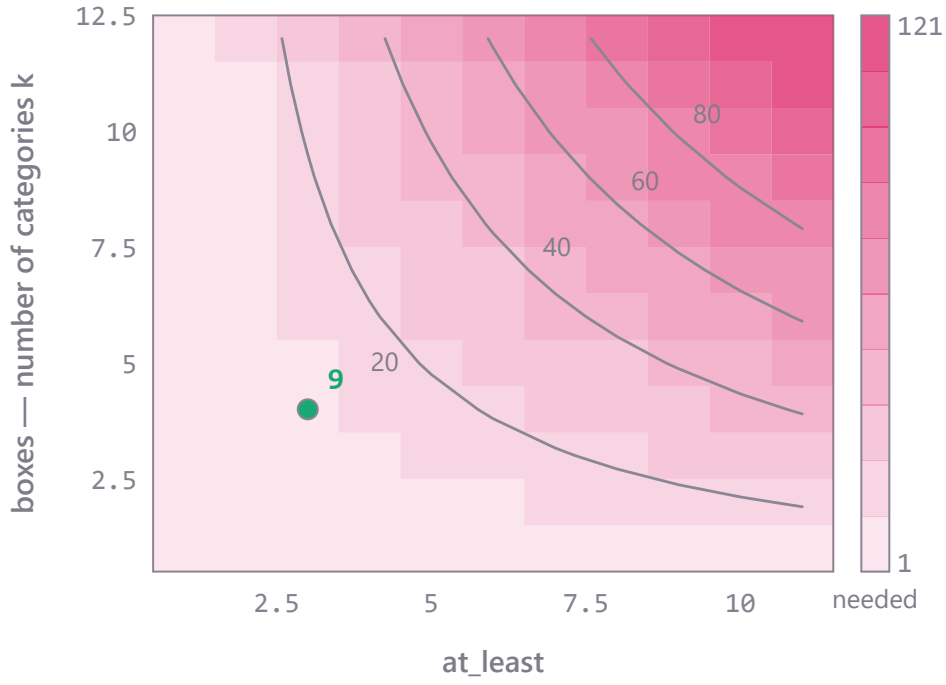
input	meaning	condition
`boxes`	number of categories k	
`at_least`	the guaranteed count r in one category	

output	meaning
`needed`	k $(r-1)+1$

- **Method:** pigeonhole: k $(r-1)$ objects can avoid r in every category
- **Assumes:** k, $r \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



needed (k (r - 1) + 1) against boxes (number of categories k) from 1 to 12. One curve for each value of at_least (the guaranteed count r in one category): 3, 4, 5. 12 points per curve computed by discrete-math-ai999x; marked: the worked example, boxes = 4, needed = 9.



needed (k (r - 1) + 1) over at_least (the guaranteed count r in one category) from 1 to 11 and boxes (number of categories k) from 1 to 12: each of the 11 × 12 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 121); iso-lines at needed = 20, 40, 60, 80. Marked: the worked example, at_least = 3, boxes = 4, needed = 9.

```
discrete_math_ai999x.solve('pigeonhole_needed', boxes=4, at_least=3)
# -> needed = 9   (SOLVED)
```

```
discrete_math_ai999x.solve('pigeonhole_needed', boxes=4, at_least=5)
# -> needed = 17   (SOLVED)
```

```
discrete_math_ai999x.solve('pigeonhole_needed', boxes=12, at_least=2)
# -> needed = 13   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many socks must be drawn from a drawer with 4 colours to guarantee 3 of the same colour?	SOLVED	needed = 9
How many cards must be chosen from 4 suits to guarantee that at least 5 of the same suit are chosen?	SOLVED	needed = 17

Necklaces (rotations identified) — `necklaces`

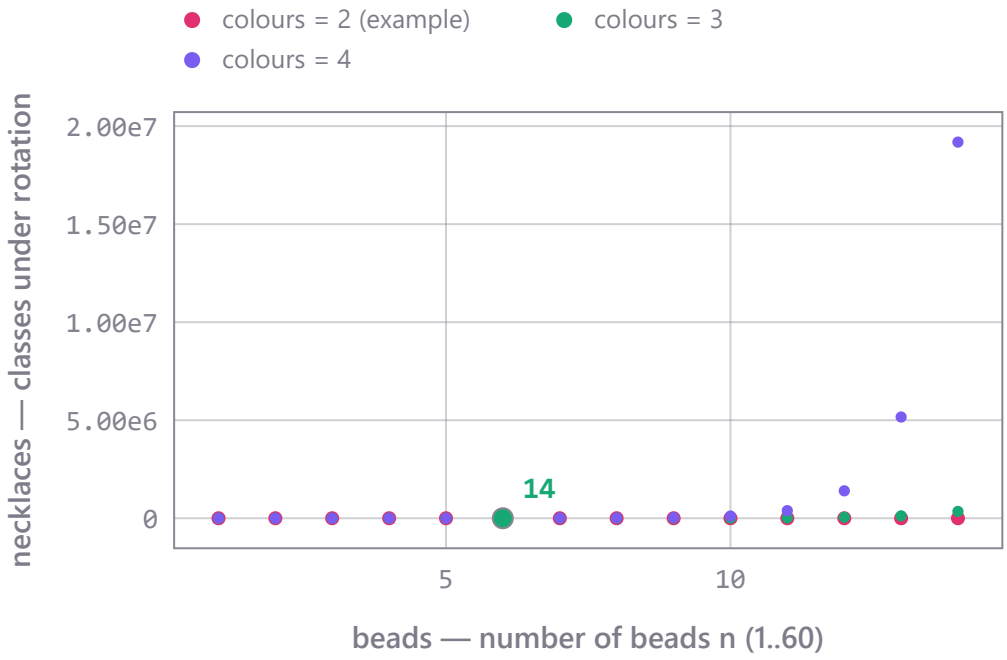
Distinct necklaces of n beads in k colours when rotations count as the same (and bracelets, also flips).

$$N(n,k) = \frac{1}{n}\sum_{d \mid n} \varphi(d)\, k^{n/d}$$

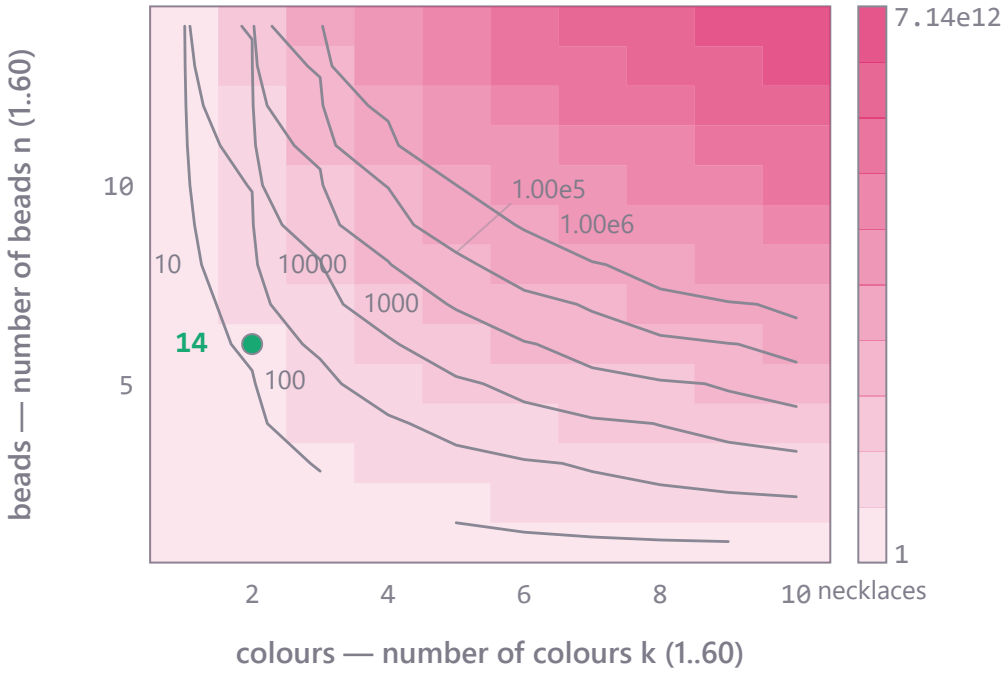
input	meaning	condition
`beads`	number of beads n (1..60)	
`colours`	number of colours k (1..60)	

output	meaning
`necklaces`	classes under rotation
`bracelets`	classes under rotation and reflection

- **Method:** Burnside's lemma over the cyclic group (and the dihedral group)
- **Assumes:** $1 \le n, k \le 60$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



necklaces (classes under rotation) against beads (number of beads n (1..60)) from 1 to 14. One curve for each value of colours (number of colours k (1..60)): 2, 3, 4. 14 points per curve computed by discrete-math-ai999x; marked: the worked example, beads = 6, necklaces = 14.



necklaces (classes under rotation) over colours (number of colours k (1..60)) from 1 to 10 and beads (number of beads n (1..60)) from 1 to 14: each of the 10 × 14 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 7.1429e+12, logarithmic shading); iso-lines at necklaces = 10, 100, 1000, 10000, 1.00e5, 1.00e6. Marked: the worked example, colours = 2, beads = 6, necklaces = 14.

```
discrete_math_ai999x.solve('necklaces', beads=6, colours=2)
# -> necklaces = 14, bracelets = 13   (SOLVED)
```

```
discrete_math_ai999x.solve('necklaces', beads=4, colours=3)
# -> necklaces = 24, bracelets = 21   (SOLVED)
```

```
discrete_math_ai999x.solve('necklaces', beads=5, colours=3)
# -> necklaces = 51, bracelets = 39   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many necklaces with 6 beads can be made using 2 colours?	SOLVED	necklaces = 14, bracelets = 13
How many distinct necklaces of 4 beads in 3 colours are there?	SOLVED	necklaces = 24, bracelets = 21

Bracelets (rotations and reflections identified) — `bracelets`

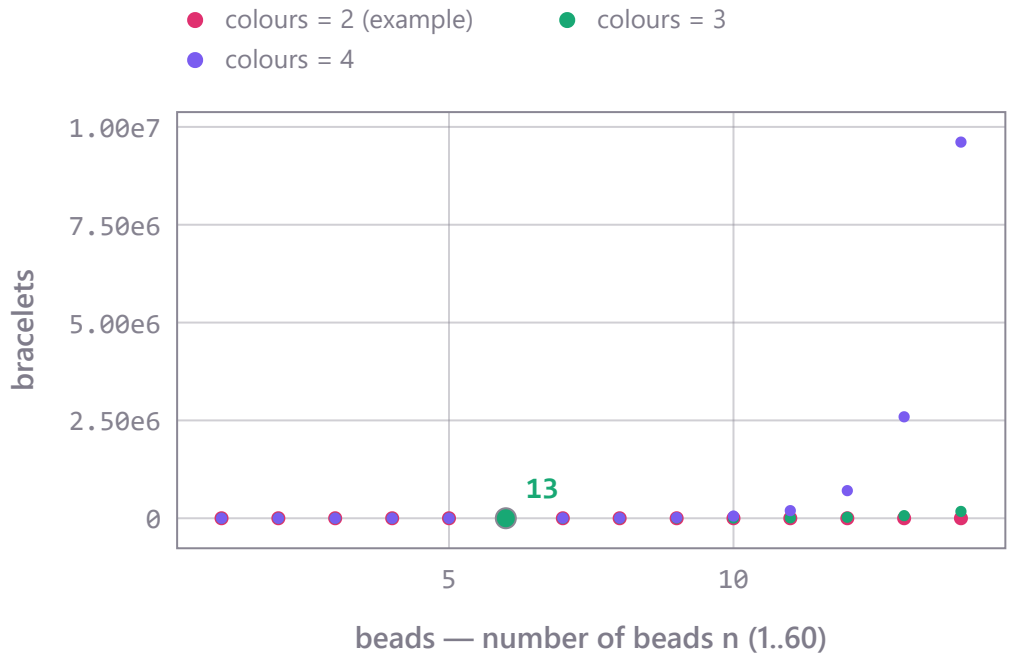
Distinct bracelets of n beads in k colours when rotations and flips count as the same.

$$B(n,k) = \tfrac{1}{2}N(n,k) + \begin{cases} \tfrac{1}{2}k^{(n+1)/2} & n \text{ odd} \\ \tfrac{1}{4}(k+1)k^{n/2} & n \text{ even} \end{cases}$$

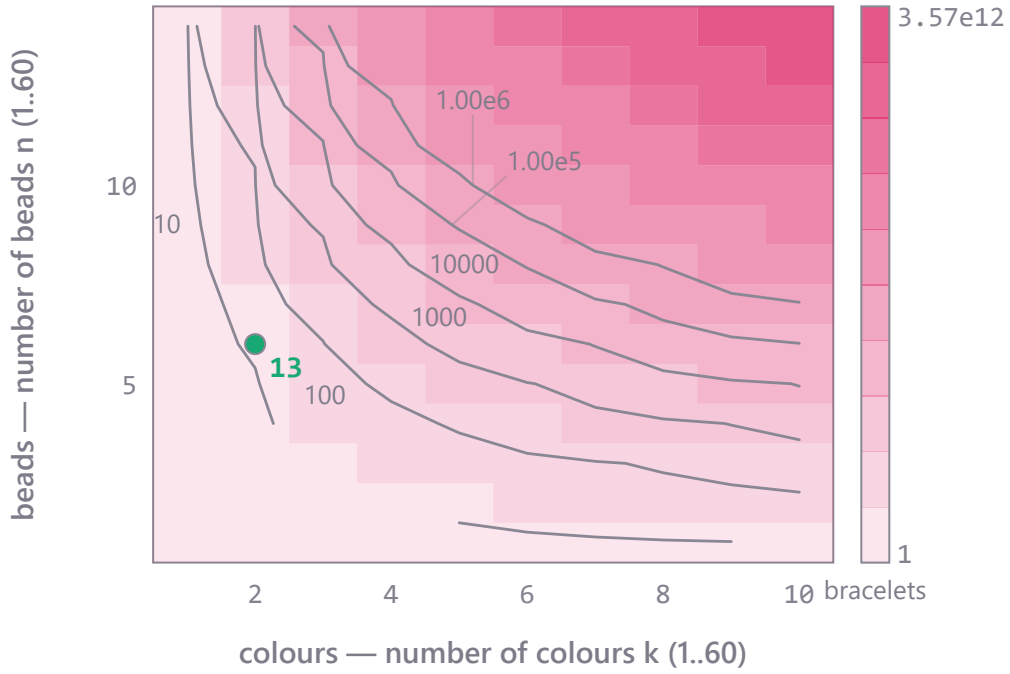
input	meaning	condition
`beads`	number of beads n (1..60)	
`colours`	number of colours k (1..60)	

output	meaning
`bracelets`	classes under rotation and reflection
`necklaces`	classes under rotation only

- **Method:** Burnside's lemma over the dihedral group
- **Assumes:** $1 \le n, k \le 60$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



bracelets (classes under rotation and reflection) against beads (number of beads n (1..60)) from 1 to 14. One curve for each value of colours (number of colours k (1..60)): 2, 3, 4. 14 points per curve computed by discrete-math-ai999x; marked: the worked example, beads = 6, bracelets = 13.



bracelets (classes under rotation and reflection) over colours (number of colours k (1..60)) from 1 to 10 and beads (number of beads n (1..60)) from 1 to 14: each of the 10 × 14 cells is one run of discrete-math-ai999x, shaded by its value (light 1 to dark 3.5715e+12, logarithmic shading); iso-lines at bracelets = 10, 100, 1000, 10000, 1.00e5, 1.00e6. Marked: the worked example, colours = 2, beads = 6, bracelets = 13.

```
discrete_math_ai999x.solve('bracelets', beads=6, colours=2)
# -> bracelets = 13, necklaces = 14   (SOLVED)
```

```
discrete_math_ai999x.solve('bracelets', beads=4, colours=3)
# -> bracelets = 21, necklaces = 24   (SOLVED)
```

```
discrete_math_ai999x.solve('bracelets', beads=5, colours=3)
# -> bracelets = 39, necklaces = 51   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many bracelets with 6 beads can be made using 2 colours?	SOLVED	bracelets = 13, necklaces = 14
How many distinct bracelets of 5 beads in 3 colours are there?	SOLVED	bracelets = 39, necklaces = 51

Coefficient in a binomial expansion — `binomial_term`

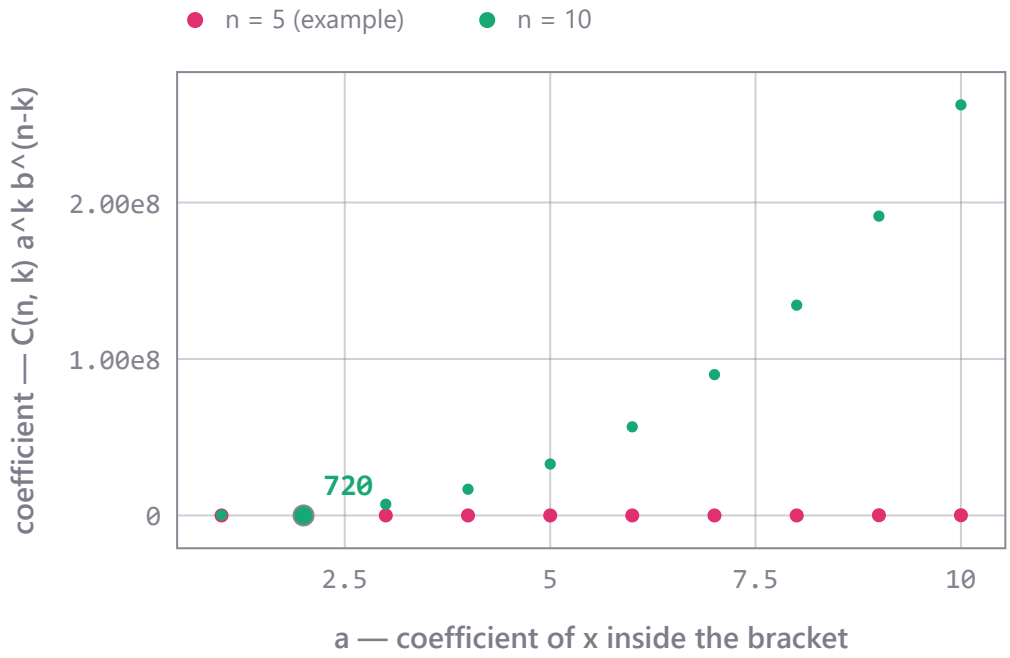
The coefficient of x^k in (a x + b)^n by the binomial theorem (exact integer).

$$(ax+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k} x^k$$

input	meaning	condition
`a`	coefficient of x inside the bracket	
`b`	constant inside the bracket	
`n`	the power n (0..1000)	
`k`	the power of x asked for	

output	meaning
`coefficient`	C(n, k) $a^k\ b^{n-k}$

- **Method:** binomial theorem
- **Assumes:** integer a, b; $0 \le k \le n \le 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



coefficient (C(n, k) a^k b^(n-k)) against a (coefficient of x inside the bracket) from 1 to 10. One curve for each value of n (the power n (0..1000)): 5, 10. Other inputs as in the worked example: b = 3, k = 3. 10 points per curve computed by discrete-math-ai999x; marked: the worked example, a = 2, coefficient = 720.

```
discrete_math_ai999x.solve('binomial_term', a=2, b=3, n=5, k=3)
# -> coefficient = 720   (SOLVED)
```

```
discrete_math_ai999x.solve('binomial_term', a=-2, b=1, n=6, k=2)
# -> coefficient = 60   (SOLVED)
```

```
discrete_math_ai999x.solve('binomial_term', a=1, b=1, n=10, k=5)
# -> coefficient = 252   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the coefficient of x^3 in (2x + 3)^5?	SOLVED	coefficient = 720
Find the coefficient of x^2 in (1 - 2x)^6.	SOLVED	coefficient = 60
Find the coefficient of x in (x - 4)^3.	SOLVED	coefficient = 48

Lattice paths in a grid — `lattice_paths`

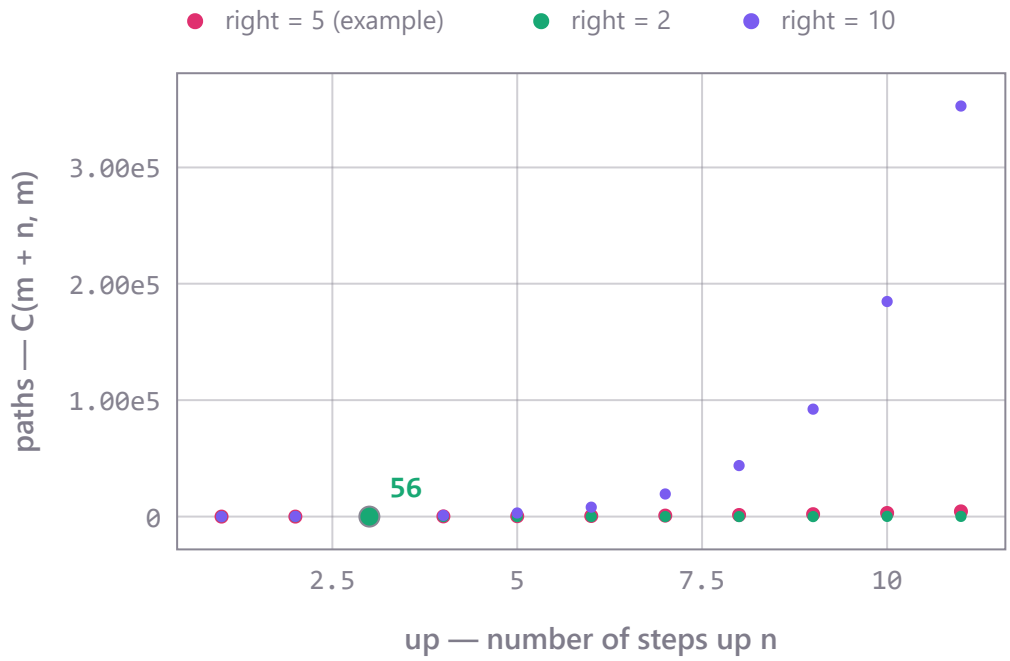
Monotone lattice paths using unit steps right and up from (0, 0) to (m, n): C(m + n, m).

$$\binom{m+n}{m}$$

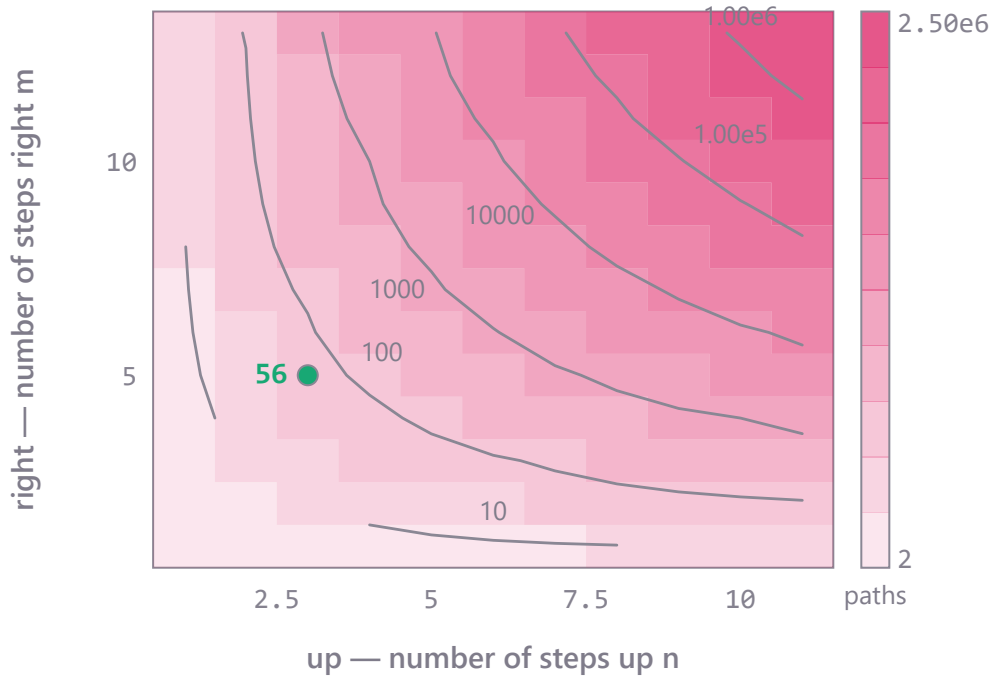
input	meaning	condition
`right`	number of steps right m	
`up`	number of steps up n	

output	meaning
`paths`	C(m + n, m)

- **Method:** choose which of the m + n steps go right
- **Assumes:** m, $n \geq 0$ (the end point is up and to the right); an a by b grid of squares has C(a + b, a) corner-to-corner paths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



paths (C(m + n, m)) against up (number of steps up n) from 1 to 11. One curve for each value of right (number of steps right m): 5, 2, 10. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, up = 3, paths = 56.



paths (C(m + n, m)) over up (number of steps up n) from 1 to 11 and right (number of steps right m) from 1 to 13: each of the 11 × 13 cells is one run of discrete-math-ai999x, shaded by its value (light 2 to dark 2.49614e+06, logarithmic shading); iso-lines at paths = 10, 100, 1000, 10000, 1.00e5, 1.00e6. Marked: the worked example, up = 3, right = 5, paths = 56.

```
discrete_math_ai999x.solve('lattice_paths', right=5, up=3)
# -> paths = 56   (SOLVED)
```

```
discrete_math_ai999x.solve('lattice_paths', right=4, up=4)
# -> paths = 70   (SOLVED)
```

```
discrete_math_ai999x.solve('lattice_paths', right=0, up=7)
# -> paths = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many lattice paths go from (0, 0) to (5, 3) moving only right and up?	SOLVED	paths = 56
How many shortest paths are there from one corner of a 4 by 4 grid to the opposite corner moving right or up?	SOLVED	paths = 70
How many lattice paths go from (2, 1) to (7, 4) moving only right and up?	SOLVED	paths = 56
How many lattice paths go from (5, 5) to (2, 7) moving only right and up?	INVALID_INPUT	right: must be at least 0, got -3.

Bernoulli numbers — `bernoulli`

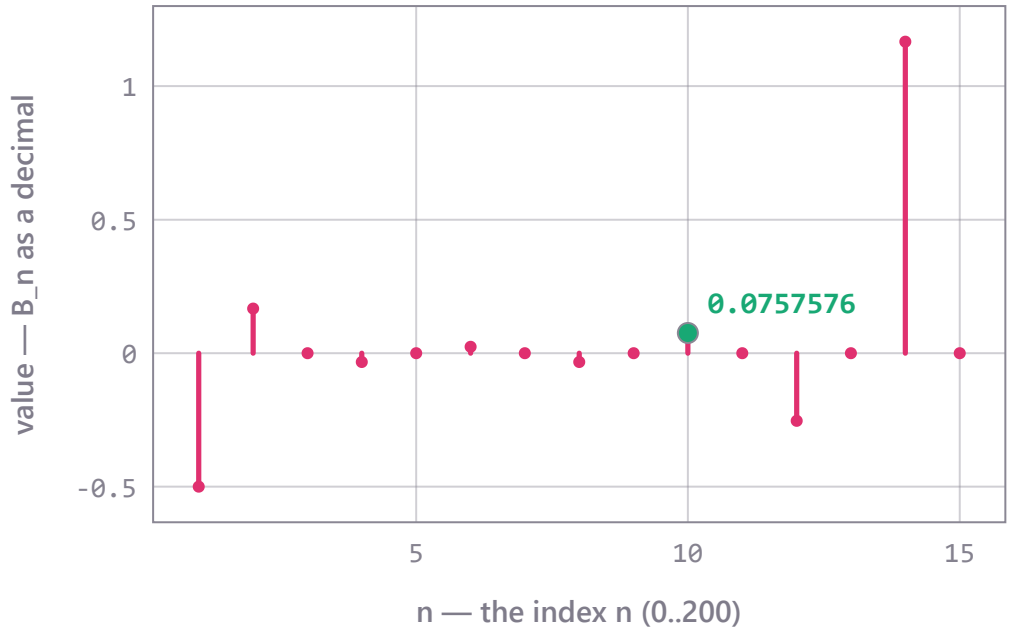
The Bernoulli number B_n as an exact fraction (convention B_1 = -1/2).

$$\sum_{k=0}^{m} \binom{m+1}{k} B_k = 0, \quad B_0 = 1$$

input	meaning	condition
`n`	the index n (0..200)	

output	meaning
`value`	B_n as a decimal
`numerator`	numerator of B_n in lowest terms
`denominator`	denominator of B_n (positive)

- **Method:** exact recurrence sum_{$k=0$}^{m} C($m+1$, k) $B_k = 0$ in rational arithmetic
- **Assumes:** $0 \le n \le 200$; $B_1 = -1/2$ (the 'first' Bernoulli numbers)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



value (B_n as a decimal) against n (the index n (0..200)) from 1 to 15. 15 points computed by discrete-math-ai999x; marked: the worked example, n = 10, value = 0.0757576.

```
discrete_math_ai999x.solve('bernoulli', n=10)
# -> numerator = 5, denominator = 66, value = 0.0757576   (SOLVED)
```

```
discrete_math_ai999x.solve('bernoulli', n=1)
# -> numerator = -1, denominator = 2, value = -0.5   (SOLVED)
```

```
discrete_math_ai999x.solve('bernoulli', n=12)
# -> numerator = -691, denominator = 2730, value = -0.253114   (SOLVED)
```

```
discrete_math_ai999x.solve('bernoulli', n=3)
# -> numerator = 0, denominator = 1, value = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the Bernoulli number B_10?	SOLVED	numerator = 5, denominator = 66, value = 0.0757576
Compute the 12th Bernoulli number.	SOLVED	numerator = -691, denominator = 2730, value = -0.253114

Vertex degrees and degree sequence — `graph_degrees`

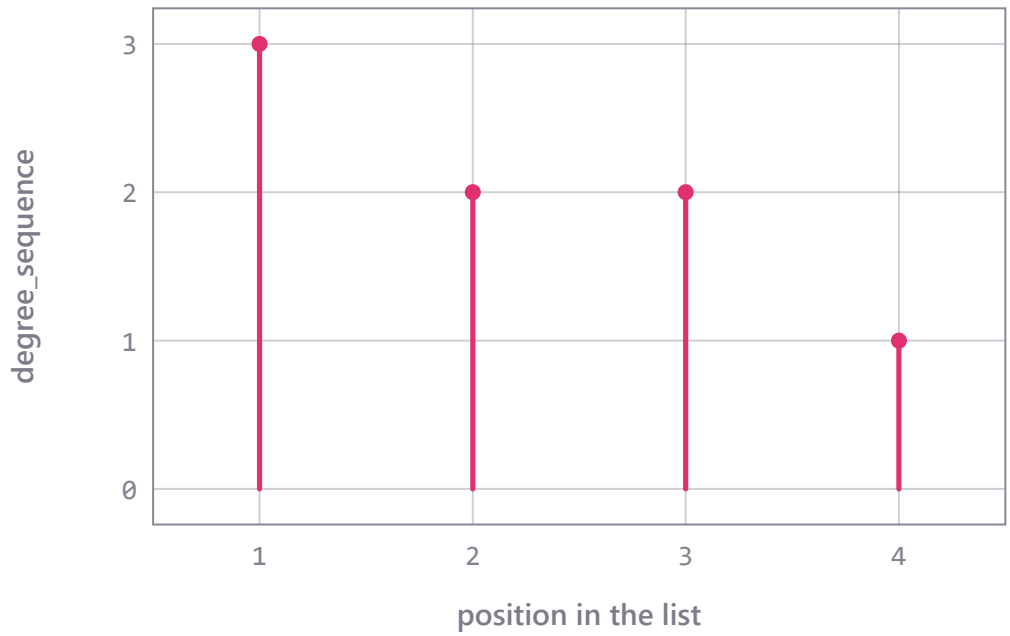
The degree of every vertex of an undirected simple graph, its degree sequence, maximum and minimum degree.

$$\sum_{v} \deg v = 2|E|$$

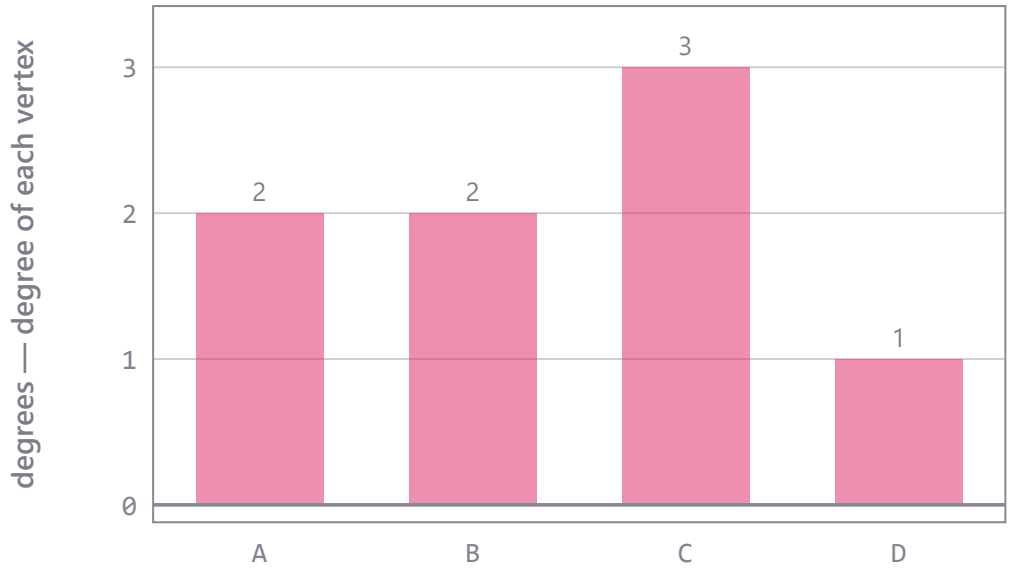
input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`vertices`	number of vertices (those named in the edges)
`edges`	number of edges
`degrees`	degree of each vertex
`degree_sequence`	degrees in descending order
`max_degree`	maximum degree
`min_degree`	minimum degree

- **Method:** count the edges at each vertex (handshake lemma: the degrees add up to twice the number of edges)
- **Assumes:** undirected simple graph (no loops, no repeated edges), unweighted; the vertices are those named in the edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



degrees in descending order (degree_sequence) returned by discrete-math-ai999x for the worked example, one value for each position in the list (4 values, first 3, last 1, largest 3). Inputs: edges = A-B, A-C, B-C, C-D.



degree of each vertex (degrees) returned by discrete-math-ai999x for the worked example (edges = A-B, A-C, B-C, C-D), one bar per entry: A = 2, B = 2, C = 3, D = 1.

```
discrete_math_ai999x.solve('graph_degrees', edges='A-B, A-C, B-C, C-D')
# -> vertices = 4, edges = 4, degrees = {'A': 2, 'B': 2, 'C': 3, 'D': 1}, degree_sequence = [3, 2, 2, 1], max_degree = 3, min_degree = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('graph_degrees', edges='1-2, 2-3, 3-1')
# -> vertices = 3, edges = 3, degrees = {'1': 2, '2': 2, '3': 2}, degree_sequence = [2, 2, 2], max_degree = 2, min_degree = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the degree sequence of the graph with edges A-B, A-C, B-C, C-D.	SOLVED	vertices = 4, edges = 4, degrees = {'A': 2, 'B': 2, 'C': 3, 'D': 1}, degree_sequence = [3, 2, 2, 1], max_degree = 3, min_degree = 1
What is the maximum degree of the graph with edges 1-2, 1-3, 1-4, 2-3?	SOLVED	vertices = 4, edges = 4, degrees = {'1': 3, '2': 2, '3': 2, '4': 1}, degree_sequence = [3, 2, 2, 1], max_degree = 3, min_degree = 1
Find the degree sequence of the graph with edges A-B, B-A.	INVALID_INPUT	Vertex degrees and degree sequence: the edge B-A is given twice.

Connected components — `connected_components`

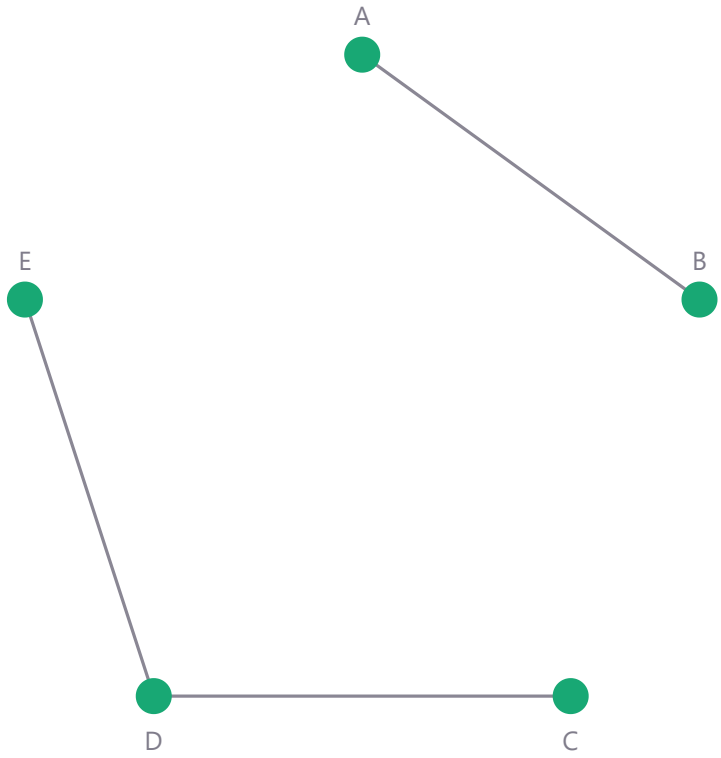
The connected components of an undirected graph: how many, their sizes and members, and whether it is connected.

$$V = C_1 \sqcup \cdots \sqcup C_c$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`count`	number of connected components
`sizes`	component sizes, descending
`is_connected`	true when there is one component
`components`	members of each component (ascending), components ordered by their first member

- **Method:** graph search from each unvisited vertex
- **Assumes:** undirected simple graph, unweighted; the vertices are those named in the edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 3 edges, laid out on a circle). Returned: count = 2.

```
discrete_math_ai999x.solve('connected_components', edges='A-B, C-D, D-E')
# -> count = 2, sizes = [3, 2], is_connected = False, components = [['A', 'B'], ['C', 'D', 'E']]
(SOLVED)
```

```
discrete_math_ai999x.solve('connected_components', edges='1-2, 2-3, 3-1')
# -> count = 1, sizes = [3], is_connected = True, components = [['1', '2', '3']]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many connected components does the graph with edges A-B, C-D, D-E have?	SOLVED	count = 2, sizes = [3, 2], is_connected = False, components = [['A', 'B'], ['C', 'D', 'E']]
Is the graph with edges 1-2, 2-3, 4-5 connected?	SOLVED	count = 2, sizes = [3, 2], is_connected = False, components = [['1', '2', '3'], ['4', '5']]

Breadth-first search order — `bfs_order`

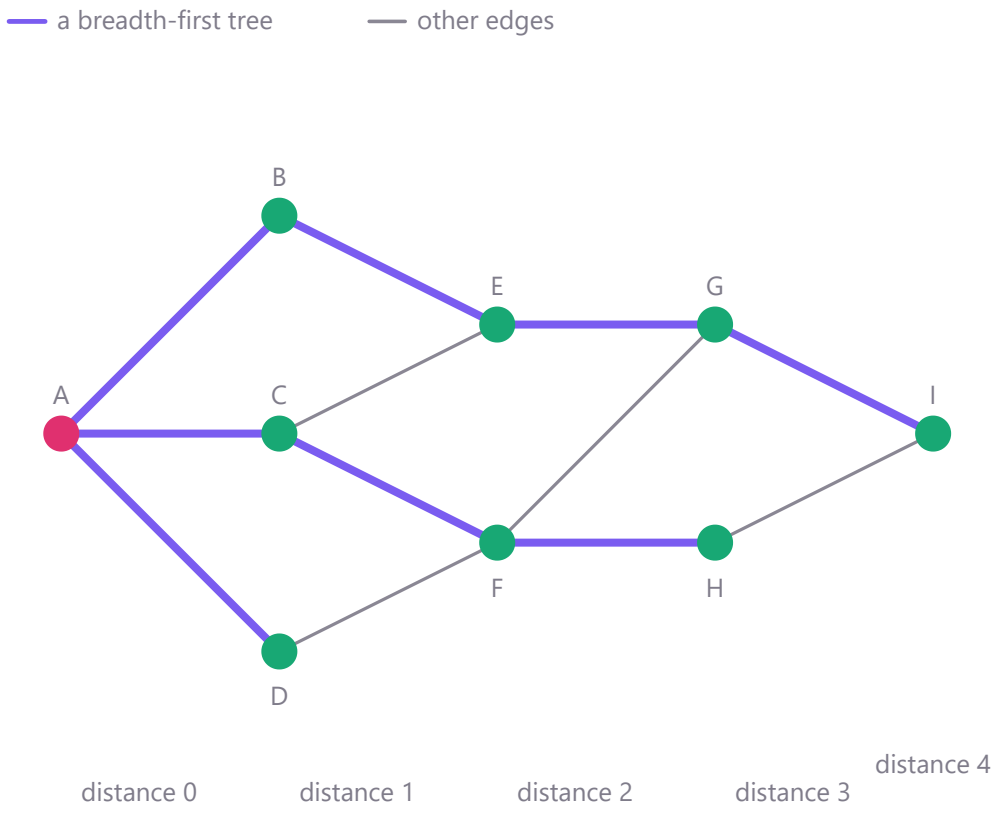
The order in which breadth-first search from a source visits the vertices, and each vertex's distance in edges.

$$d(v) = d(u) + 1 \text{ when } v \text{ is first seen from } u$$

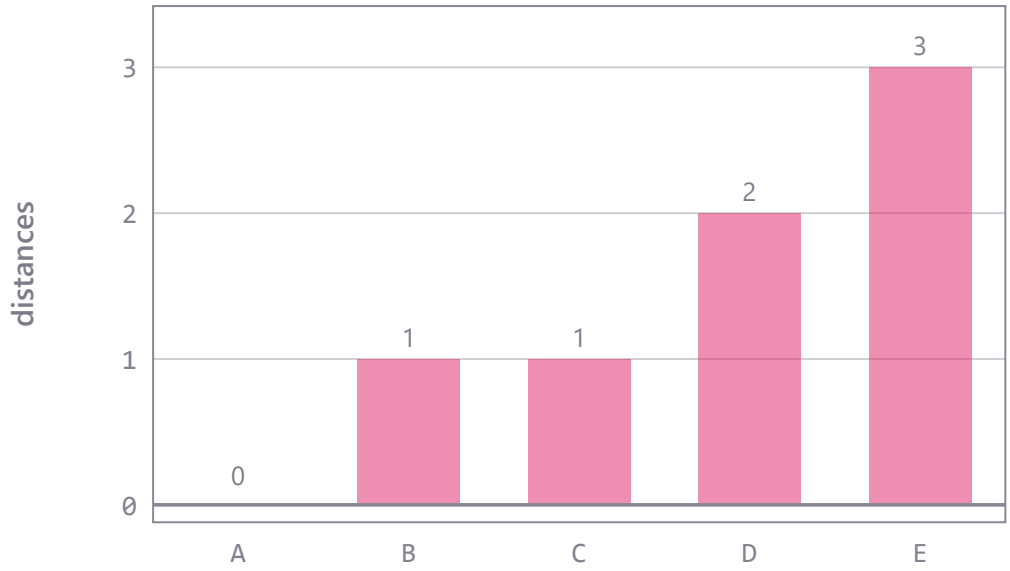
input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	
`source`	the start vertex	
`directed`	true for a directed graph (edges u->v)	

output	meaning
`order`	visiting order
`reached`	number of vertices reached
`distances`	number of edges from the source to each reached vertex

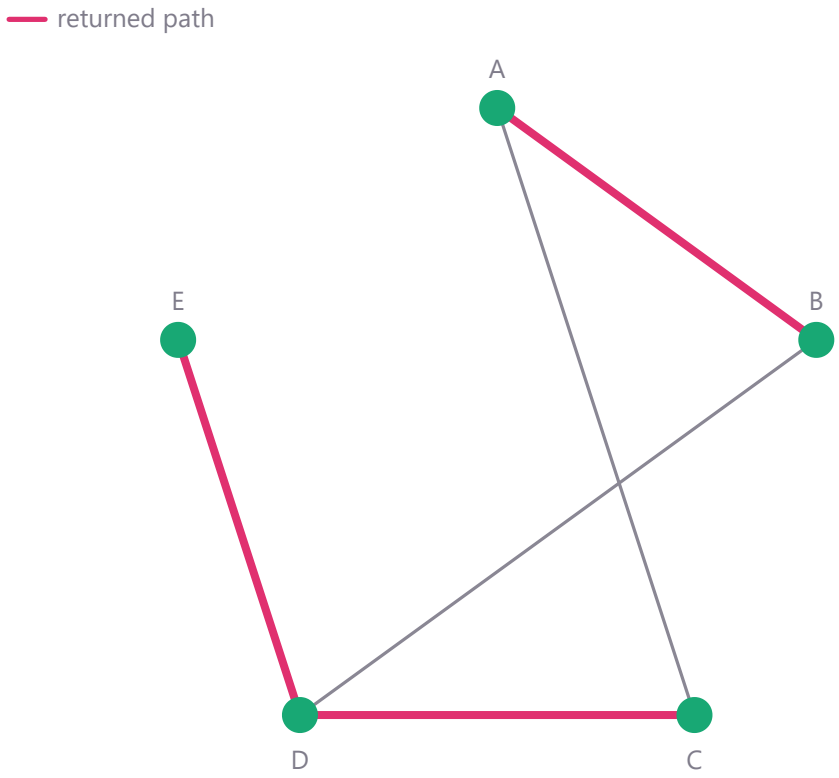
- **Method:** queue-based breadth-first search
- **Assumes:** simple unweighted graph; TIE RULE: neighbours are visited in ascending order of their names (numerically when the names are integers)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Breadth-first search from A on a graph of 9 vertices and 12 edges, each vertex placed in the column of its distance from A as returned by the package (visit order A, B, C, D, E, F, G, H, I; 9 reached); highlighted, one edge per vertex to a neighbour one layer closer.



number of edges from the source to each reached vertex (distances) returned by discrete-math-ai999x for the worked example (edges = A-B, A-C, B-D, C-D, D-E, source = A), one bar per entry: A = 0, B = 1, C = 1, D = 2, E = 3.



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 5 edges, laid out on a circle); highlighted: the returned path A → B → C → D → E. Returned: reached = 5. Other inputs: source = A.

```
discrete_math_ai999x.solve('bfs_order', edges='A-B, A-C, B-D, C-D, D-E', source='A')
# -> order = ['A', 'B', 'C', 'D', 'E'], reached = 5, distances = {'A': 0, 'B': 1, 'C': 1, 'D': 2,
'E': 3}   (SOLVED)
```

```
discrete_math_ai999x.solve('bfs_order', edges='1-10, 1-2, 2-3', source='1')
# -> order = ['1', '2', '10', '3'], reached = 4, distances = {'1': 0, '2': 1, '10': 1, '3': 2}
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Give the breadth-first search order from A in the graph with edges A-B, A-C, B-D, C-D, D-E.	SOLVED	order = ['A', 'B', 'C', 'D', 'E'], reached = 5, distances = {'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 3}
Run BFS starting at 1 on the directed graph with edges 1->3, 1->2, 3->4, 2->4.	SOLVED	order = ['1', '2', '3', '4'], reached = 4, distances = {'1': 0, '2': 1, '3': 1, '4': 2}
Run BFS starting at Z on the graph with edges A-B, B-C.	INVALID_INPUT	Breadth-first search order: Z is not a vertex of the graph.

Depth-first search order — `dfs_order`

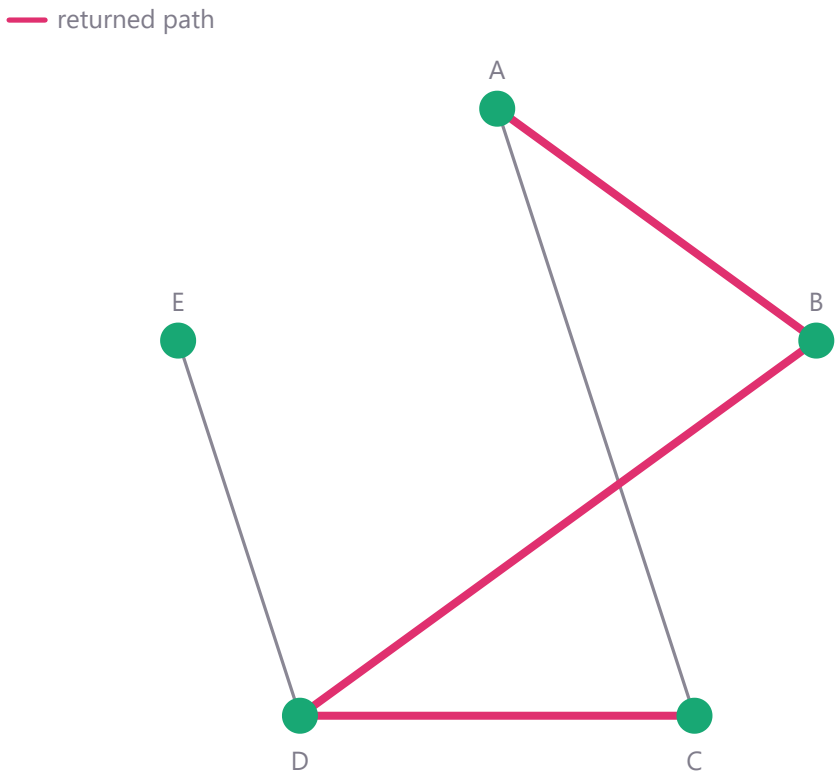
The preorder in which depth-first search from a source visits the vertices.

$$\mathbf{visit}(u):\ \mathbf{mark}\ u;\ \forall v \in N(u)\ \mathbf{ascending,\ unmarked:}\ \mathbf{visit}(v)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	
`source`	the start vertex	
`directed`	true for a directed graph (edges u->v)	

output	meaning
`order`	visiting order (preorder)
`reached`	number of vertices reached

- **Method:** depth-first search (recursive semantics, kept on an explicit stack of neighbour iterators)
- **Assumes:** simple unweighted graph; TIE RULE: recursive DFS that tries neighbours in ascending order of their names (numerically when the names are integers)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 5 edges, laid out on a circle); highlighted: the returned path A → B → D → C → E. Returned: reached = 5. Other inputs: source = A.

```
discrete_math_ai999x.solve('dfs_order', edges='A-B, A-C, B-D, C-D, D-E', source='A')
# -> order = ['A', 'B', 'D', 'C', 'E'], reached = 5   (SOLVED)
```

```
discrete_math_ai999x.solve('dfs_order', edges='1-2, 1-3, 2-4, 3-4', source='1')
# -> order = ['1', '2', '4', '3'], reached = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Give the depth-first search order from A in the graph with edges A-B, A-C, B-D, C-D, D-E.	SOLVED	order = ['A', 'B', 'D', 'C', 'E'], reached = 5
Run DFS starting at 1 on the directed graph with edges 1->2, 2->3, 1->3.	SOLVED	order = ['1', '2', '3'], reached = 3

Diameter, radius and center — `graph_diameter`

Eccentricities of a connected unweighted graph: the diameter (largest), radius (smallest), center and periphery.

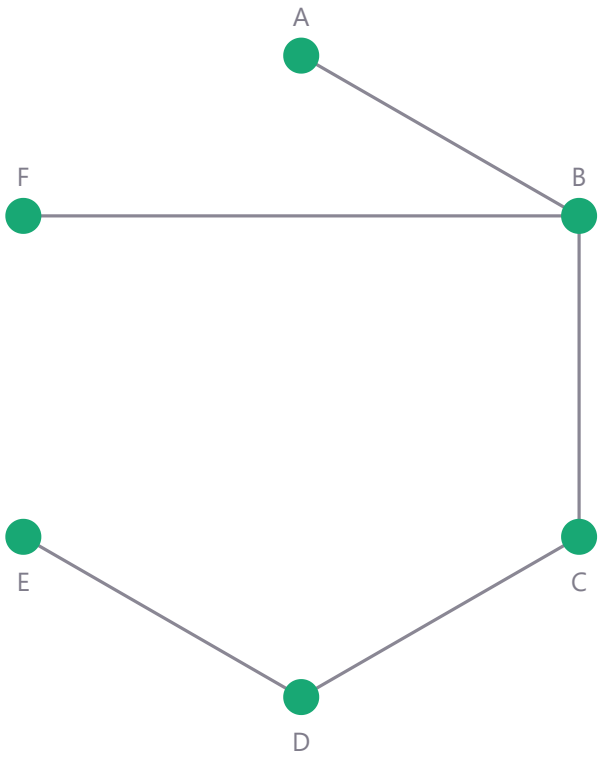
$$\operatorname{diam} G = \max_v e(v), \;\; \operatorname{rad} G = \min_v e(v)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`diameter`	largest distance between two vertices
`radius`	smallest eccentricity

output	meaning
`center`	vertices of smallest eccentricity (ascending)
`periphery`	vertices of largest eccentricity (ascending)

- **Method:** breadth-first search from every vertex
- **Assumes:** connected undirected simple graph, unweighted (distances count edges)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (6 nodes, 5 edges, laid out on a circle); highlighted: the returned path A → E → F. Returned: diameter = 4, radius = 2.

```
discrete_math_ai999x.solve('graph_diameter', edges='A-B, B-C, C-D, D-E, B-F')
# -> diameter = 4, radius = 2, center = ['C'], periphery = ['A', 'E', 'F']   (SOLVED)
```

```
discrete_math_ai999x.solve('graph_diameter', edges='1-2, 2-3, 3-1')
# -> diameter = 1, radius = 1, center = ['1', '2', '3'], periphery = ['1', '2', '3']   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the diameter of the graph with edges A-B, B-C, C-D, D-E, B-F?	SOLVED	diameter = 4, radius = 2, center = ['C'], periphery = ['A', 'E', 'F']
Find the radius and center of the graph with edges 1-2, 2-3, 3-4.	SOLVED	diameter = 3, radius = 2, center = ['2', '3'], periphery = ['1', '4']
What is the diameter of the graph with edges A-B, C-D?	INVALID_INPUT	Diameter, radius and center: the graph is not connected (the diameter is infinite).

Topological sort — `topological_sort`

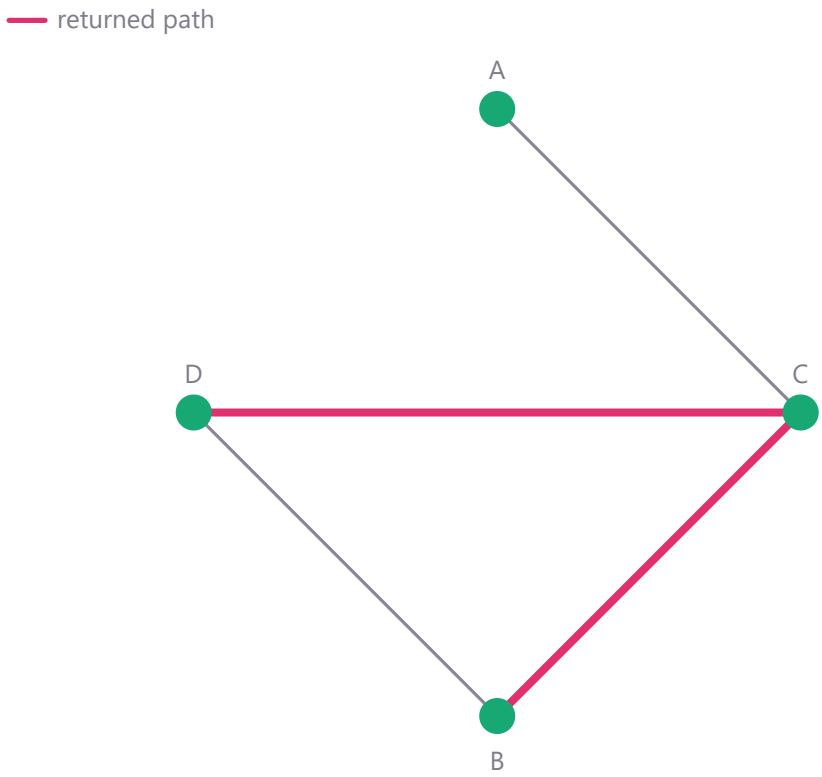
An ordering of a directed acyclic graph's vertices in which every edge u->v has u before v.

$$(u \to v) \in E \Rightarrow \pi(u) < \pi(v)$$

input	meaning	condition
`edges`	the directed graph as edges u->v	

output	meaning
`order`	the lexicographically smallest topological order
`vertices`	number of vertices

- **Method:** Kahn's algorithm with a min-priority queue of in-degree-zero vertices
- **Assumes:** directed acyclic simple graph, unweighted; TIE RULE: among the vertices free at each step the smallest name comes first (numerically when integers): the lexicographically smallest order
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the directed graph as edges u->v) of the worked example (4 nodes, 4 edges, laid out on a circle); highlighted: the returned path A → B → C → D. Returned: vertices = 4.

```
discrete_math_ai999x.solve('topological_sort', edges=[['A', 'C'], ['B', 'C'], ['C', 'D'], ['B',
'D']])
# -> order = ['A', 'B', 'C', 'D'], vertices = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('topological_sort', edges=[['5', '11'], ['7', '11'], ['7', '8'], ['3',
'8'], ['3', '10'], ['11', '2'], ['11', '9'], ['11', '10'], ['8', '9']])
# -> order = ['3', '5', '7', '8', '11', '2', '9', '10'], vertices = 8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find a topological sort of the directed graph with edges A->C, B->C, C->D, B->D.	SOLVED	order = ['A', 'B', 'C', 'D'], vertices = 4
Find a topological order of the directed graph with edges 1->2, 2->3, 3->1.	INVALID_INPUT	Topological sort: the graph has a directed cycle, so no topological order exists.

Cycles in an undirected graph (cyclomatic number) — `cycle_rank`

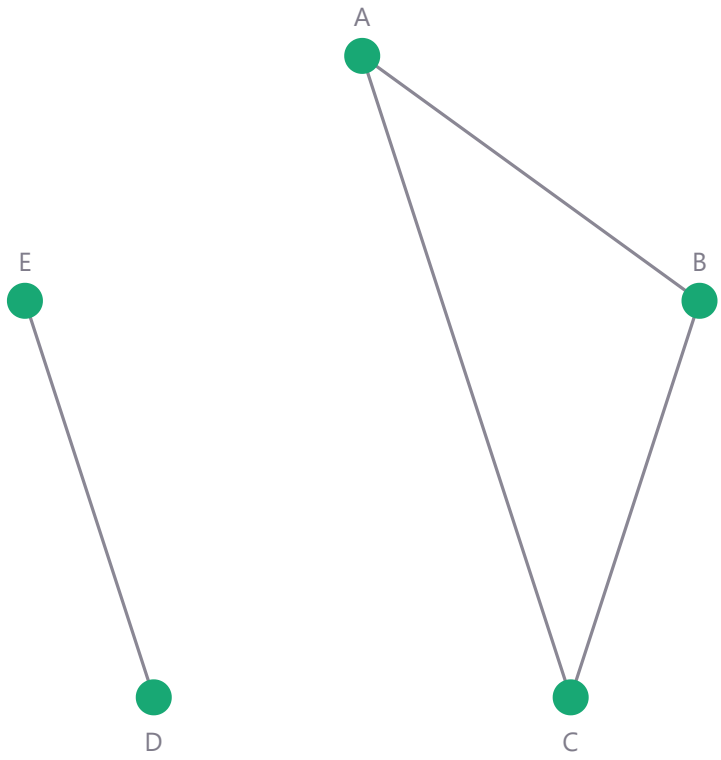
Whether an undirected graph has a cycle, and its cyclomatic number m - n + c (independent cycles).

$$\mu(G) = |E| - |V| + c(G)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`cyclomatic_number`	m - n + c: the number of independent cycles
`has_cycle`	true when the graph contains a cycle

- **Method:** count components; every edge beyond a spanning forest closes one independent cycle
- **Assumes:** undirected simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 4 edges, laid out on a circle). Returned: cyclomatic_number = 1.

```
discrete_math_ai999x.solve('cycle_rank', edges='A-B, B-C, C-A, D-E')
# -> cyclomatic_number = 1, has_cycle = True   (SOLVED)
```

```
discrete_math_ai999x.solve('cycle_rank', edges='1-2, 2-3, 3-4')
# -> cyclomatic_number = 0, has_cycle = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the cyclomatic number of the graph with edges A-B, B-C, C-A, D-E?	SOLVED	cyclomatic_number = 1, has_cycle = True
Does the graph with edges 1-2, 2-3, 3-4 contain a cycle?	SOLVED	cyclomatic_number = 0, has_cycle = False

Cycle detection in a directed graph — `directed_cycle`

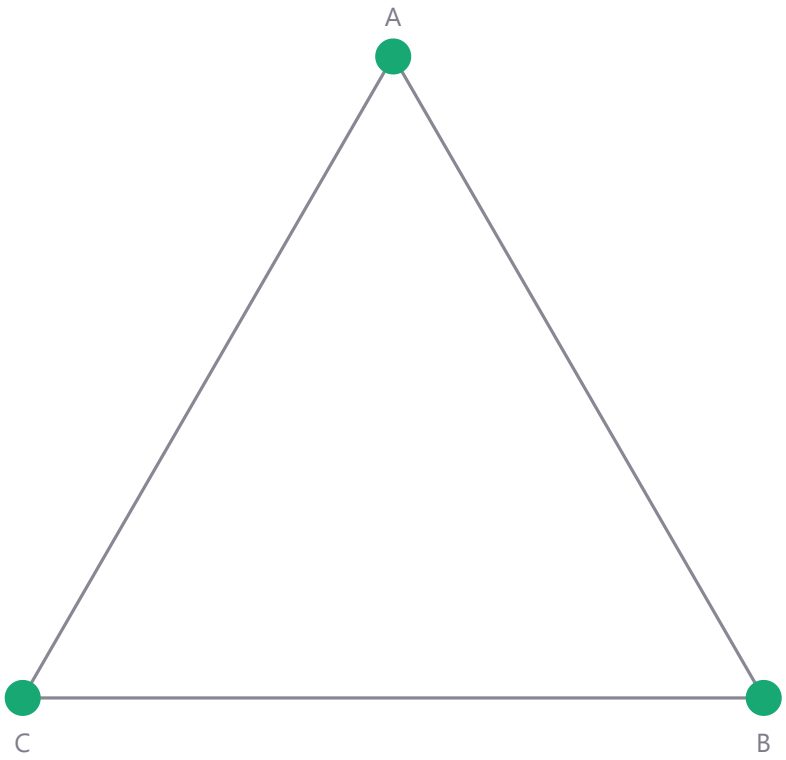
Whether a directed graph contains a directed cycle (equivalently, whether it is a DAG).

$$G \text{ acyclic} \iff \text{a topological order exists}$$

input	meaning	condition
`edges`	the directed graph as edges u->v	

output	meaning
`has_cycle`	true when a directed cycle exists
`is_dag`	true when the graph is acyclic

- **Method:** Kahn's algorithm: the graph is acyclic exactly when every vertex is removed
- **Assumes:** directed simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the directed graph as edges u->v) of the worked example (3 nodes, 3 edges, laid out on a circle).

```
discrete_math_ai999x.solve('directed_cycle', edges=[['A', 'B'], ['B', 'C'], ['C', 'A']])
# -> has_cycle = True, is_dag = False   (SOLVED)
```

```
discrete_math_ai999x.solve('directed_cycle', edges=[['1', '2'], ['1', '3'], ['2', '3']])
# -> has_cycle = False, is_dag = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Does the directed graph with edges A->B, B->C, C->A have a cycle?	SOLVED	has_cycle = True, is_dag = False
Is the directed graph with edges 1->2, 1->3, 2->3 acyclic?	SOLVED	has_cycle = False, is_dag = True

Bipartiteness test — `bipartite`

Whether the vertices of an undirected graph split into two sets with every edge between them (no odd cycle).

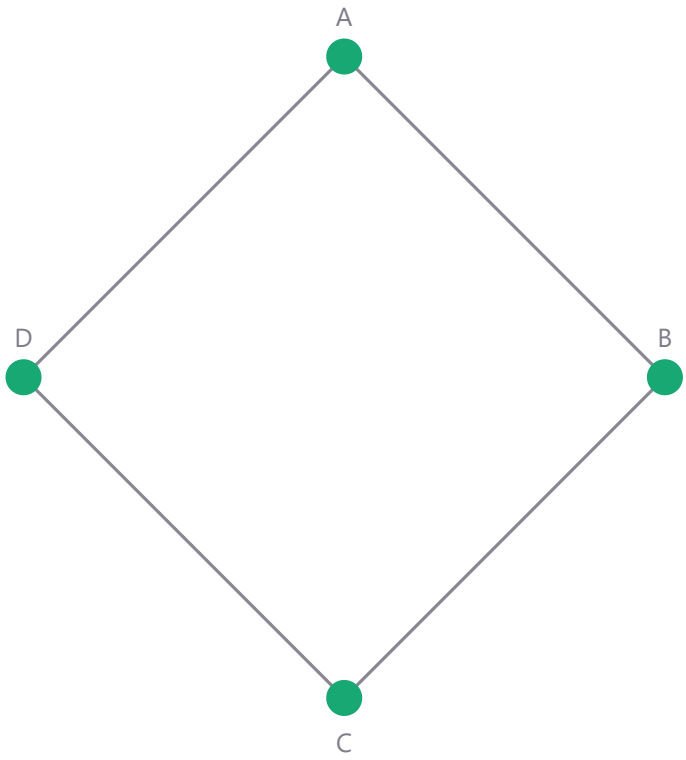
$$G \text{ bipartite} \iff G \text{ has no odd cycle}$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`is_bipartite`	true when the graph is 2-colourable

- **Method:** 2-colouring by graph search; a conflict exposes an odd cycle
- **Assumes:** undirected simple graph, unweighted

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (4 nodes, 4 edges, laid out on a circle).

```
discrete_math_ai999x.solve('bipartite', edges='A-B, B-C, C-D, D-A')
# -> is_bipartite = True   (SOLVED)
```

```
discrete_math_ai999x.solve('bipartite', edges='1-2, 2-3, 3-4, 4-5, 5-1')
# -> is_bipartite = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the graph with edges A-B, B-C, C-D, D-A bipartite?	SOLVED	is_bipartite = True
Is the graph with edges 1-2, 2-3, 3-4, 4-5, 5-1 bipartite?	SOLVED	is_bipartite = False

Eulerian circuits and paths — `eulerian`

Whether an undirected graph has an Euler circuit (closed trail using every edge once) or an Euler path.

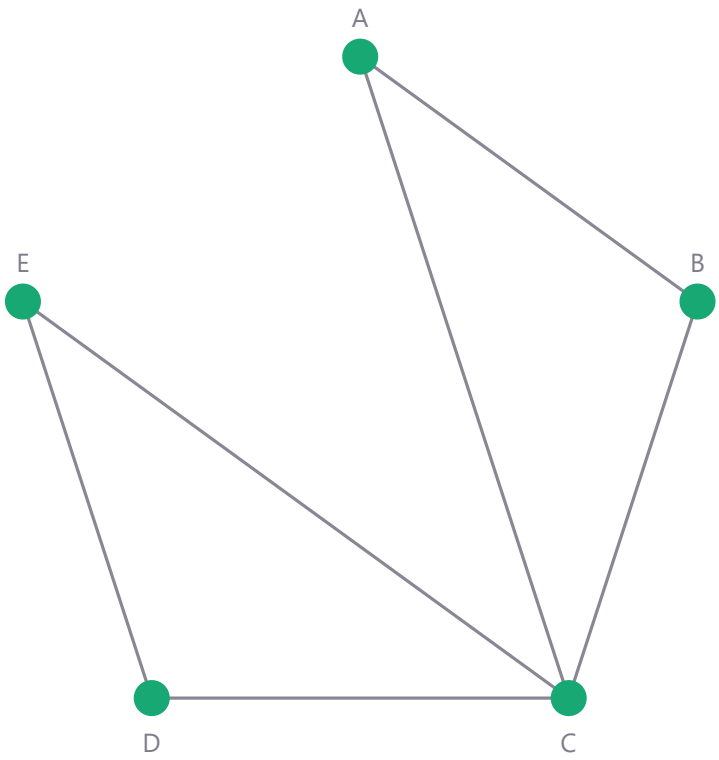
$$\text{circuit} \iff \text{connected} \wedge \forall v:\ 2 \mid \deg v$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`odd_vertices`	number of vertices of odd degree

output	meaning
`connected`	true when the graph is connected
`has_circuit`	Euler circuit exists: connected and no odd vertex
`has_path`	Euler path exists: connected and 0 or 2 odd vertices

- **Method:** Euler's theorem: degrees and connectivity
- **Assumes:** undirected simple graph, unweighted; the vertices are those named in the edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 6 edges, laid out on a circle). Returned: odd_vertices = 0.

```
discrete_math_ai999x.solve('eulerian', edges='A-B, B-C, C-A, C-D, D-E, E-C')
# -> odd_vertices = 0, connected = True, has_circuit = True, has_path = True   (SOLVED)
```

```
discrete_math_ai999x.solve('eulerian', edges='1-2, 2-3, 3-4')
# -> odd_vertices = 2, connected = True, has_circuit = False, has_path = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Does the graph with edges A-B, B-C, C-A, C-D, D-E, E-C have an Euler circuit?	SOLVED	odd_vertices = 0, connected = True, has_circuit = True, has_path = True
Does the graph with edges 1-2, 2-3, 3-4 have an Eulerian path?	SOLVED	odd_vertices = 2, connected = True, has_circuit = False, has_path = True

Chromatic number of a small graph — `chromatic_number`

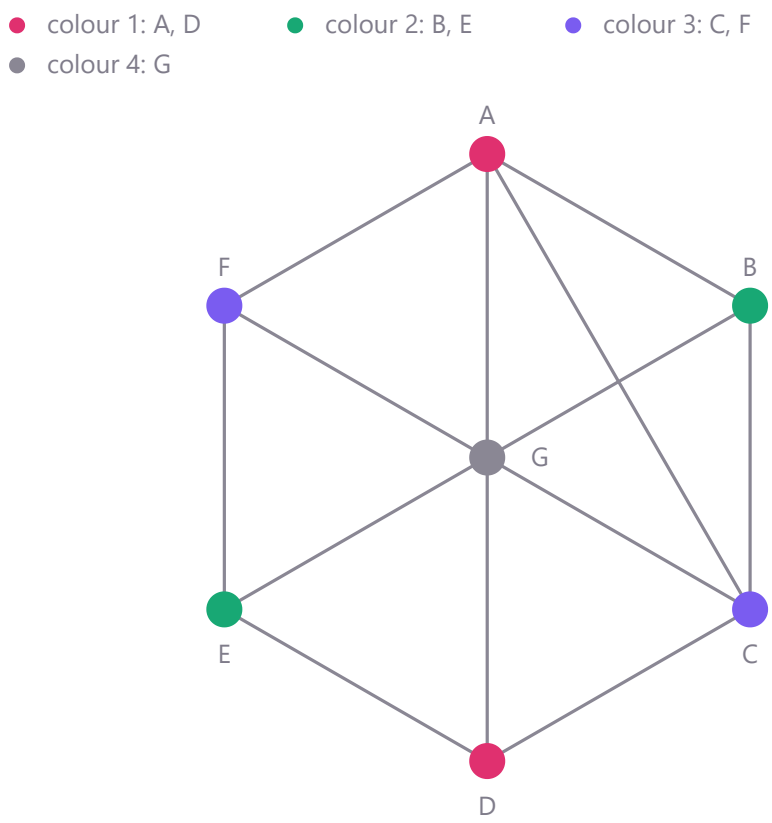
The fewest colours needed so that adjacent vertices differ, with one optimal colouring.

$$\chi(G) = \min\{k : G \text{ is } k\text{-colourable}\}$$

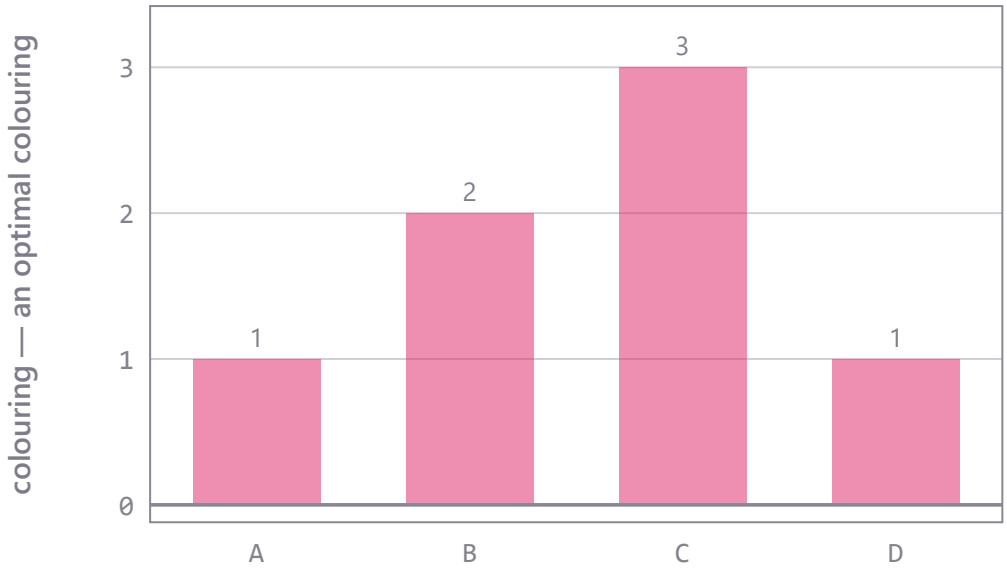
input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`chromatic_number`	chi(G)
`colouring`	an optimal colouring (colours 1..chi)

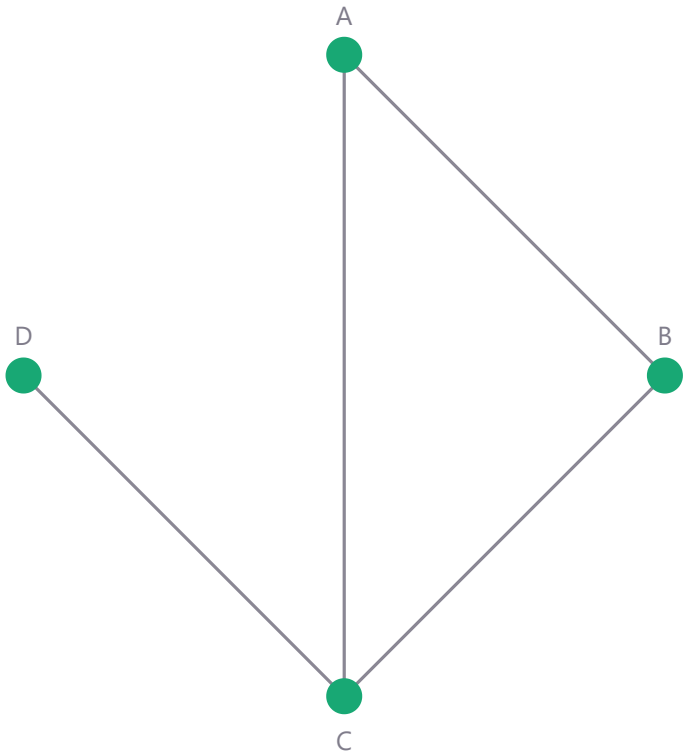
- **Method:** exact backtracking for $k = 1$, 2, ... colours
- **Assumes:** undirected simple graph, unweighted, at most 12 vertices; the colouring is the first one found trying vertices in ascending name order, colours 1, 2, ...
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A wheel graph on 7 vertices with the chord A–C (13 edges): the package finds the chromatic number 4 and the colouring drawn here — no edge joins two vertices of the same colour (colour 1: A, D; colour 2: B, E; colour 3: C, F; colour 4: G).



an optimal colouring (colours 1..chi) (colouring) returned by discrete-math-ai999x for the worked example (edges = A-B, B-C, C-A, C-D), one bar per entry: A = 1, B = 2, C = 3, D = 1.



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (4 nodes, 4 edges, laid out on a circle). Returned: chromatic_number = 3.

```
discrete_math_ai999x.solve('chromatic_number', edges='A-B, B-C, C-A, C-D')
# -> chromatic_number = 3, colouring = {'A': 1, 'B': 2, 'C': 3, 'D': 1}   (SOLVED)
```

```
discrete_math_ai999x.solve('chromatic_number', edges='1-2, 2-3, 3-4, 4-5, 5-1')
# -> chromatic_number = 3, colouring = {'1': 1, '2': 2, '3': 1, '4': 2, '5': 3}   (SOLVED)
```

```
discrete_math_ai999x.solve('chromatic_number', edges='1-2, 2-3, 3-4, 4-1')
# -> chromatic_number = 2, colouring = {'1': 1, '2': 2, '3': 1, '4': 2}   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the chromatic number of the graph with edges A-B, B-C, C-A, C-D?	SOLVED	chromatic_number = 3, colouring = {'A': 1, 'B': 2, 'C': 3, 'D': 1}
What is the minimum number of colours needed for the graph with edges 1-2, 2-3, 3-4, 4-5, 5-1?	SOLVED	chromatic_number = 3, colouring = {'1': 1, '2': 2, '3': 1, '4': 2, '5': 3}

Number of spanning trees (Kirchhoff) — `spanning_tree_count`

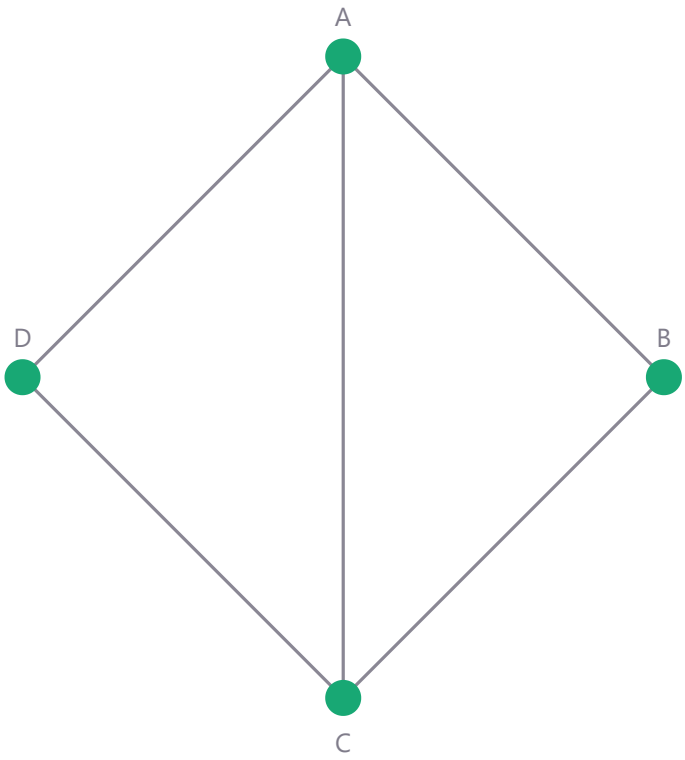
How many spanning trees an undirected simple graph has (0 if it is disconnected).

$$\tau(G) = \det L_{11}, \quad L = D - A$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`spanning_trees`	number of spanning trees tau(G)

- **Method:** Kirchhoff's matrix-tree theorem: any cofactor of the Laplacian, by exact rational elimination
- **Assumes:** undirected simple graph, unweighted, at most 60 vertices
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (4 nodes, 5 edges, laid out on a circle). Returned: spanning_trees = 8.

```
discrete_math_ai999x.solve('spanning_tree_count', edges='A-B, B-C, C-D, D-A, A-C')
# -> spanning_trees = 8   (SOLVED)
```

```
discrete_math_ai999x.solve('spanning_tree_count', edges='1-2, 1-3, 1-4, 2-3, 2-4, 3-4')
# -> spanning_trees = 16   (SOLVED)
```

```
discrete_math_ai999x.solve('spanning_tree_count', edges='1-2, 3-4')
# -> spanning_trees = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many spanning trees does the graph with edges A-B, B-C, C-D, D-A, A-C have?	SOLVED	spanning_trees = 8
Find the number of spanning trees of the graph with edges 1-2, 1-3, 1-4, 2-3, 2-4, 3-4.	SOLVED	spanning_trees = 16

Articulation points (cut vertices) — `articulation_points`

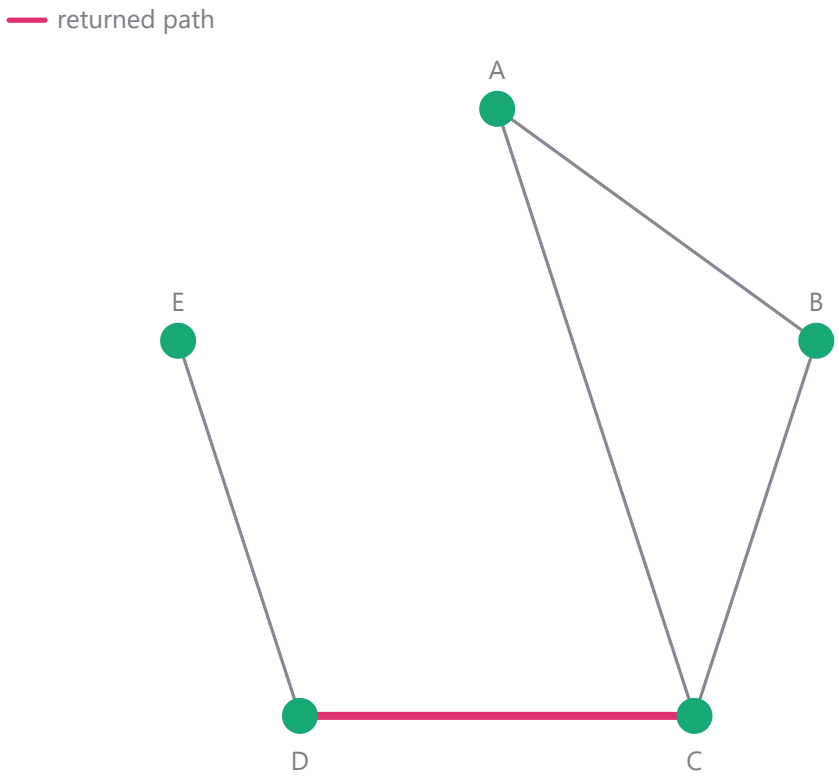
The vertices whose removal increases the number of connected components.

$$u \text{ cut vertex} \iff \exists \text{ child } v:\ \operatorname{low}(v) \geq \operatorname{disc}(u)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`count`	number of articulation points
`points`	the articulation points, ascending

- **Method:** Tarjan's depth-first lowlink algorithm
- **Assumes:** undirected simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 5 edges, laid out on a circle); highlighted: the returned path C → D. Returned: count = 2.

```
discrete_math_ai999x.solve('articulation_points', edges='A-B, B-C, C-A, C-D, D-E')
# -> count = 2, points = ['C', 'D']   (SOLVED)
```

```
discrete_math_ai999x.solve('articulation_points', edges='1-2, 2-3, 3-1')
# -> count = 0, points = []   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the articulation points of the graph with edges A-B, B-C, C-A, C-D, D-E.	SOLVED	count = 2, points = ['C', 'D']
How many cut vertices does the graph with edges 1-2, 2-3, 3-4 have?	SOLVED	count = 2, points = ['2', '3']

Bridges (cut edges) — `bridges`

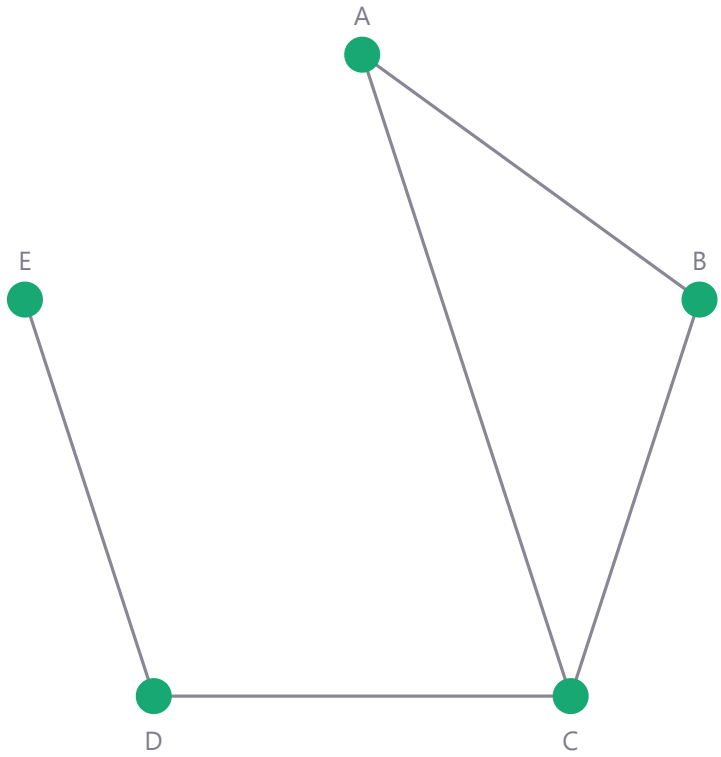
The edges whose removal increases the number of connected components.

$$(u, v) \text{ bridge} \iff \text{low}(v) > \text{disc}(u)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`count`	number of bridges
`bridges`	the bridges [u, v] with $u < v$, sorted

- **Method:** Tarjan's depth-first lowlink algorithm
- **Assumes:** undirected simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 5 edges, laid out on a circle). Returned: count = 2.

```
discrete_math_ai999x.solve('bridges', edges='A-B, B-C, C-A, C-D, D-E')
# -> count = 2, bridges = [['C', 'D'], ['D', 'E']]   (SOLVED)
```

```
discrete_math_ai999x.solve('bridges', edges='1-2, 2-3, 3-1')
# -> count = 0, bridges = []   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the bridges of the graph with edges A-B, B-C, C-A, C-D, D-E.	SOLVED	count = 2, bridges = [['C', 'D'], ['D', 'E']]
How many cut edges does the graph with edges 1-2, 2-3, 3-1, 3-4 have?	SOLVED	count = 1, bridges = [['3', '4']]

Strongly connected components — `strongly_connected`

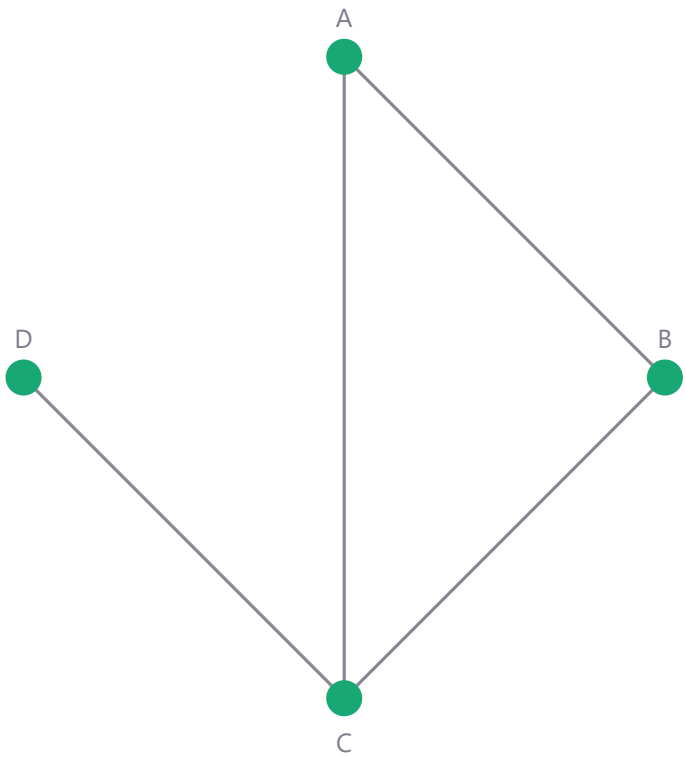
The strongly connected components of a directed graph (vertices mutually reachable).

$$u \sim v \iff u \rightsquigarrow v \wedge v \rightsquigarrow u$$

input	meaning	condition
`edges`	the directed graph as edges u->v	

output	meaning
`count`	number of strongly connected components
`sizes`	component sizes, descending
`components`	members of each component (ascending), ordered by first member

- **Method:** Tarjan's algorithm
- **Assumes:** directed simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the directed graph as edges u->v) of the worked example (4 nodes, 4 edges, laid out on a circle). Returned: count = 2.

```
discrete_math_ai999x.solve('strongly_connected', edges=[['A', 'B'], ['B', 'C'], ['C', 'A'], ['C',
'D']])
# -> count = 2, sizes = [3, 1], components = [['A', 'B', 'C'], ['D']]   (SOLVED)
```

```
discrete_math_ai999x.solve('strongly_connected', edges=[['1', '2'], ['2', '3']])
# -> count = 3, sizes = [1, 1, 1], components = [['1'], ['2'], ['3']]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the strongly connected components of the directed graph with edges A->B, B->C, C->A, C->D.	SOLVED	count = 2, sizes = [3, 1], components = [['A', 'B', 'C'], ['D']]
How many strongly connected components does the directed graph with edges 1->2, 2->1, 2->3 have?	SOLVED	count = 2, sizes = [2, 1], components = [['1', '2'], ['3']]

Hamiltonian cycle of a small graph — `hamiltonian_cycle`

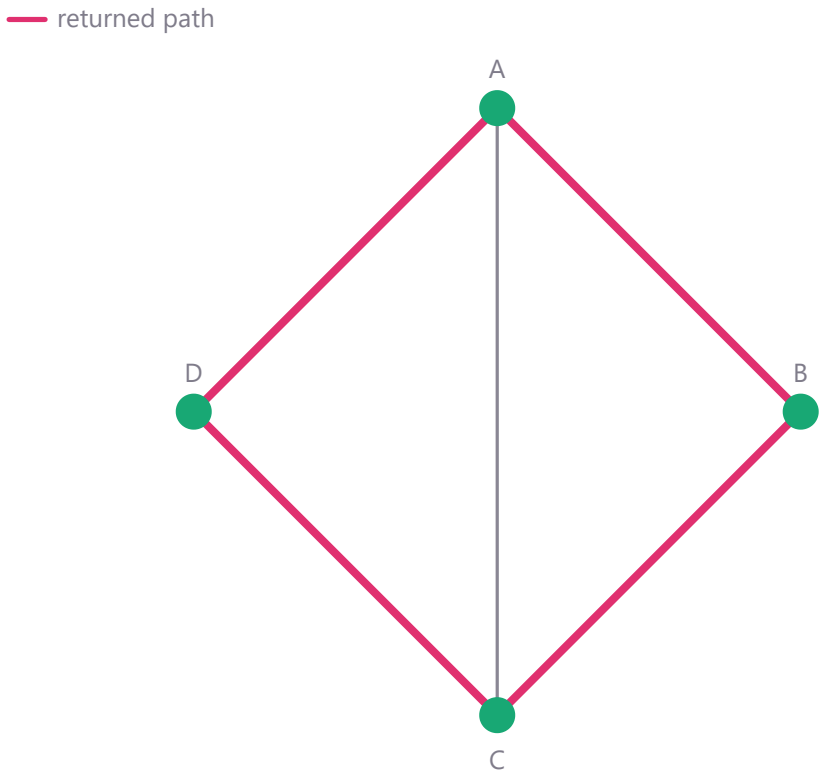
Whether an undirected graph has a cycle through every vertex exactly once, with one such cycle.

$$v_1 v_2 \cdots v_n v_1,\ \{v_i\} = V$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`exists`	true when a Hamiltonian cycle exists
`cycle`	a Hamiltonian cycle starting and ending at the smallest vertex (none if it does not exist)

- **Method:** backtracking search from the smallest vertex
- **Assumes:** undirected simple graph, unweighted, at most 12 vertices; the cycle shown is the first found extending through neighbours in ascending order
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (4 nodes, 5 edges, laid out on a circle); highlighted: the returned path A → B → C → D → A.

```
discrete_math_ai999x.solve('hamiltonian_cycle', edges='A-B, B-C, C-D, D-A, A-C')
# -> exists = True, cycle = ['A', 'B', 'C', 'D', 'A']   (SOLVED)
```

```
discrete_math_ai999x.solve('hamiltonian_cycle', edges='1-2, 2-3, 3-1, 3-4')
# -> exists = False, cycle = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Does the graph with edges A-B, B-C, C-D, D-A, A-C have a Hamiltonian cycle?	SOLVED	exists = True, cycle = ['A', 'B', 'C', 'D', 'A']
Find a Hamiltonian cycle in the graph with edges 1-2, 2-3, 3-1, 3-4.	SOLVED	exists = False, cycle = none

Clique number — `clique_number`

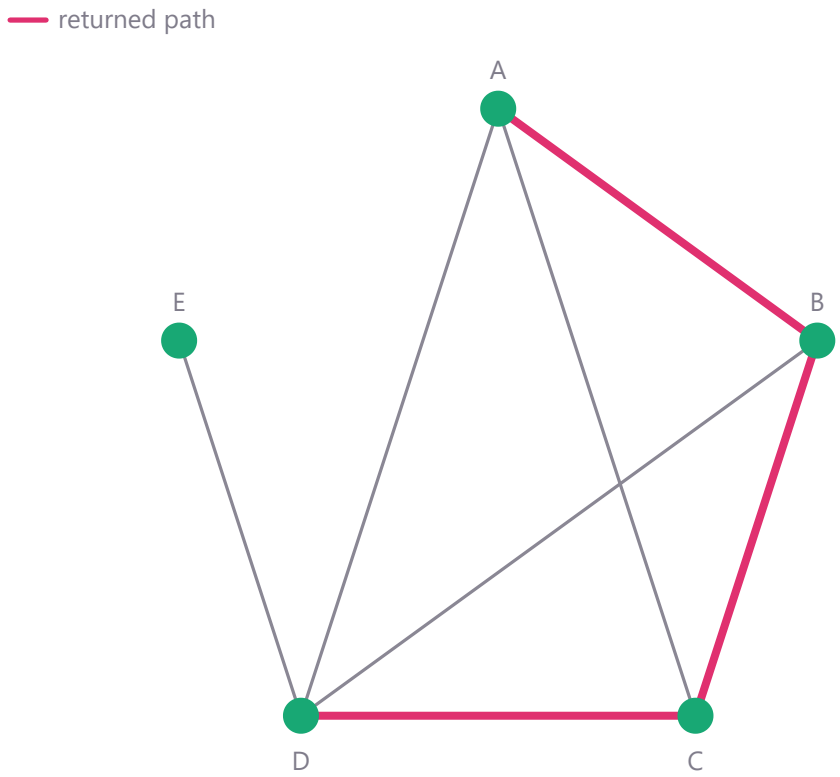
The size of a largest complete subgraph, with one maximum clique.

$$\omega(G) = \max\{|K| : K \text{ complete}\}$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`clique_number`	omega(G)
`clique`	a maximum clique (the lexicographically smallest), ascending

- **Method:** Bron-Kerbosch with pivoting
- **Assumes:** undirected simple graph, unweighted, at most 60 vertices
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 7 edges, laid out on a circle); highlighted: the returned path A → B → C → D. Returned: clique_number = 4.

```
discrete_math_ai999x.solve('clique_number', edges='A-B, B-C, C-A, C-D, B-D, A-D, D-E')
# -> clique_number = 4, clique = ['A', 'B', 'C', 'D']   (SOLVED)
```

```
discrete_math_ai999x.solve('clique_number', edges='1-2, 2-3, 3-4')
# -> clique_number = 2, clique = ['1', '2']   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the clique number of the graph with edges A-B, B-C, C-A, C-D, B-D, A-D, D-E?	SOLVED	clique_number = 4, clique = ['A', 'B', 'C', 'D']
Find a maximum clique in the graph with edges 1-2, 2-3, 3-1, 3-4.	SOLVED	clique_number = 3, clique = ['1', '2', '3']

Independence number — `independence_number`

The size of a largest set of pairwise non-adjacent vertices, with one maximum independent set.

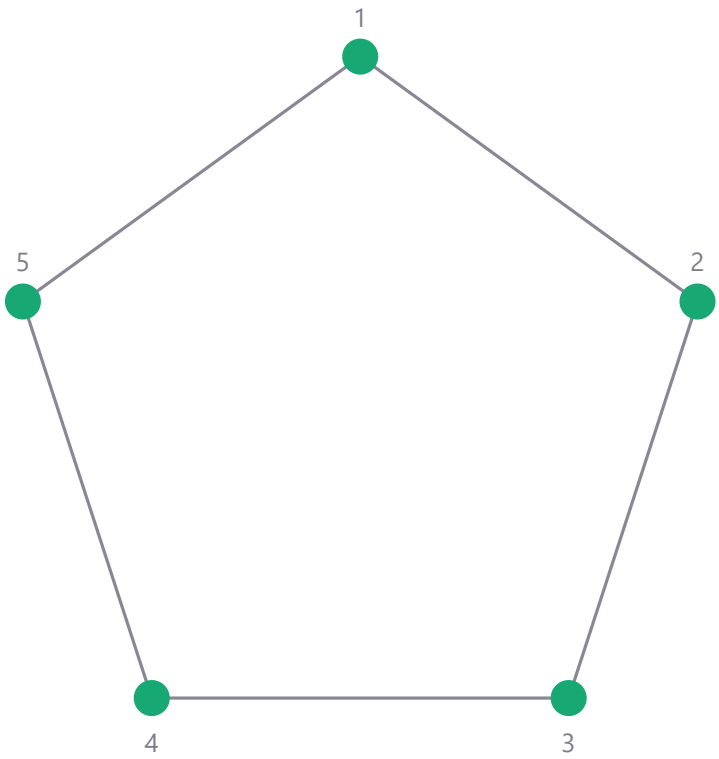
$$\alpha(G) = \omega(\overline{G})$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`independence_number`	alpha(G)

output	meaning
`independent_set`	a maximum independent set (the lexicographically smallest), ascending

- **Method:** maximum clique of the complement graph (Bron-Kerbosch)
- **Assumes:** undirected simple graph, unweighted, at most 60 vertices; the vertices are those named in the edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 5 edges, laid out on a circle); highlighted: the returned path 1 → 3. Returned: independence_number = 2.

```
discrete_math_ai999x.solve('independence_number', edges='1-2, 2-3, 3-4, 4-5, 5-1')
# -> independence_number = 2, independent_set = ['1', '3']   (SOLVED)
```

```
discrete_math_ai999x.solve('independence_number', edges='A-B, A-C, A-D')
# -> independence_number = 3, independent_set = ['B', 'C', 'D']   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the independence number of the graph with edges 1-2, 2-3, 3-4, 4-5, 5-1?	SOLVED	independence_number = 2, independent_set = ['1', '3']
Find a maximum independent set of the graph with edges A-B, A-C, A-D.	SOLVED	independence_number = 3, independent_set = ['B', 'C', 'D']

Girth (shortest cycle length) — `girth`

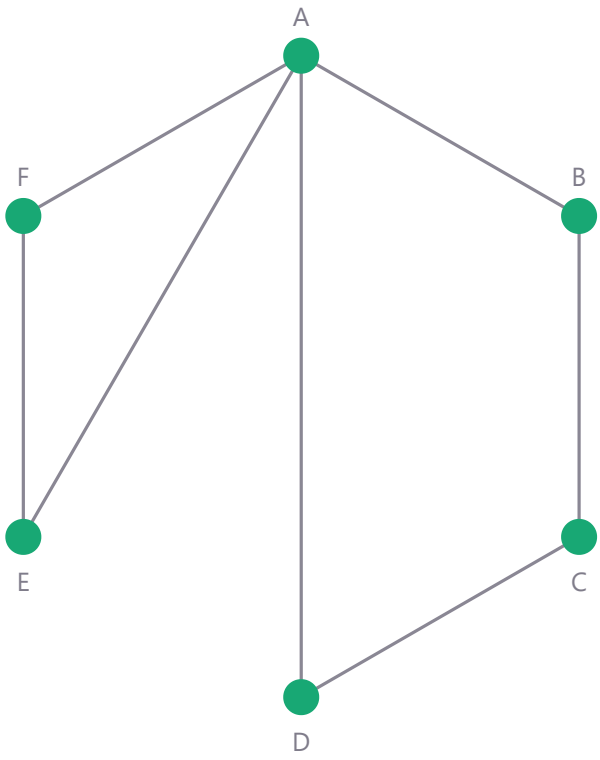
The length of a shortest cycle of an undirected graph (none for a forest).

$$g(G) = \min\{|C| : C \text{ a cycle of } G\}$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`girth`	length of the shortest cycle (none when the graph is acyclic)

- **Method:** breadth-first search from every vertex; a non-tree edge (u, v) closes a cycle of length d(u) + d(v) + 1
- **Assumes:** undirected simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (6 nodes, 7 edges, laid out on a circle). Returned: girth = 3.

```
discrete_math_ai999x.solve('girth', edges='A-B, B-C, C-D, D-A, A-E, E-F, F-A')
# -> girth = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('girth', edges='1-2, 2-3, 3-4, 4-5, 5-1')
# -> girth = 5   (SOLVED)
```

```
discrete_math_ai999x.solve('girth', edges='1-2, 2-3')
# -> girth = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the girth of the graph with edges A-B, B-C, C-D, D-A, A-E, E-F, F-A?	SOLVED	girth = 3
What is the length of the shortest cycle in the graph with edges 1-2, 2-3, 3-4, 4-5, 5-1?	SOLVED	girth = 5

Prüfer code of a labelled tree — `prufer_encode`

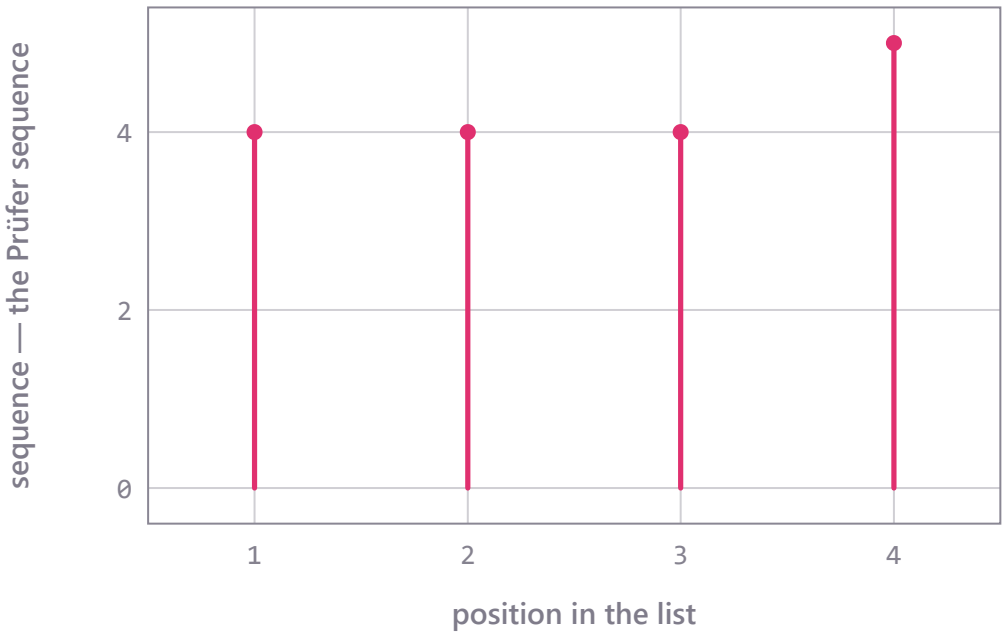
The Prüfer sequence (length n - 2) of a tree on vertices labelled 1..n.

$$(a_1, \ldots, a_{n-2}),\ a_i = \text{neighbour of the smallest leaf}$$

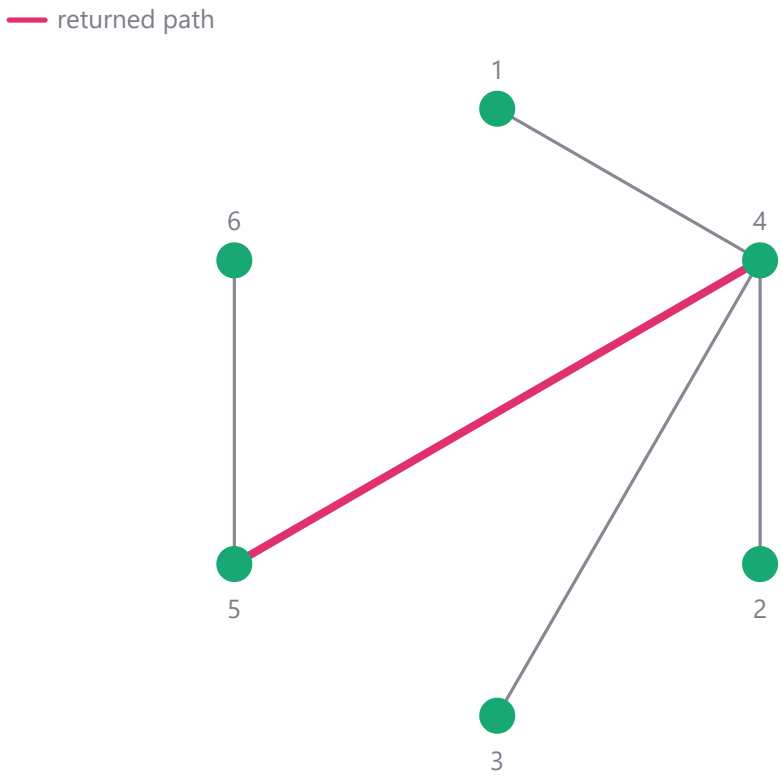
input	meaning	condition
`edges`	the tree's edges, vertices numbered 1..n (or 0..n-1)	

output	meaning
`sequence`	the Prüfer sequence
`vertices`	number of vertices n

- **Method:** repeatedly remove the smallest leaf and record its neighbour
- **Assumes:** a tree whose vertices are numbered 1..n or $0..n-1$; the smallest leaf is removed at every step
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the Prüfer sequence (sequence) returned by discrete-math-ai999x for the worked example, one value for each position in the list (4 values, first 4, last 5, largest 5). Inputs: edges = 1-4, 2-4, 3-4, 4-5, 5-6.



The network edges (the tree's edges, vertices numbered 1..n (or 0..n-1)) of the worked example (6 nodes, 5 edges, laid out on a circle); highlighted: the returned path 4 → 4 → 4 → 5. Returned: vertices = 6.

```
discrete_math_ai999x.solve('prufer_encode', edges='1-4, 2-4, 3-4, 4-5, 5-6')
# -> sequence = [4, 4, 4, 5], vertices = 6   (SOLVED)
```

```
discrete_math_ai999x.solve('prufer_encode', edges='1-2, 2-3, 3-4')
# -> sequence = [2, 3], vertices = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Prüfer code of the tree with edges 1-4, 2-4, 3-4, 4-5, 5-6.	SOLVED	sequence = [4, 4, 4, 5], vertices = 6
Find the Prufer sequence of the tree with edges 1-2, 2-3, 3-1.	INVALID_INPUT	Prüfer code of a labelled tree: the graph is not a tree.

Tree from a Prüfer code — `prufer_decode`

The labelled tree on 1..n (n = length + 2) whose Prüfer sequence is given.

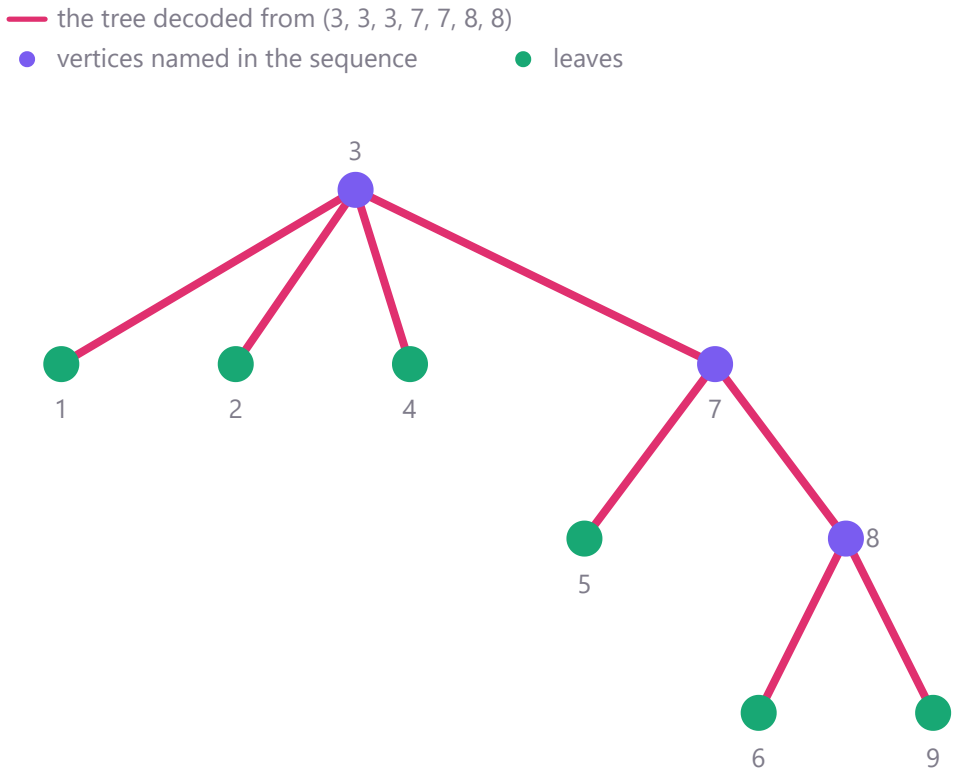
$$\deg(v) = 1 + \#\{i : a_i = v\}$$

input	meaning	condition
`sequence`	the Prüfer sequence (entries between 1 and n)	

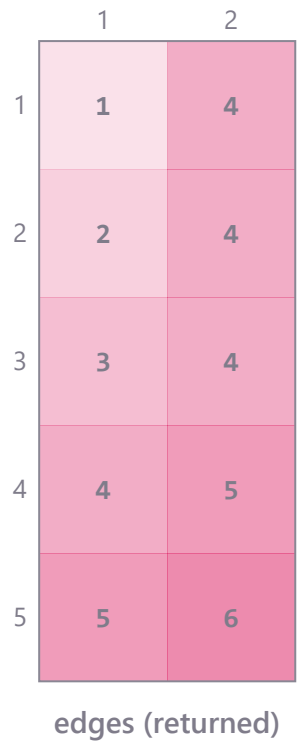
output	meaning
`edges`	the tree's edges [u, v] with $u < v$, sorted

output	meaning
`vertices`	number of vertices n

- **Method:** attach each entry to the smallest current leaf; join the last two leaves
- **Assumes:** vertices are numbered 1..n with n = length + 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The labelled tree that the package decodes from the Prüfer sequence (3, 3, 3, 7, 7, 8, 8): 9 vertices, edges 1–3, 2–3, 3–4, 3–7, 5–7, 6–8, 7–8, 8–9. A vertex appears in the sequence one time fewer than its degree, so the leaves are exactly the labels the sequence never names.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): edges (returned). Returned with them by discrete-math-ai999x: vertices = 6. Other inputs: sequence = 4, 4, 4, 5.

```
discrete_math_ai999x.solve('prufer_decode', sequence='4, 4, 4, 5')
# -> edges = [[1, 4], [2, 4], [3, 4], [4, 5], [5, 6]], vertices = 6   (SOLVED)
```

```
discrete_math_ai999x.solve('prufer_decode', sequence='3, 3, 3')
# -> edges = [[1, 3], [2, 3], [3, 4], [3, 5]], vertices = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decode the Prüfer sequence [4, 4, 4, 5].	SOLVED	edges = [[1, 4], [2, 4], [3, 4], [4, 5], [5, 6]], vertices = 6
Find the tree whose Prüfer code is 3, 3, 3.	SOLVED	edges = [[1, 3], [2, 3], [3, 4], [3, 5]], vertices = 5
Decode the Prüfer sequence [9, 1].	INVALID_INPUT	Tree from a Prüfer code: every entry must be between 1 and n = 4.

Graphical degree sequence — `graphical_sequence`

Whether a list of non-negative integers is the degree sequence of some simple graph.

$$\sum_{i \le k} d_i \le k(k-1) + \sum_{i > k} \min(d_i, k) \;\; \forall k, \quad \sum d_i \text{ even}$$

input	meaning	condition
`degrees`	the proposed degrees (whole numbers >= 0)	

output	meaning
`is_graphical`	true when a simple graph has exactly these degrees

- **Method:** Havel-Hakimi reduction
- **Assumes:** whole numbers >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Graphical degree sequence as inputs and results: stated degrees = 3, 3, 2, 2, 2; returned by discrete-math-ai999x: is_graphical = True.

```
discrete_math_ai999x.solve('graphical_sequence', degrees='3, 3, 2, 2, 2')
# -> is_graphical = True   (SOLVED)
```

```
discrete_math_ai999x.solve('graphical_sequence', degrees='4, 4, 1, 1')
# -> is_graphical = False   (SOLVED)
```

```
discrete_math_ai999x.solve('graphical_sequence', degrees='3, 3, 3, 1')
# -> is_graphical = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the sequence 3, 3, 2, 2, 2 graphical?	SOLVED	is_graphical = True
Is there a simple graph with degrees 4, 4, 1, 1?	SOLVED	is_graphical = False

Number of walks of a given length — `walks_count`

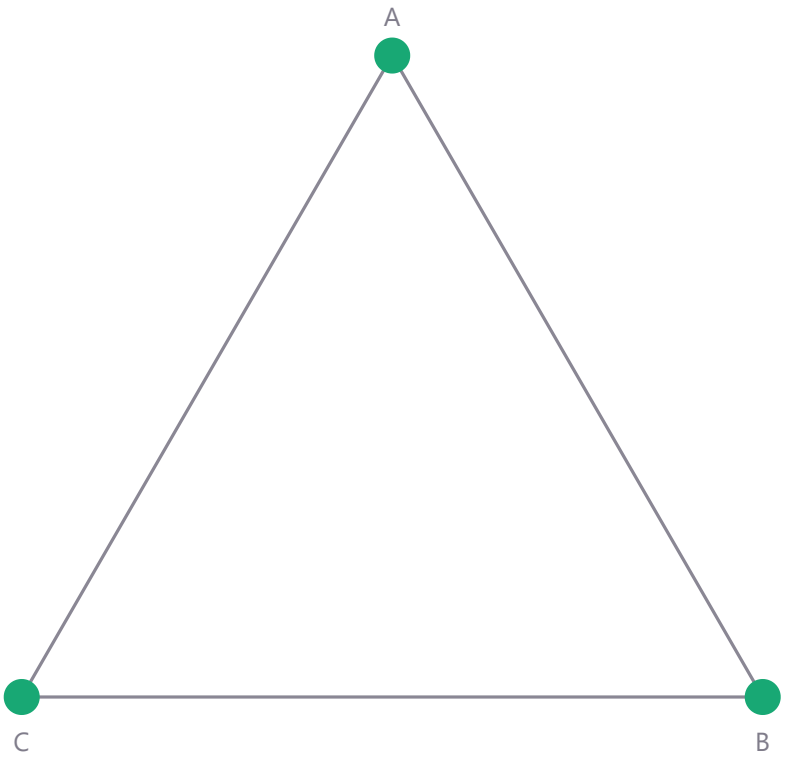
How many walks of exactly k edges lead from u to v (the (u, v) entry of A^k).

$$\#\mathrm{walks}_k(u, v) = (A^k)_{uv}$$

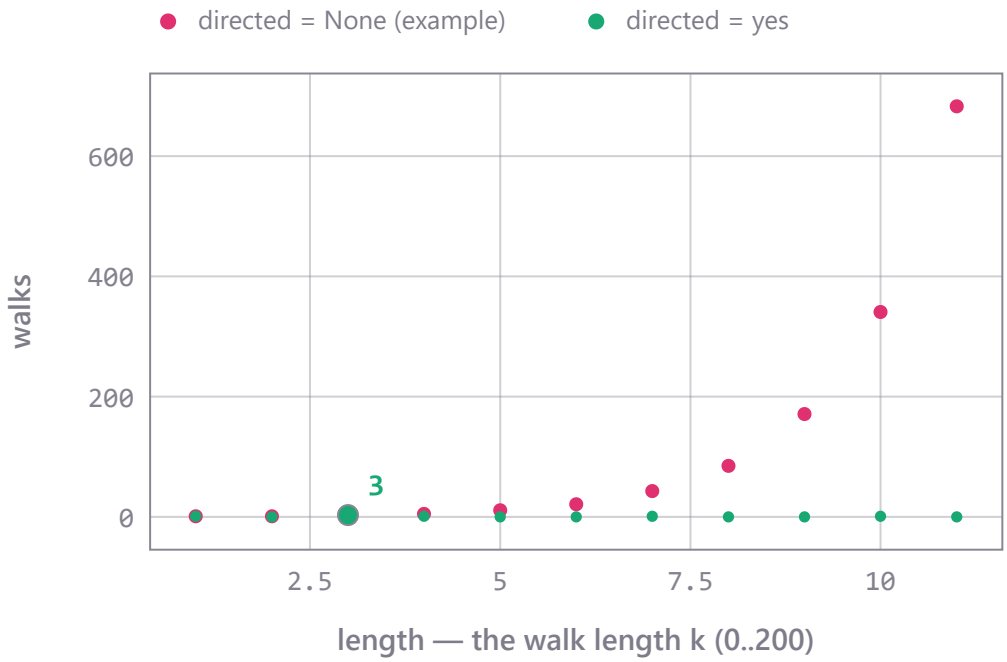
input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	
`source`	start vertex u	
`target`	end vertex v	
`length`	the walk length k (0..200)	
`directed`	true for a directed graph (edges u->v)	

output	meaning
`walks`	number of walks of length k from u to v

- **Method:** propagate walk counts one step at a time (a vector times the adjacency matrix, k times)
- **Assumes:** simple unweighted graph; vertices may repeat along a walk
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (3 nodes, 3 edges, laid out on a circle). Returned: walks = 3. Other inputs: source = A, target = B, length = 3.



walks (number of walks of length k from u to v) against length (the walk length k (0..200)) from 1 to 11. One curve for each value of directed (true for a directed graph (edges u->v)): None, yes. Other inputs as in the worked example: edges = A-B, B-C, C-A, source = A, target = B. 11 points per curve computed by discrete-math-ai999x; marked: the worked example, length = 3, walks = 3.

```
discrete_math_ai999x.solve('walks_count', edges='A-B, B-C, C-A', source='A', target='B', length=3)
# -> walks = 3   (SOLVED)
```

```
discrete_math_ai999x.solve('walks_count', edges='1-2, 2-3', source='1', target='1', length=2)
# -> walks = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many walks of length 3 from A to B are there in the graph with edges A-B, B-C, C-A?	SOLVED	walks = 3
How many walks of length 2 from 1 to 3 are there in the directed graph with edges 1->2, 2->3, 1->3?	SOLVED	walks = 1

Number of triangles — `graph_triangles`

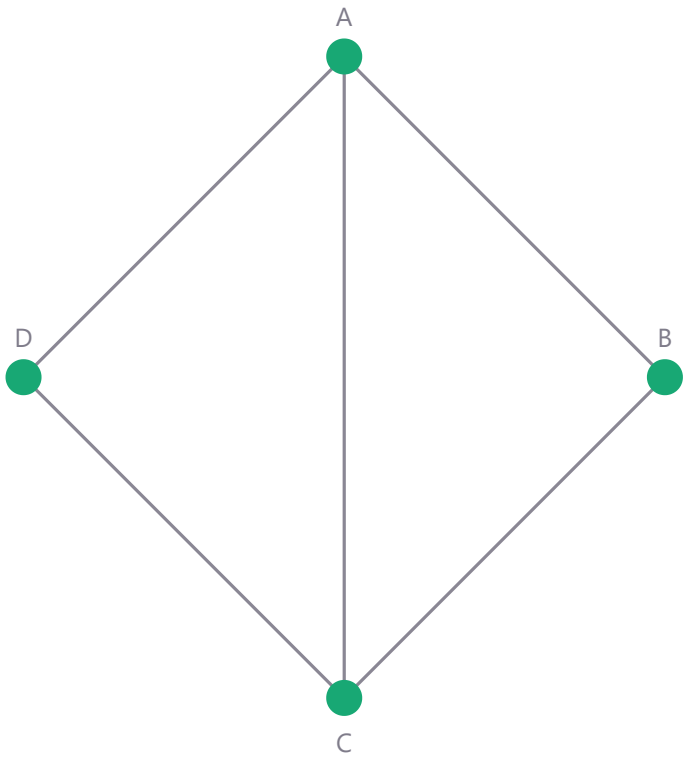
How many triangles (3-cycles) an undirected graph contains.

$$t(G) = \tfrac{1}{6}\mathrm{tr}(A^3)$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`triangles`	number of triangles

- **Method:** for each edge, count common neighbours; every triangle is seen from its three edges
- **Assumes:** undirected simple graph, unweighted
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (4 nodes, 5 edges, laid out on a circle). Returned: triangles = 2.

```
discrete_math_ai999x.solve('graph_triangles', edges='A-B, B-C, C-A, C-D, D-A')
# -> triangles = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('graph_triangles', edges='1-2, 1-3, 1-4, 2-3, 2-4, 3-4')
# -> triangles = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many triangles does the graph with edges A-B, B-C, C-A, C-D, D-A have?	SOLVED	triangles = 2
Count the triangles in the graph with edges 1-2, 1-3, 1-4, 2-3, 2-4, 3-4.	SOLVED	triangles = 4

Tree test — `is_tree`

Whether an undirected graph is a tree: connected with exactly $n - 1$ edges (equivalently, connected and acyclic).

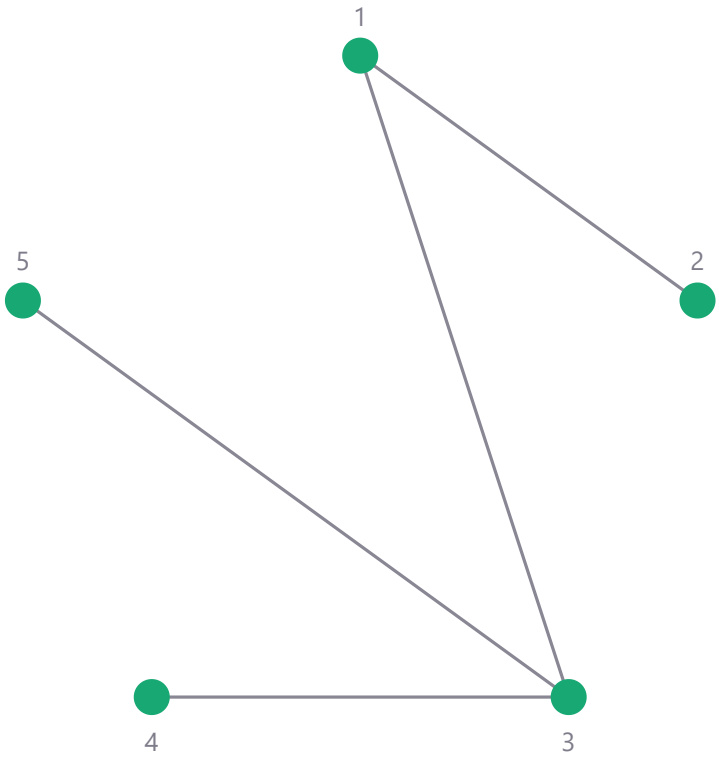
$$T \text{ tree} \iff T \text{ connected} \wedge |E| = |V| - 1$$

input	meaning	condition
`edges`	the graph as a list of edges u-v (an optional third entry is the weight)	

output	meaning
`is_tree`	true when the graph is a tree
`vertices`	number of vertices

output	meaning
`edges`	number of edges
`connected`	true when the graph is connected

- **Method:** count edges and test connectivity
- **Assumes:** undirected simple graph, unweighted; the vertices are those named in the edges
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network edges (the graph as a list of edges u-v (an optional third entry is the weight)) of the worked example (5 nodes, 4 edges, laid out on a circle). Returned: vertices = 5, edges = 4.

```
discrete_math_ai999x.solve('is_tree', edges='1-2, 1-3, 3-4, 3-5')
# -> is_tree = True, vertices = 5, edges = 4, connected = True   (SOLVED)
```

```
discrete_math_ai999x.solve('is_tree', edges='A-B, B-C, C-A, D-E')
# -> is_tree = False, vertices = 5, edges = 4, connected = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the graph with edges 1-2, 1-3, 3-4, 3-5 a tree?	SOLVED	is_tree = True, vertices = 5, edges = 4, connected = True
Is the graph with edges A-B, B-C, C-A, D-E a tree?	SOLVED	is_tree = False, vertices = 5, edges = 4, connected = False

Evaluate a propositional formula — `logic_evaluate`

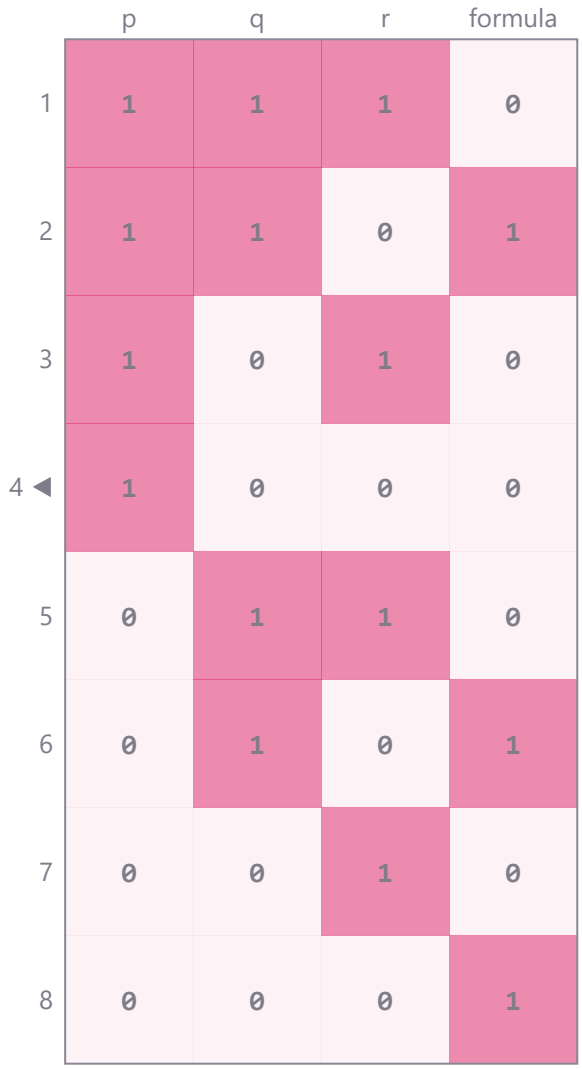
The truth value of a propositional formula under a given assignment of true/false to its variables.

$$p \to q \equiv \neg p \lor q$$

input	meaning	condition
`formula`	the formula, e.g. (p -> q) & ~r (operators ~ & | ^ -> <->, or not/and/or/xor/implies/iff)	
`assignment`	a truth value for every variable, e.g. 'p = true, $q = 0$, r = false'	

output	meaning
`value`	the formula's truth value

- **Method:** recursive-descent parse into a syntax tree, evaluated bottom-up
- **Assumes:** every variable gets a value and no other name is given; precedence: ~ binds tightest, then &, ^, |, -> (right-associative), <-> loosest
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of (p -> q) & ~r (1 = true), every row a run of the package; the stated assignment (p = true, q = false, r = false) is row 4 (◀), for which the package returned False. Marked: that row.

```
discrete_math_ai999x.solve('logic_evaluate', formula='(p -> q) & ~r', assignment='p = true, q =
false, r = false')
# -> value = False   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_evaluate', formula='p | q -> r', assignment='p = 1, q = 0, r = 0')
# -> value = False   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_evaluate', formula='A <-> ~B', assignment='A = true, B = false')
# -> value = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate (p -> q) & ~r when p = true, q = false, r = false.	SOLVED	value = False
What is the truth value of p | q -> r for p = 1, q = 0, r = 0?	SOLVED	value = False
Evaluate p & q when p = true.	INVALID_INPUT	Evaluate a propositional formula: no truth value given for q.

Truth table and minterms — `truth_table`

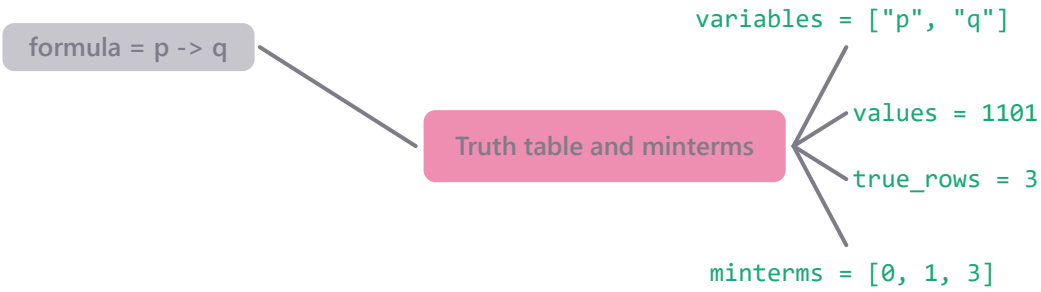
The full truth table of a propositional formula, the number of true rows and its minterms.

$$f = \sum m(\text{true rows})$$

input	meaning	condition
`formula`	the formula (operators ~ & | ^ -> <->, or words)	

output	meaning
`variables`	the variables in alphabetical order
`values`	the formula's value on rows 0, 1, ... as a $0/1$ string
`true_rows`	number of true rows
`minterms`	indices of the true rows

- **Method:** evaluate the syntax tree on every assignment
- **Assumes:** at most 12 variables; row r assigns the binary digits of r to the variables, the first (alphabetically) variable being the most significant bit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Truth table and minterms as inputs and results: stated formula = p -> q; returned by discrete-math-ai999x: variables = ["p", "q"], values = 1101, true_rows = 3, minterms = [0, 1, 3].

```
discrete_math_ai999x.solve('truth_table', formula='p -> q')
# -> variables = ['p', 'q'], values = 1101, true_rows = 3, minterms = [0, 1, 3]   (SOLVED)
```

```
discrete_math_ai999x.solve('truth_table', formula='A & ~B | C')
# -> variables = ['A', 'B', 'C'], values = 01011101, true_rows = 5, minterms = [1, 3, 4, 5, 7]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Give the truth table of (p -> q) & (q -> r).	SOLVED	variables = ['p', 'q', 'r'], values = 11010001, true_rows = 4, minterms = [0, 1, 3, 7]
Find the minterms of A & ~B | C.	SOLVED	variables = ['A', 'B', 'C'], values = 01011101, true_rows = 5, minterms = [1, 3, 4, 5, 7]
Give the truth table of p & & q.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: discrete_math_ai999x.templates();

Satisfiability of a propositional formula — `logic_satisfiable`

Whether some assignment makes the formula true, how many do, and the first such assignment.

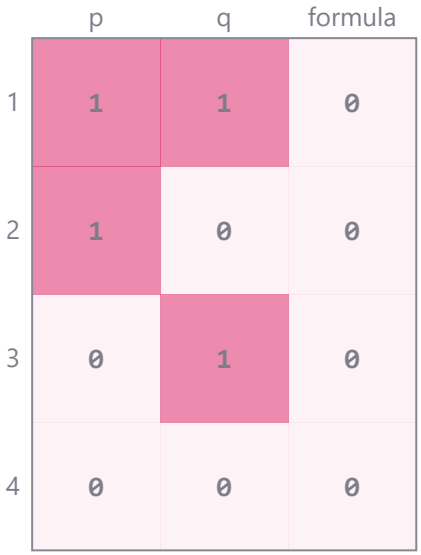
$$\exists v:\ f(v) = 1$$

input	meaning	condition
`formula`	the formula (operators ~ & | ^ -> <->, or words)	

output	meaning
`satisfiable`	true when some assignment satisfies the formula
`models`	number of satisfying assignments
`first_model`	the satisfying assignment with the smallest row number (none if unsatisfiable)

- **Method:** exhaustive truth table

- **Assumes:** at most 12 variables; rows are numbered with the first (alphabetical) variable as the most significant bit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of (p | q) & ~p & ~q over every assignment of p, q (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (4 runs). Satisfiable: the package returned satisfiable = False, models = 0, first_model = None.

```
discrete_math_ai999x.solve('logic_satisfiable', formula='(p | q) & ~p & ~q')
# -> satisfiable = False, models = 0, first_model = none   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_satisfiable', formula='(A | B) & (~A | C) & ~C')
# -> satisfiable = True, models = 1, first_model = {'A': False, 'B': True, 'C': False}   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is (p | q) & ~p & ~q satisfiable?	SOLVED	satisfiable = False, models = 0, first_model = none
Find a satisfying assignment for (A | B) & (~A | C) & ~C.	SOLVED	satisfiable = True, models = 1, first_model = {'A': False, 'B': True, 'C': False}

Tautology test — `logic_tautology`

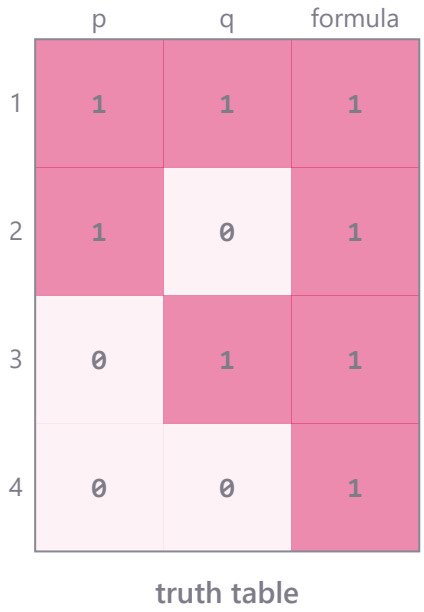
Whether a formula is true under every assignment, and the first assignment that falsifies it if not.

$$\models f \iff \forall v:\ f(v) = 1$$

input	meaning	condition
`formula`	the formula (operators ~ & | ^ -> <->, or words)	

output	meaning
`tautology`	true when every assignment satisfies the formula
`false_rows`	number of falsifying assignments
`counterexample`	the falsifying assignment with the smallest row number (none for a tautology)

- **Method:** exhaustive truth table
- **Assumes:** at most 12 variables
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of (p -> q) | (q -> p) over every assignment of p, q (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (4 runs). Tautology: the package returned tautology = True, false_rows = 0, counterexample = None.

```
discrete_math_ai999x.solve('logic_tautology', formula='(p -> q) | (q -> p)')
# -> tautology = True, false_rows = 0, counterexample = none   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_tautology', formula='(p -> q) -> p')
# -> tautology = False, false_rows = 2, counterexample = {'p': False, 'q': False}   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_tautology', formula='((p -> q) -> p) -> p')
# -> tautology = True, false_rows = 0, counterexample = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is (p -> q) | (q -> p) a tautology?	SOLVED	tautology = True, false_rows = 0, counterexample = none
Is the formula (p -> q) -> p valid?	SOLVED	tautology = False, false_rows = 2, counterexample = {'p': False, 'q': False}

Contradiction test — `logic_contradiction`

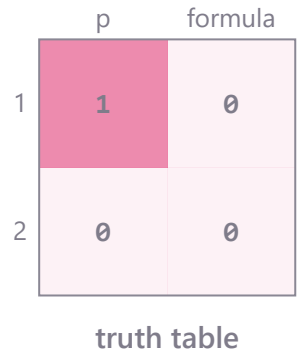
Whether a formula is false under every assignment (unsatisfiable), with a satisfying assignment if not.

$$\forall v:\ f(v) = 0$$

input	meaning	condition
`formula`	the formula (operators ~ & | ^ -> <->, or words)	

output	meaning
`contradiction`	true when no assignment satisfies the formula
`true_rows`	number of satisfying assignments
`witness`	the satisfying assignment with the smallest row number (none for a contradiction)

- **Method:** exhaustive truth table
- **Assumes:** at most 12 variables
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of p & ~p over every assignment of p (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (2 runs). Contradiction: the package returned contradiction = True, true_rows = 0, witness = None.

```
discrete_math_ai999x.solve('logic_contradiction', formula='p & ~p')
# -> contradiction = True, true_rows = 0, witness = none   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_contradiction', formula='(p -> q) & p & ~q')
# -> contradiction = True, true_rows = 0, witness = none   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_contradiction', formula='p & q')
# -> contradiction = False, true_rows = 1, witness = {'p': True, 'q': True}   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is p & ~p a contradiction?	SOLVED	contradiction = True, true_rows = 0, witness = none
Is (p -> q) & p & ~q unsatisfiable?	SOLVED	contradiction = True, true_rows = 0, witness = none

Logical equivalence of two formulas — `logic_equivalence`

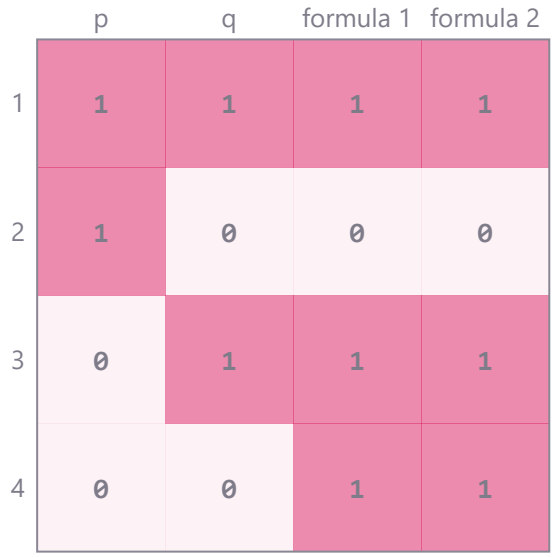
Whether two formulas have the same truth value under every assignment, with a distinguishing assignment if not.

$$f \equiv g \iff \models f \leftrightarrow g$$

input	meaning	condition
`formula1`	the first formula	
`formula2`	the second formula	

output	meaning
`equivalent`	true when the formulas agree on every assignment
`differing_rows`	number of assignments where they differ
`counterexample`	the first distinguishing assignment (none if equivalent)

- **Method:** compare the two truth tables
- **Assumes:** at most 12 variables in total; rows range over the union of both formulas' variables
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



truth table

Truth table of p -> q and ~p | q over every assignment of p, q (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (8 runs). Equivalent: the package returned equivalent = True, differing_rows = 0, counterexample = None.

```
discrete_math_ai999x.solve('logic_equivalence', formula1='p -> q', formula2='~p | q')
# -> equivalent = True, differing_rows = 0, counterexample = none   (SOLVED)
```

```
discrete_math_ai999x.solve('logic_equivalence', formula1='p -> q', formula2='q -> p')
# -> equivalent = False, differing_rows = 2, counterexample = {'p': False, 'q': True}   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Are (p -> q) and (~p | q) logically equivalent?	SOLVED	equivalent = True, differing_rows = 0, counterexample = none
Is p -> q equivalent to q -> p?	SOLVED	equivalent = False, differing_rows = 2, counterexample = {'p': False, 'q': True}
Is ~(p & q) equivalent to ~p | ~q?	SOLVED	equivalent = True, differing_rows = 0, counterexample = none

Minimal sum of products (DNF) — `minimal_dnf`

A minimum disjunctive normal form of a formula: fewest product terms, then fewest literals.

$$f = \sum_{\text{cover}} \prod \ell_i$$

input	meaning	condition
`formula`	the formula (at most 6 variables)	

output	meaning
`expression`	a minimal sum of products (terms joined by |, literals by &)
`terms`	number of product terms
`literals`	number of literals

- **Method:** Quine-McCluskey prime implicants, then an exact branch-and-bound minimum cover
- **Assumes:** at most 6 variables; cost: fewest terms first, then fewest literals; when several minimal forms exist one is shown (all have these counts)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	A	B	C	formula 1	formula 2
1	1	1	1	1	1
2	1	1	0	1	1
3	1	0	1	1	1
4	1	0	0	0	0
5	0	1	1	0	0
6	0	1	0	0	0
7	0	0	1	0	0
8	0	0	0	0	0

truth table

Truth table of (A & B) | (A & ~B & C) and A & B | A & C over every assignment of A, B, C (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (16 runs). Minimal form (last column): the package returned expression = A & B | A & C, terms = 2, literals = 4.

```
discrete_math_ai999x.solve('minimal_dnf', formula='(A & B) | (A & ~B & C)')
# -> expression = A & B | A & C, terms = 2, literals = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('minimal_dnf', formula='(p & q) | (~p & q) | (p & ~q)')
# -> expression = p | q, terms = 2, literals = 2   (SOLVED)
```

```
discrete_math_ai999x.solve('minimal_dnf', formula='p | ~p')
# -> expression = 1, terms = 1, literals = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the minimal DNF of (A & B) | (A & ~B & C).	SOLVED	expression = A & B | A & C, terms = 2, literals = 4

question	outcome	result
Find the minimal sum of products of (p & q) | (~p & q) | (p & ~q).	SOLVED	expression = p | q, terms = 2, literals = 2

Minimal product of sums (CNF) — `minimal_cnf`

A minimum conjunctive normal form of a formula: fewest clauses, then fewest literals.

$$f = \prod_{\text{cover of } \bar{f}} \sum \bar{\ell}_i$$

input	meaning	condition
`formula`	the formula (at most 6 variables)	

output	meaning
`expression`	a minimal product of sums (clauses joined by &)
`clauses`	number of clauses
`literals`	number of literals

- **Method:** minimal sum of products of the negation, complemented (De Morgan)
- **Assumes:** at most 6 variables; cost: fewest clauses first, then fewest literals
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

	A	B	C	formula 1	formula 2
1	1	1	1	1	1
2	1	1	0	1	1
3	1	0	1	1	1
4	1	0	0	1	1
5	0	1	1	1	1
6	0	1	0	0	0
7	0	0	1	0	0
8	0	0	0	0	0

truth table

Truth table of (A | B) & (A | ~B | C) and (A | B) & (A | C) over every assignment of A, B, C (1 = true, 0 = false; rows from all true to all false); each formula column is the value the package's evaluator returned for that row (16 runs). Minimal form (last column): the package returned expression = (A | B) & (A | C), clauses = 2, literals = 4.

```
discrete_math_ai999x.solve('minimal_cnf', formula='(A | B) & (A | ~B | C)')
# -> expression = (A | B) & (A | C), clauses = 2, literals = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('minimal_cnf', formula='p ^ q')
# -> expression = (p | q) & (~p | ~q), clauses = 2, literals = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the minimal CNF of (A | B) & (A | ~B | C).	SOLVED	expression = (A | B) & (A | C), clauses = 2, literals = 4
Find the minimal product of sums of p ^ q.	SOLVED	expression = (p | q) & (~p | ~q), clauses = 2, literals = 4

Minimal sum of products from minterms (with don't-cares) — `minimal_sop_minterms`

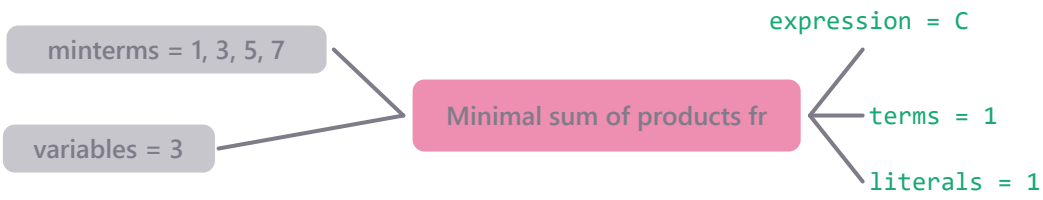
The minimal sum of products of a Boolean function given by its minterms and optional don't-care rows (Karnaugh-map problems).

$$F(A,B,C) = \sum m(\ldots) + d(\ldots)$$

input	meaning	condition
`minterms`	rows where the function is 1	
`variables`	number of variables (1..6), named A, B, C, ...	
`dont_cares`	rows whose value does not matter (optional)	

output	meaning
`expression`	a minimal sum of products over A, B, C, ... (A is the most significant bit)
`terms`	number of product terms
`literals`	number of literals

- **Method:** Quine-McCluskey prime implicants (don't-cares included), exact minimum cover of the minterms
- **Assumes:** 1..6 variables named A, B, C, D, E, F with A the most significant bit of the row number; cost: fewest terms first, then fewest literals; don't-cares may be used or not
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Minimal sum of products from minterms (with don't-cares) as inputs and results: stated minterms = 1, 3, 5, 7, variables = 3; returned by discrete-math-ai999x: expression = C, terms = 1, literals = 1.

```
discrete_math_ai999x.solve('minimal_sop_minterms', minterms='1, 3, 5, 7', variables=3)
# -> expression = C, terms = 1, literals = 1   (SOLVED)
```

```
discrete_math_ai999x.solve('minimal_sop_minterms', minterms='0, 2, 8, 10, 13, 15', variables=4,
dont_cares='5, 7')
# -> expression = B & D | ~B & ~D, terms = 2, literals = 4   (SOLVED)
```

```
discrete_math_ai999x.solve('minimal_sop_minterms', minterms='0, 1, 2, 5, 6, 7', variables=3)
# -> expression = A & C | ~A & ~B | B & ~C, terms = 3, literals = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the minimal sum of products for F(A, B, C) = Σm(1, 3, 5, 7).	SOLVED	expression = C, terms = 1, literals = 1
Minimize F(A, B, C, D) = Σm(0, 2, 8, 10, 13, 15) with don't cares d(5, 7).	SOLVED	expression = B & D | ~B & ~D, terms = 2, literals = 4
Minimize F(A, B) = Σm(0, 9).	INVALID_INPUT	Minimal sum of products from minterms (with don't-cares): minterms must be at most 3 in size.

RSA key from p, q and e — `rsa_keygen`

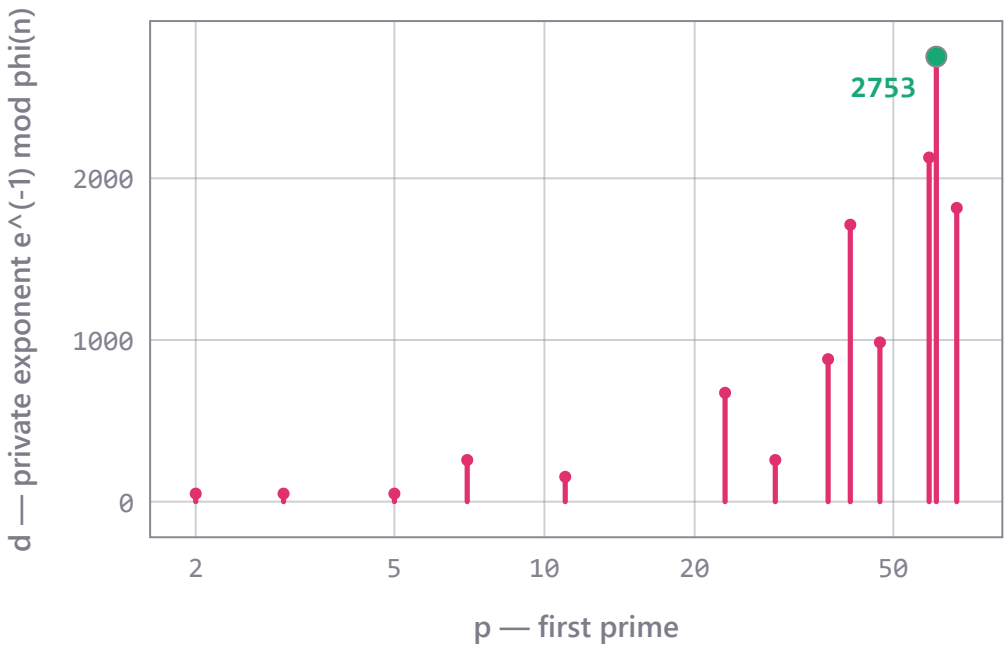
The RSA modulus $n = p$ q, phi(n) = (p-1)$(q - 1)$ and the private exponent $d = e^{-1}$ mod phi(n).

$$n = pq,\ \varphi = (p-1)(q-1),\ d \equiv e^{-1} \pmod{\varphi}$$

input	meaning	condition
`p`	first prime	
`q`	second prime	
`e`	public exponent	

output	meaning
`n`	modulus p q
`phi`	Euler's phi(n) = (p-1)$(q - 1)$
`d`	private exponent e^{-1} mod phi(n)

- **Method:** extended Euclidean algorithm for the inverse of e modulo phi(n)
- **Assumes:** p, q distinct primes; $1 < e < \phi$(n) and gcd(e, phi(n)) = 1; d is taken modulo phi(n) (not lambda(n))
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (private exponent e^(-1) mod phi(n)) against p (first prime) from 1 to 122 on a logarithmic axis. Other inputs as in the worked example: q = 53, e = 17. 13 points computed by discrete-math-ai999x; marked: the worked example, p = 61, d = 2753.

```
discrete_math_ai999x.solve('rsa_keygen', p=61, q=53, e=17)
# -> n = 3233, phi = 3120, d = 2753   (SOLVED)
```

```
discrete_math_ai999x.solve('rsa_keygen', p=11, q=13, e=7)
# -> n = 143, phi = 120, d = 103   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the RSA private key d for p = 61, q = 53, e = 17.	SOLVED	n = 3233, phi = 3120, d = 2753
Find the RSA private key d for p = 61, q = 55, e = 17.	INVALID_INPUT	RSA key from p, q and e: q must be prime.
Find the RSA private key d for p = 61, q = 53.	NEEDS_INPUT	rsa_keygen needs e (public exponent). Example: solve('rsa_keygen', p=61, q=53, e=17).

RSA encryption — `rsa_encrypt`

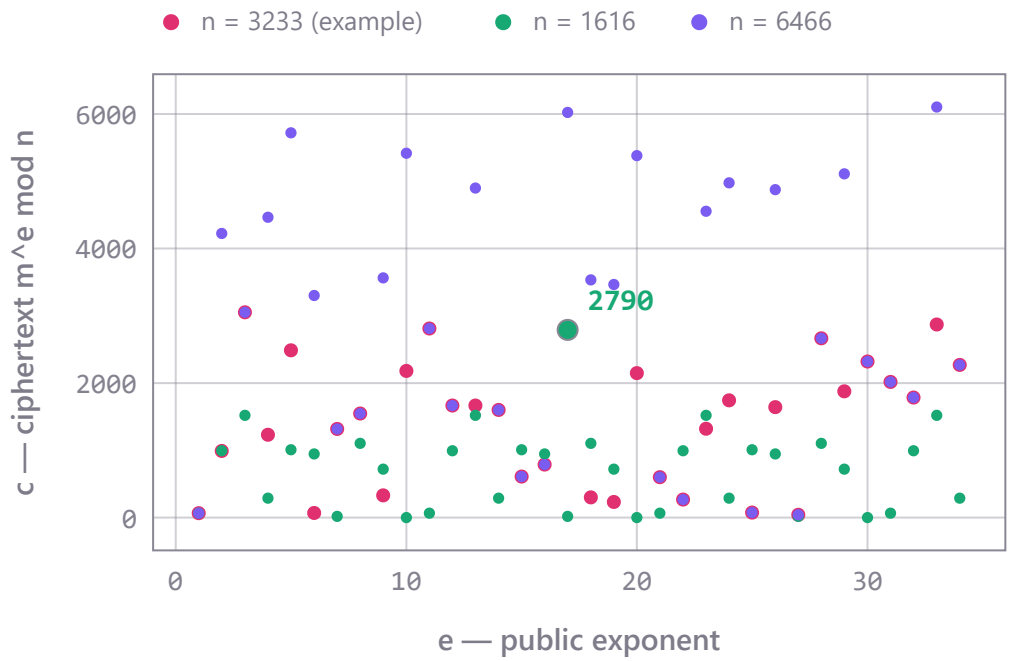
The RSA ciphertext $c = m^e$ mod n of a message number m.

$$c = m^e \bmod n$$

input	meaning	condition
`m`	the message as a number $(0 \le m < n)$	
`e`	public exponent	
`n`	modulus	

output	meaning
c	ciphertext m^e mod n

- **Method:** modular exponentiation
- **Assumes:** $0 \leq m < n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



c (ciphertext m^e mod n) against e (public exponent) from 1 to 34. One curve for each value of n (modulus): 3233, 1616, 6466. Other inputs as in the worked example: m = 65. 34 points per curve computed by discrete-math-ai999x; marked: the worked example, e = 17, c = 2790.

```
discrete_math_ai999x.solve('rsa_encrypt', m=65, e=17, n=3233)
# -> c = 2790   (SOLVED)
```

```
discrete_math_ai999x.solve('rsa_encrypt', m=9, e=7, n=143)
# -> c = 48   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RSA: encrypt m = 65 with e = 17 and n = 3233.	SOLVED	c = 2790
RSA: encrypt m = 4000 with e = 17 and n = 3233.	INVALID_INPUT	RSA encryption: the message m must satisfy 0 <= m < n.

RSA decryption — `rsa_decrypt`

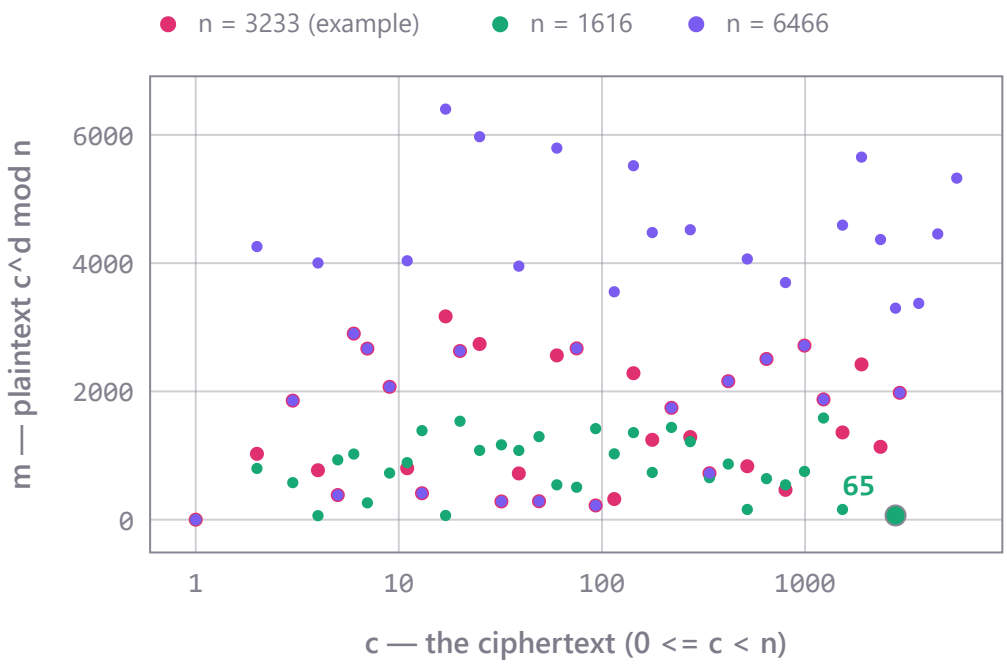
The RSA plaintext $m = c^d$ mod n of a ciphertext c.

$$m = c^d \bmod n$$

input	meaning	condition
`c`	the ciphertext $(0 \leq c < n)$	
`d`	private exponent	
`n`	modulus	

output	meaning
`m`	plaintext c^d mod n

- **Method:** modular exponentiation
- **Assumes:** $0 \leq c < n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m (plaintext c^d mod n) against c (the ciphertext (0 <= c < n)) from 1 to 5580 on a logarithmic axis. One curve for each value of n (modulus): 3233, 1616, 6466. Other inputs as in the worked example: d = 2753. 36 points per curve computed by discrete-math-ai999x; marked: the worked example, c = 2790, m = 65.

```
discrete_math_ai999x.solve('rsa_decrypt', c=2790, d=2753, n=3233)
# -> m = 65   (SOLVED)
```

```
discrete_math_ai999x.solve('rsa_decrypt', c=48, d=103, n=143)
# -> m = 9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RSA: decrypt c = 2790 with d = 2753 and n = 3233.	SOLVED	m = 65

RSA decryption from p, q and e (CRT) — `rsa_decrypt_pq`

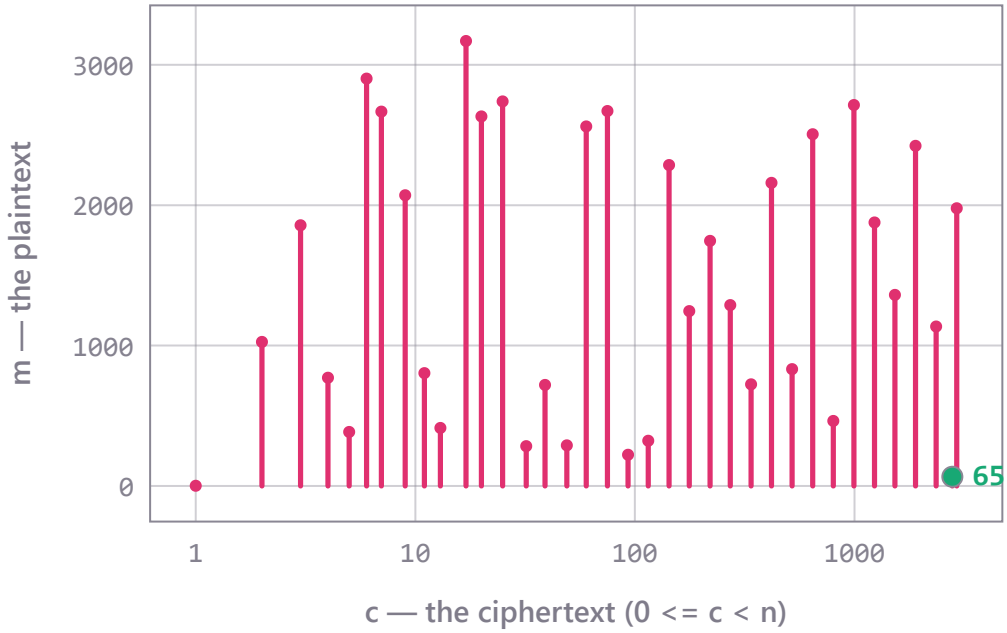
Decrypt an RSA ciphertext knowing the primes: compute d, then m by the Chinese remainder theorem.

$$m = m_q + q\left(q^{-1}(m_p - m_q) \bmod p\right)$$

input	meaning	condition
c	the ciphertext $(0 \le c < n)$	
p	first prime	
q	second prime	
e	public exponent	

output	meaning
m	the plaintext
d	private exponent e^{-1} mod phi(n)
n	modulus p q

- **Method:** $d = e^{-1}$ mod phi; m_p = c^(d mod p-1) mod p, m_q likewise; Garner's recombination
- **Assumes:** p, q distinct primes; gcd(e, phi(n)) = 1; $0 \le c < n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m (the plaintext) against c (the ciphertext (0 <= c < n)) from 1 to 5580 on a logarithmic axis. Other inputs as in the worked example: p = 61, q = 53, e = 17. 36 points computed by discrete-math-ai999x; marked: the worked example, c = 2790, m = 65.

```
discrete_math_ai999x.solve('rsa_decrypt_pq', c=2790, p=61, q=53, e=17)
# -> m = 65, d = 2753, n = 3233   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RSA: decrypt c = 2790 with p = 61, q = 53 and e = 17.	SOLVED	m = 65, d = 2753, n = 3233

Diffie-Hellman key exchange — `diffie_hellman`

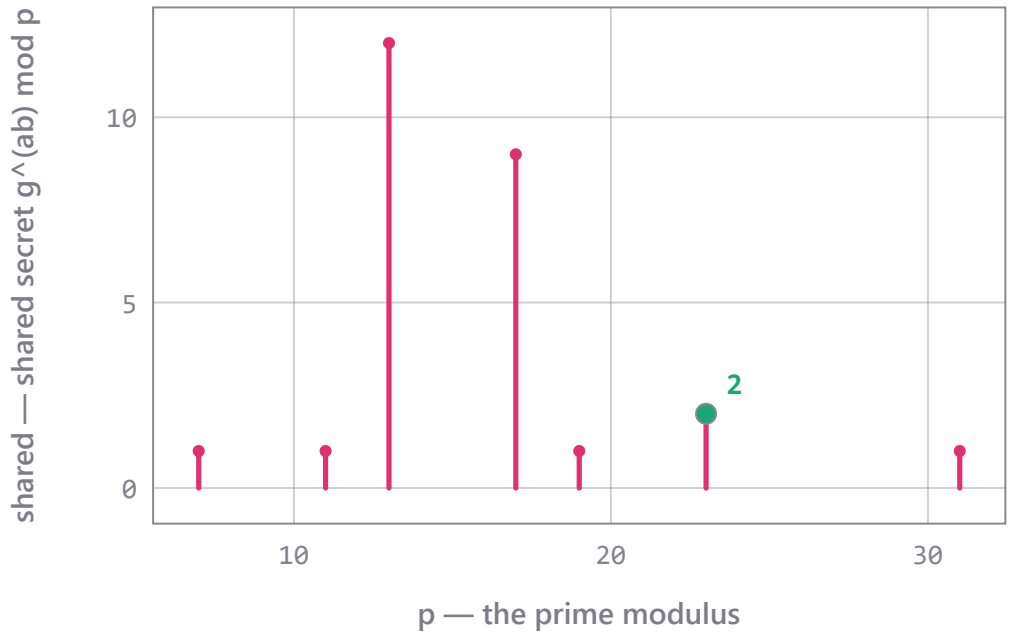
The public values $A = g^a$, $B = g^b$ (mod p) and the shared secret g^{ab} mod p.

$$s = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$$

input	meaning	condition
p	the prime modulus	
g	the generator $(1 < g < p)$	
a	Alice's secret exponent	
b	Bob's secret exponent	

output	meaning
A	Alice's public value g^a mod p
B	Bob's public value g^b mod p
shared	shared secret g^{ab} mod p

- **Method:** modular exponentiation: $B^a = A^b = g^{ab}$
- **Assumes:** p prime; $1 < g < p$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



shared (shared secret g^(ab) mod p) against p (the prime modulus) from 1 to 46. Other inputs as in the worked example: g = 5, a = 6, b = 15. 7 points computed by discrete-math-ai999x; marked: the worked example, p = 23, shared = 2.

```
discrete_math_ai999x.solve('diffie_hellman', p=23, g=5, a=6, b=15)
# -> A = 8, B = 19, shared = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffie-Hellman with p = 23, g = 5, a = 6, b = 15: find the shared secret.	SOLVED	A = 8, B = 19, shared = 2
Diffie-Hellman with p = 24, g = 5, a = 6, b = 15: find the shared secret.	INVALID_INPUT	Diffie-Hellman key exchange: p must be prime.

ElGamal encryption — `elgamal_encrypt`

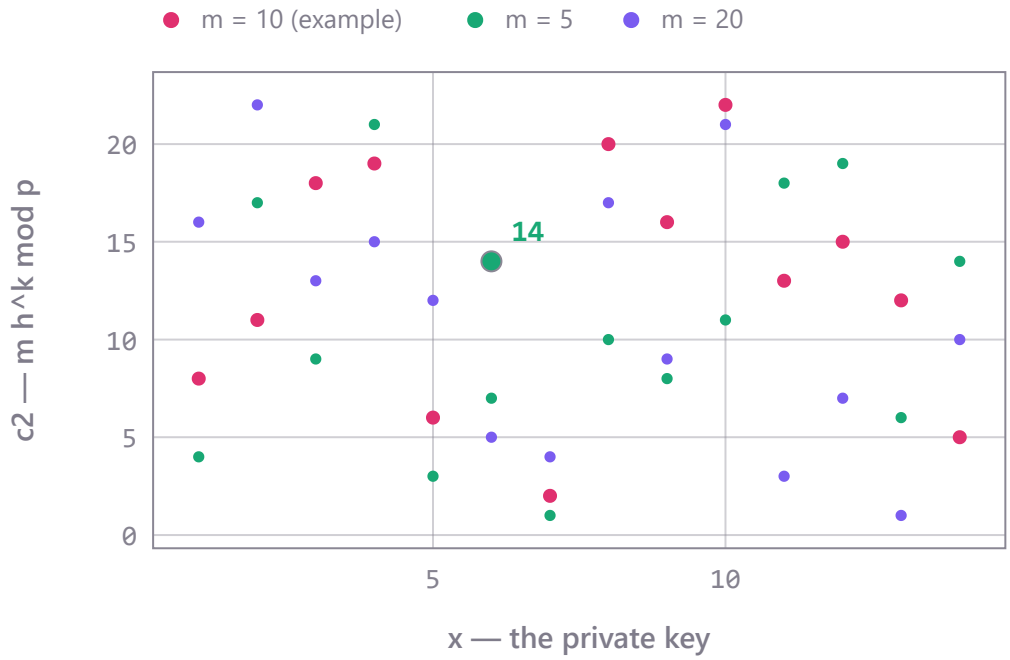
ElGamal ciphertext (c1, c_2) = (g^k, m h^k) mod p with public key $h = g^x$.

$$c_1 = g^k,\ c_2 = m\,h^k \pmod{p}, \quad m = c_2\,c_1^{-x}$$

input	meaning	condition
`p`	the prime modulus	
`g`	the generator	
`x`	the private key	
`k`	the ephemeral exponent	
`m`	the message $(1 \le m < p)$	

output	meaning
`h`	public key g^x mod p
`c1`	g^k mod p
`c2`	m h^k mod p

- **Method:** modular exponentiation
- **Assumes:** p prime; $1 < g < p$; $1 \le m < p$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



c2 (m h^k mod p) against x (the private key) from 1 to 14. One curve for each value of m (the message (1 <= m < p)): 10, 5, 20. Other inputs as in the worked example: p = 23, g = 5, k = 3. 14 points per curve computed by discrete-math-ai999x; marked: the worked example, x = 6, c2 = 14.

```
discrete_math_ai999x.solve('elgamal_encrypt', p=23, g=5, x=6, k=3, m=10)
# -> h = 8, c1 = 10, c2 = 14   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ElGamal: encrypt m = 10 with p = 23, g = 5, x = 6, k = 3.	SOLVED	h = 8, c1 = 10, c2 = 14

Caesar (shift) cipher — `caesar_cipher`

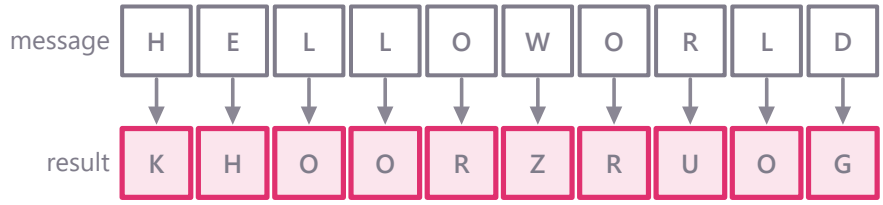
Encrypt or decrypt a message with a Caesar shift of k letters.

$$E(x) = (x + k) \bmod 26$$

input	meaning	condition
`message`	the message (letters are kept, everything else is dropped)	
`shift`	the shift k	
`mode`	encrypt or decrypt	

output	meaning
`result`	the result in capital letters, without spaces or punctuation

- **Method:** each letter x becomes x + k (mod 26); decryption subtracts k
- **Assumes:** letters A-Z only are enciphered; other characters are removed; output is upper case
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated message 'HELLO WORLD' (letters only, top) and the text the package returned (shift = 3, mode = encrypt): 'KHOORZRUOG', letter by letter. Marked: each letter's image.

```
discrete_math_ai999x.solve('caesar_cipher', message='HELLO WORLD', shift=3, mode='encrypt')
# -> result = KHOORZRUOG   (SOLVED)
```

```
discrete_math_ai999x.solve('caesar_cipher', message='khoor', shift=3, mode='decrypt')
# -> result = HELLO   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Encrypt "HELLO WORLD" with a Caesar shift of 3.	SOLVED	result = KHOORZRUOG
Decrypt "KHOOR" with a Caesar shift of 3.	SOLVED	result = HELLO
Encrypt "HELLO" with a Caesar cipher.	NEEDS_INPUT	caesar_cipher needs shift (the shift k). Example: solve('caesar_cipher', message='HELLO WORLD', shift=3, mode='encrypt')

Affine cipher — `affine_cipher`

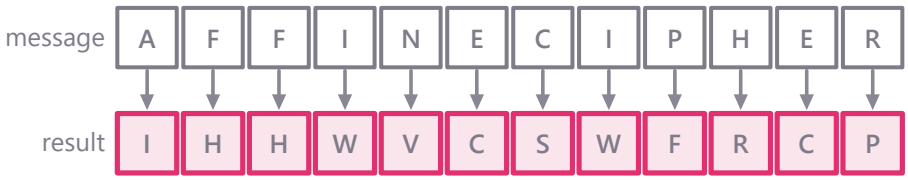
Encrypt or decrypt with the affine cipher E(x) = a x + b (mod 26).

$$E(x) = (ax + b) \bmod 26,\ D(y) = a^{-1}(y - b) \bmod 26$$

input	meaning	condition
`message`	the message (letters are kept, everything else is dropped)	
`a`	multiplier (coprime to 26)	
`b`	shift	
`mode`	encrypt or decrypt	

output	meaning
`result`	the result in capital letters, without spaces or punctuation

- **Method:** E(x) = a x + b, D(y) = a^(-1) (y - b) modulo 26
- **Assumes:** gcd(a, 26) = 1; letters A-Z only; other characters are removed; output is upper case
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated message 'AFFINE CIPHER' (letters only, top) and the text the package returned (a = 5, b = 8, mode = encrypt): 'IHHWVCSWFRCP', letter by letter. Marked: each letter's image.

```
discrete_math_ai999x.solve('affine_cipher', message='AFFINE CIPHER', a=5, b=8, mode='encrypt')
# -> result = IHHWVCSWFRCP   (SOLVED)
```

```
discrete_math_ai999x.solve('affine_cipher', message='IHHWVCSWFRCP', a=5, b=8, mode='decrypt')
# -> result = AFFINECIPHER   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Encrypt "AFFINE CIPHER" with the affine cipher a = 5, b = 8.	SOLVED	result = IHHWVCSWFRCP
Decrypt "IHHWVCSWFRCP" with the affine cipher a = 5, b = 8.	SOLVED	result = AFFINECIPHER
Encrypt "HELLO" with the affine cipher a = 2, b = 3.	INVALID_INPUT	Affine cipher: a must be coprime to 26.

Vigenère cipher — `vigenere_cipher`

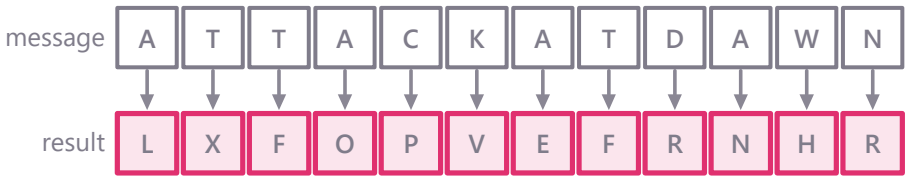
Encrypt or decrypt with the Vigenère cipher under a keyword.

$$c_j = (m_j + k_{j \bmod |k|}) \bmod 26$$

input	meaning	condition
`message`	the message (letters are kept, everything else is dropped)	
`key`	the keyword (letters)	
`mode`	encrypt or decrypt	

output	meaning
`result`	the result in capital letters, without spaces or punctuation

- **Method:** add (encrypt) or subtract (decrypt) the repeating key letters modulo 26
- **Assumes:** letters A-Z only; other characters are removed first, so the key runs over letters only; output is upper case
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The stated message 'ATTACK AT DAWN' (letters only, top) and the text the package returned (key = LEMON, mode = encrypt): 'LXFOPVEFRNHR', letter by letter. Marked: each letter's image.

```
discrete_math_ai999x.solve('vigenere_cipher', message='ATTACK AT DAWN', key='LEMON',
mode='encrypt')
# -> result = LXFOPVEFRNHR   (SOLVED)
```

```
discrete_math_ai999x.solve('vigenere_cipher', message='LXFOPVEFRNHR', key='lemon', mode='decrypt')
# -> result = ATTACKATDAWN   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Encrypt "ATTACK AT DAWN" with the Vigenère cipher and key LEMON.	SOLVED	result = LXFOPVEFRNHR
Decrypt "LXFOPVEFRNHR" with the Vigenere cipher and key LEMON.	SOLVED	result = ATTACKATDAWN